

CCAR-P · Domain 3 · Integration · explainer

# The Dock Walk

The Loading Dock, explained from zero



Its job: take Watson from “I have never heard this word” to “I can read a 3.5 stem and see the trick” — plain language first, small examples second, vocabulary third, exam framing last.

A slow teaching companion to The Loading Dock (41 pp), which stays canon and stays on the calendar. Four rungs per concept · word cards before terms · twelve pencil labs · nothing sealed · nothing to hand-type.

**R1**

THE SCENE · a moment at the dock, zero technical words

**R2**

PLAIN WORDS · the idea in everyday English + one TOY example

**R3**

THE REAL NAME · the word card, then the term, then the canon numbers

**R4**

THE EXAM · how a CCAR-P stem disguises it, the tell, two mini-stems

Kaveri Freight & Fulfilment Pvt. Ltd., Pune · Holmes walks, Watson writes

Read Tue 29 Sep – Fri 2 Oct 2026 (evenings) · fog dictionary from Day A onward

**Prepared by Ravi Ranjan · CCAO-F, CCDV-F & CCAR-F** · Companion to CCAR-P\_D3\_Loading\_Dock.pdf

# How to walk this book

The Loading Dock gives each of the eight Domain 3 objectives about two pages, written for someone who already half-knows the words. Four of those objectives — **3.2** authentication and authorization, **3.4** observability at scale, **3.5** RAG pipeline design, **3.6** retrieval strategy — are new territory: no rung of the ladder (CCAO-F, CCDV-F, CCAR-F) taught them. A stranger needs about ten pages each, written from zero. This book is the ten. It re-teaches *on purpose*. Nothing in it replaces the Dock; everything in it makes the Dock readable.

## The ladder — every concept climbs four rungs, in order, never skipping

| Rung                      | What you get                                                                                                                                | Rule                                                                                                   |
|---------------------------|---------------------------------------------------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------|
| <b>R1 · THE SCENE</b>     | A moment at the dock: people, objects, a small consequence.                                                                                 | Zero technical words. If the scene is missing, the build is defective — ask for it.                    |
| <b>R2 · PLAIN WORDS</b>   | The idea in everyday English plus ONE toy example small enough to do in your head.                                                          | Round numbers, $\leq 12$ items, always labelled <b>TOY</b> so it is never mistaken for a canon figure. |
| <b>R3 · THE REAL NAME</b> | The WORD CARD first, then the technical version, then the canon numbers in <b>gold</b> with their Dock address.                             | A term used before its card is a build defect; the validator checks it.                                |
| <b>R4 · THE EXAM</b>      | How a CCAR-P stem disguises the idea, the tell to <u>underline</u> , the family it baits, two mini-stems with the answer SHAPE written out. | Shapes, not letters — you will never see options here.                                                 |

## The furniture

**Word cards** sit inline at first use (always at R3) and are indexed at the back. Fields, always in this order: plain meaning · what it is NOT · a sentence you would hear at Kaveri · how the exam words it · the Dock glossary entry it expands. There are more cards than the Dock has glossary entries because the Walk needs words the Dock assumed.

**Pencil labs** (P1-P12) are the layman's sabotage labs: no computer, task on the page, worked answer below a **fold line**. Fold the sheet, work with a pencil, unfold. Nothing in this book is sealed — it is a teaching book, not a practice book (the Dock Book is).

**Boxes**, same grammar as the Dock: **TRAP** in crimson · **D3 LAW** in green, born at its station · **ADD IT** where adding the tool, stage, server or retrieval IS the right move (so the trimming reflex does not over-learn) · **DRIFT** where an older workbook or stem may show a reading the current docs have moved past · **CANON ERRATUM** where the Dock's own arithmetic slipped · **WRONG BELIEF, CORRECTED** at every station · **SAME WORD, TWO WORLDS** at every ambush.

**Colour grammar**: **indigo** = tokens and identifiers · **gold** = numbers · **crimson** = traps · **green** = correct moves · **underline** = the tell. **What Watson wrote down** closes every station: the exam-ready formulation in  $\leq 8$  lines, the law in force, and what to read next in the Dock. **Check yourself** follows every R4, answers below a fold line; at least one question per station is “explain this to Meera in two sentences”, because 3.3-grade justification is exactly that skill.

## Where it sits in the calendar — the Dock's day pages remain the schedule of record

| When                          | Read                                     | Station                              | Then                                              |
|-------------------------------|------------------------------------------|--------------------------------------|---------------------------------------------------|
| Tue 29 Sep, evening (~45 min) | Station 1 · Shelving                     | 3.5 RAG pipeline, chunking, indexing | Loading Dock ch. 3.5 — should read like a summary |
| Wed 30 Sep, evening           | Station 2 · The register and the foreman | 3.6 Retrieval by data shape          | Loading Dock ch. 3.6                              |
| Thu 1 Oct, evening            | Station 3 · The gate                     | 3.2 AuthN / authZ gaps               | Loading Dock ch. 3.2                              |

| When                            | Read                           | Station                       | Then                                                         |
|---------------------------------|--------------------------------|-------------------------------|--------------------------------------------------------------|
| Fri 2 Oct, evening              | Station 4 · The camera room    | 3.4 Observability at scale    | Loading Dock ch. 3.4                                         |
| Sat 3 Oct (Day A), first 30 min | Stations 5 and 6 (15 min each) | 3.1 Toolbelt · 3.3 Dial board | each just before its Dock chapter, then type E1-E4           |
| Sun 4 Oct (Day B), first 30 min | Stations 7 and 8 (15 min each) | 3.7 Connector · 3.8 Catalogue | then type E5-E8 (E8 with the ERRATUM box open)               |
| Day A onward                    | The reverse map at the back    | any                           | whenever a Dock sentence is fog, the map names the Walk page |

IF EVENINGS SLIP

Stations **1 (3.5)** and **3 (3.2)** must be read before Day A; Stations **2 (3.6)** and **4 (3.4)** before Day B. The Walk adds evenings and moves nothing.

## Front-matter DRIFT — one of the Dock's four boxes has moved since it was built

UPDATED DRIFT · Prompt-cache minimums by model tier

| Loading Dock / older workbook reading                                                                                                                                                | Current docs (checked 28 Sep 2026)                                                                                                                                                                                                                                                                                             |
|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Minimum cacheable prompt varies by tier, <b>1,024-4,096</b> tokens; <b>Haiku 4.5 = 4,096</b> (not 2,048 as some materials say); cache reads $\approx 90\%$ off the base input price. | Range is now <b>512-4,096</b> : newest tier (Fable / Mythos / Opus 5.x) = <b>512</b> ; Opus 4.5 / 4.6 and <b>Haiku 4.5 = 4,096</b> (unchanged); Sonnet 4.x = <b>1,024</b> ; Opus 4.7 = 2,048. Reads are <b>0.1×</b> ( $\approx 90\%$ off) as the standard rule, with deeper discounts (95-97.5 % off) on the newest tier only. |

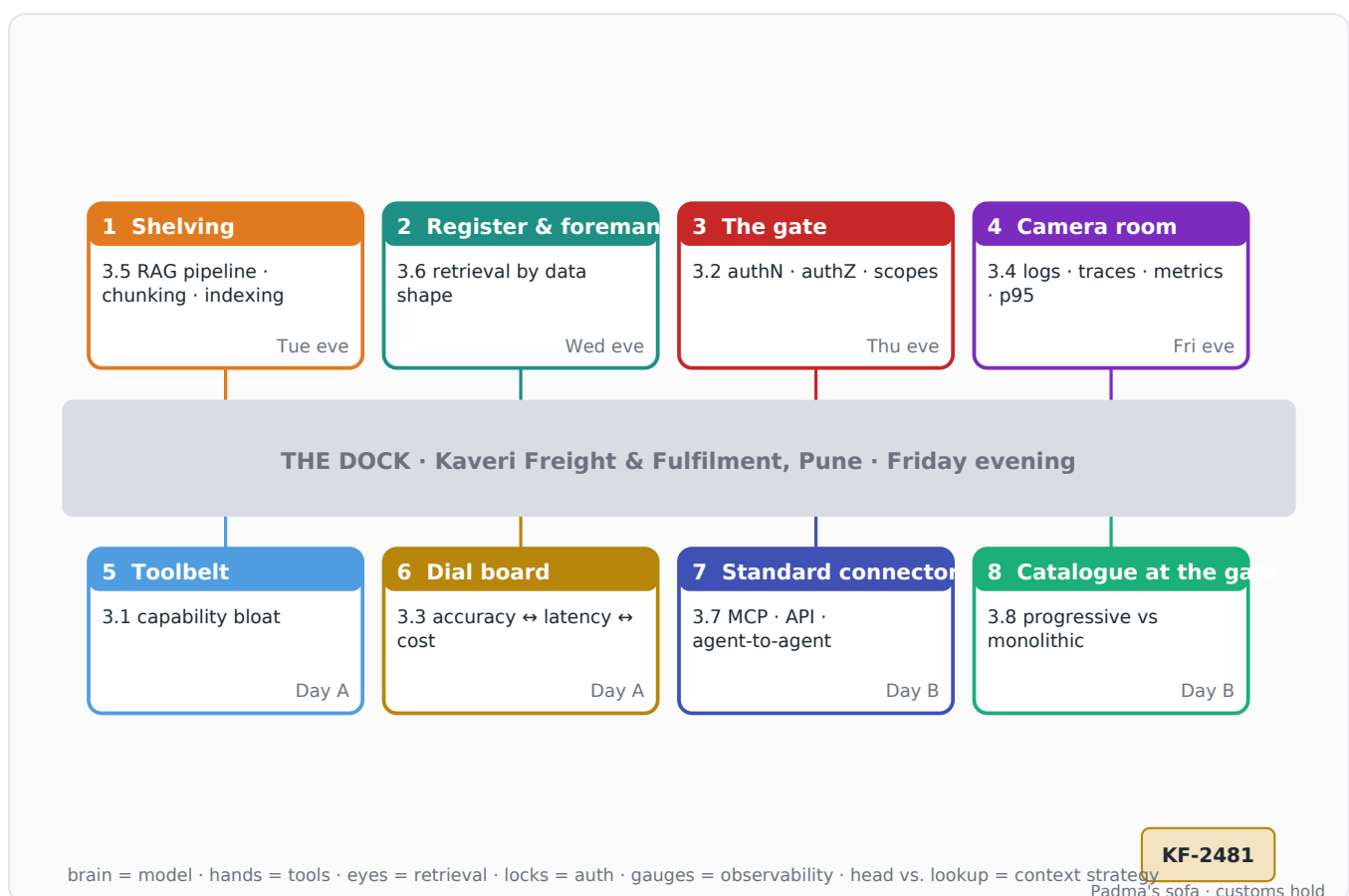
**IN PLAIN WORDS** · What changed: a new top tier appeared with a smaller minimum and a bigger read discount. Why an older workbook or a stem may still show the old reading: the Dock was built from the table as it stood, and exam stems are written months ahead. What to answer: **the minimum varies by model — check the table for the model named; Haiku 4.5 is 4,096; reads are about 90 % off.** A stem that says “cache the 800-token system prompt on Haiku 4.5” is still baiting you: it is under the minimum and will not cache. The Dock's box needs a patch (listed in the delivery note); the other three DRIFT boxes were re-verified on 28 Sep 2026 and hold verbatim.

# Chapter 0 · The nervous system

Friday evening. The tower's shape was approved this afternoon — Meera signed the router-chains-agent drawing and went home. Arjun has a connection week ahead of him and Padma Deshmukh's sofa (consignment [KF-2481](#)) is sitting in a customs hold until the right paragraph of SOP 214 is found and the e-way bill re-filed. Holmes takes Watson round the dock, one station at a time, before anyone plugs anything in. This chapter is the map of the walk.

## “Integration”, in plain words

A brain in a jar is not a worker. Domain 3 is about everything that turns the model into a worker on a real dock: **tools** are its hands (it can press a button, look up an order, issue a refund); **retrieval** is its eyes (it can read the SOP it was never trained on); **authentication and authorization** are the locks (which hands may press which buttons); **observability** is the wall of gauges (you can see what 40,000 shipments a day actually did); **context strategy** is what the worker carries in its head versus what it looks up. Eight objectives, eight stations, one dock.



**W1** · The dock map: eight stations, one objective each. Walk them in the order of the evenings (1 → 2 → 3 → 4), then the short ones on Days A and B. Redraw the map from memory on Day D — it is the skeleton of the Memory Map.

## Why the exam weights this heaviest

Domain 3 is **19 %** of CCAR-P — about **12 of 63** questions, the largest single share. The reason is practical: an architecture that is right on paper dies at the dock. “It worked in the demo” is a sentence about a system with no locks, no gauges, no filing cabinet and a toolbelt full of wrenches it never uses. Four of the eight objectives are new territory for you (3.2, 3.4, 3.5, 3.6); those four get the four evenings and about ten pages each here. The other four (3.1, 3.3, 3.7, 3.8) you half-own from CCAR-F and the D1 Brief; they get fifteen minutes each on Days A and B.

## How a stem is built

Every CCAR-P question is the same machine: a **scenario** (Kaveri-sized: a company, a system, a number or two) → a **constraint** (a latency, a budget, a compliance line, a data shape) → **four options** that all sound competent → one **tell**: the phrase that makes three of them wrong. Reading a stem is finding the tell. At R4 of every concept in this book you will see the same drill: the disguise, the tell to underline, the family it baits, and two mini-stems where you write the answer's *shape* rather than pick a letter. The shape is what transfers to the real room; letters do not.

## Base words the whole walk assumes — seven cards before we set off

These are CCAR-F words. The Walk still cards them, because four of them are ambushes (►) and the exam counts on you mixing them up.

| WORD CARD · token (LLM sense) |                                                                                                                                       |
|-------------------------------|---------------------------------------------------------------------------------------------------------------------------------------|
| PLAIN MEANING                 | A word-piece: the unit the model reads and is billed in — roughly three-quarters of an English word, fewer for Hindi-in-Roman-script. |
| WHAT IT IS NOT                | Not the security pass a caller shows at a gate (Station 3 cards that one as “token (auth sense)”).                                    |
| HEARD AT KAVERI               | “The nine tool schemas alone cost us 2,314 tokens on every single call, Arjun.”                                                       |
| HOW THE EXAM WORDS IT         | “input tokens”, “context window”, “token budget”, “tokens per call × calls per day”.                                                  |
| EXPANDS                       | <i>Loading Dock glossary ‘token (auth sense, disambiguated from LLM token!)’ — this card is the other half of that pair</i>           |

► **SAME WORD, TWO WORLDS · token** — **(1)** LLM sense: a word-piece, counted in thousands, billed. **(2)** auth sense: a pass a caller shows to prove who they are. **The tell**: if it is counted, priced or has a window, it is the LLM sense; if it is issued, expires, or is checked at a door, it is the auth sense.

| WORD CARD · context window |                                                                                                                                               |
|----------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------|
| PLAIN MEANING              | Everything the model can see at once on one call — system prompt, tool schemas, retrieved text, the conversation so far — measured in tokens. |
| WHAT IT IS NOT             | Not the model's memory between calls (there is none unless you send it again) and not “business context” (the situation at Kaveri).           |
| HEARD AT KAVERI            | “Every SOP stuffed into the window is 362,460 tokens before the dispatcher has typed a word.”                                                 |
| HOW THE EXAM WORDS IT      | “context budget”, “fits in a single context”, “under ~200k tokens the whole corpus can be included”.                                          |
| EXPANDS                    | <i>Loading Dock glossary ‘monolithic context’, ‘progressive disclosure’ — Station 8 expands both</i>                                          |

| WORD CARD · tool      |                                                                                                                                                            |
|-----------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------|
| PLAIN MEANING         | A named action the model may ask for — with a written description of what it does and what inputs it takes — that your code then runs and reports back on. |
| WHAT IT IS NOT        | Not something the model runs itself; the model only asks. Your code (or an MCP server) does the doing and enforces the rules.                              |
| HEARD AT KAVERI       | “KAVI has four tools: get_customer, lookup_order, process_refund, escalate_to_human — and process_refund is the one that can cost money.”                  |
| HOW THE EXAM WORDS IT | “tool definition”, “tool schema”, “tool-use loop”, “capability”, “the agent is granted access to...”.                                                      |
| EXPANDS               | <i>Loading Dock ch. 3.1 (capability bloat) · E2 · Station 5 here</i>                                                                                       |

**WORD CARD · system prompt**

|                              |                                                                                                                                                  |
|------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>PLAIN MEANING</b>         | The standing instructions you send with every call: who the assistant is, what it must and must not do, the style to answer in.                  |
| <b>WHAT IT IS NOT</b>        | Not an MCP prompt (Station 7: a reusable template a server offers) and not a control — a firm instruction is persuasion, not a lock (Station 3). |
| <b>HEARD AT KAVERI</b>       | “The system prompt says ‘never issue a refund above ₹5,000’ — and that sentence is exactly what the Refund Heist walked through.”                |
| <b>HOW THE EXAM WORDS IT</b> | “instruct the model to...”, “add to the system prompt that...” — usually the <b>sounds-diligent</b> option when a code-level control exists.     |
| <b>EXPANDS</b>               | <i>Loading Dock glossary ‘MCP prompt’ (the other sense) · D1 Law on config beats instruction</i>                                                 |

**WORD CARD · latency**

|                              |                                                                                                                   |
|------------------------------|-------------------------------------------------------------------------------------------------------------------|
| <b>PLAIN MEANING</b>         | How long the caller waits between asking and getting the answer — the clock a dispatcher watches.                 |
| <b>WHAT IT IS NOT</b>        | Not throughput (how many per hour) and not cost; a slow system can be cheap and a fast one dear.                  |
| <b>HEARD AT KAVERI</b>       | “Dispatchers give up after two seconds, Arjun. Two. Not ‘on average two’.”                                        |
| <b>HOW THE EXAM WORDS IT</b> | “response time”, “p95 latency”, “the dispatcher must see an answer within N seconds”.                             |
| <b>EXPANDS</b>               | <i>Loading Dock glossary ‘p50/p95/p99 latency’ · ch. 3.3 (accuracy-latency trade-offs) · Station 4 and 6 here</i> |

**WORD CARD · MCP (Model Context Protocol)**

|                              |                                                                                                                                                                                |
|------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>PLAIN MEANING</b>         | The standard plug: a published way for a model-side program (the client) to ask a separate program (the server) what tools, documents and templates it offers and to use them. |
| <b>WHAT IT IS NOT</b>        | Not the only way to connect (a direct API call or a CLI is a weld, Station 7) and not a security control by itself.                                                            |
| <b>HEARD AT KAVERI</b>       | “kaveri_mcp.py is our MCP server: two tools, one resource, one prompt, and KAVI talks to it as mcp_kaveri...”                                                                  |
| <b>HOW THE EXAM WORDS IT</b> | “expose the capability as an MCP server”, “mcp_server_tool”, “transport”, “stdio vs Streamable HTTP”.                                                                          |
| <b>EXPANDS</b>               | <i>Loading Dock glossary ‘MCP server / resource / prompt’, ‘transport’ · ch. 3.7 · E1 · Atlas A4 · Station 7 here</i>                                                          |

**WORD CARD · prompt cache**

|                              |                                                                                                                                                                                                  |
|------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>PLAIN MEANING</b>         | The provider remembers an identical prefix of your prompt: you pay a little more once to write it, then about a tenth of the input price to read it on later calls, until it changes or expires. |
| <b>WHAT IT IS NOT</b>        | ► Not a browser cache and not a memory of answers: it stores your prefix, not the model's replies; and it saves nothing on the parts that vary.                                                  |
| <b>HEARD AT KAVERI</b>       | “Schemas + standing instructions + the catalogue = one stable block at the front of every call; cached, it reads at ~10 %.”                                                                      |
| <b>HOW THE EXAM WORDS IT</b> | “prompt caching”, “cache_control breakpoint”, “cache hit / miss”, “stable prefix first, variable content last”.                                                                                  |
| <b>EXPANDS</b>               | <i>Loading Dock DRIFT box ‘cache minimums by tier’ · ch. 3.8 (when monolithic actually wins)</i>                                                                                                 |

## Miss families, slowly — one Kaveri example each

Every wrong option on the exam belongs to a family. Tagging your misses by family is how the ledger tells you what to fix. Domain 3's predicted dominant family is **VG** — because four objectives arrive with forty new words, and a word you half-know is a word the stem can hide the trick behind. That is why this book is mostly cards.

| Family                                                | Plain words                                                               | At Kaveri it looks like                                                                                                                                   | Defence                                                                  |
|-------------------------------------------------------|---------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------|
| <b>VG</b> Vocabulary gap                              | You did not know the word, so you could not see the trick.                | The stem says 'p95' and you read it as 'about 95 % accurate' — it is a latency tail. Wrong option, confidently.                                           | Word cards. Read every R3 card before its term appears in the Dock.      |
| <b>OE</b> Over-engineer                               | You added a stage, a server or a subagent the constraint never asked for. | A reranker on a 40-document SOP store that fits in one context window.                                                                                    | Ask what the constraint demands; the smallest thing that clears it wins. |
| <b>TR</b> Trade-off reversed                          | You optimised the currency the stem did not name.                         | Chose the cheapest model when the stem said 'dispatcher waits $\leq 2$ s and cost is not a concern this quarter'.                                         | Underline the constraint word; it names the currency.                    |
| <b>CM</b> Concept mismatch                            | Right idea, wrong place — a D1 pattern answer to a D3 retrieval question. | 'Add a router' when the question was which chunking strategy retrieves \$4.                                                                               | Name the objective before the option.                                    |
| <b>HL</b> Hallucinated law                            | You applied a rule that does not exist or is reversed.                    | 'Keyword search is deprecated now that embeddings exist.'                                                                                                 | The laws, slowly (back matter). Five D3 laws; nothing else is a law.     |
| <b>SF / SD / OP</b> (CCAR-F families, where they fit) | Silent failure · Sounds diligent · Over-permissive.                       | A carrier token that can read <b>all bays</b> (OP) · 'add a sentence to the prompt' (SD) · a swallowed retrieval error that returns an empty answer (SF). | The Refund Heist reflexes carry over unchanged.                          |

### THE TWO LAWS OF LAWS, CARRIED FROM CCAR-F

**Config beats instruction.** A rule in code or configuration is a control; a rule in a prompt is a request.  
**Valid is not true.** Well-formed output, a well-formed token, a well-formed chunk — none of these is the same as the right one. Both return at Stations 1, 2 and 3 wearing Professional clothes.

## The five D3 laws — a preview; each is born at its own station

| Law                                                                         | Born at                                        | In one line                                                                     |
|-----------------------------------------------------------------------------|------------------------------------------------|---------------------------------------------------------------------------------|
| <b>D3 Law 1</b> · Every tool you add is a door; know who holds the key.     | Station 3 · The gate (3.2)                     | A capability without an owner and a scope is a hole, not a feature.             |
| <b>D3 Law 2</b> · Chunk along meaning, not length.                          | Station 1 · Shelving (3.5)                     | Cut where the document changes subject, not every N words.                      |
| <b>D3 Law 3</b> · Retrieval follows the shape of the data, not the fashion. | Station 2 · The register and the foreman (3.6) | An id wants a lookup, a count wants a query, a policy wants a search.           |
| <b>D3 Law 4</b> · You cannot fix what you cannot see.                       | Station 4 · The camera room (3.4)              | A system with no gauges fails silently and is repaired by guessing.             |
| <b>D3 Law 5</b> · Send the map first, the territory on demand.              | Station 8 · The catalogue at the gate (3.8)    | Small, stable, always-used things ride in the head; everything else is fetched. |

 **WHAT WATSON WROTE DOWN**

- Domain 3 = **19 %**  $\approx$  **12 of 63**; four objectives new (3.2, 3.4, 3.5, 3.6); predicted dominant miss family **VG**.
- A stem = scenario  $\rightarrow$  constraint  $\rightarrow$  four options  $\rightarrow$  the tell. Find the tell; write the shape.
-  token: counted = LLM sense; issued/checked = auth sense.
- A tool is a request the model makes; the doing and the rules live in code.
- Two laws of laws: config beats instruction · valid is not true.

**Law in force:** none yet — the first D3 law is born at Station 1

**Read next in the Loading Dock:** Loading Dock ch. 0 / the story page · the five-laws box on the Memory Map

## TONIGHT YOU WILL

## Tuesday 29 Sep · ~45 min · Station 1 · Shelving

- ☐ see why a brilliant new hire with no filing cabinet answers Padma wrongly, fluently
- ☐ cut a 300-word TOY SOP three ways with a pencil and watch one way hand Padma the cancellation steps (P1)
- ☐ place eight Kaveri phrases on a meaning map and read off nearest neighbours — no maths beyond 'same direction' (P2)
- ☐ own the words: RAG, chunking (four kinds), embedding, vector index, top\_k, contextual retrieval, reranking, recall / precision, context poisoning
- ☐ carry the canon ladder **5.7 → 3.7 → 2.9 → 1.9 %**, **~150 → top-20**, **≤ 200k tokens**

**Then:** read Loading Dock ch. 3.5. It should read like a summary. Star anything still foggy; the Walk page is in the margin of the reverse map.

## STATION 1

## Shelving

Design a RAG pipeline with appropriate chunking and indexing strategies — the sofa, sawn into six boxes

## OBJECTIVE

## 3.5

expands Loading Dock ch. 3.5 · E4-E5 · Atlas A1-A2 · glossary (chunking, embedding, vector index, reranking, top\_k, recall/precision, context poisoning)

## Climb 1 · A brilliant new hire with no filing cabinet

## R1

## THE SCENE

## The scene — Padma asks, the new hire answers, and it is wrong

Friday, 6.40 pm. Padma Deshmukh rings the dispatch desk for the third time: her sofa, [KF-2481](#), is sitting in a customs hold at the Pune bonded yard, and the officer said something about an e-way bill that has to be filed again. The new hire on the desk — brilliant, polite, read every logistics textbook ever printed — answers at once and at length. He explains e-way bills in general. He is fluent. He is confident. He is also describing the rules of a company he has never worked for, because **nobody has ever shown him Kaveri's own SOP 214**. Padma hangs up no closer to her sofa.

Meera, listening from the next desk: "He's not wrong because he's stupid. He's wrong because we never gave him the cabinet."

## R2

## PLAIN WORDS

## Plain words — give the hire a clerk and a cabinet

The model is that hire. It knows the world in general and nothing about Kaveri in particular, because Kaveri's documents were written after it was trained and were never public. Two ways to fix that: (a) read every document to it before every question, or (b) give it a **clerk** who, for each question, runs to the cabinet, fetches the one paragraph that matters, and lays it on the desk before the hire answers. Option (a) is slow and expensive for every question, and above a certain size it does not fit at all. Option (b) is what this whole station is about: the cabinet, the clerk, and how to shelve so the clerk finds the right paragraph.

## TOY

TOY: a desk drawer holds 12 cards, one per SOP section. The clerk's whole job is: read the question, pull one card, put it on the desk. If the cards are cut badly — half a procedure on one card, half on the next — the clerk pulls a card that is *about* the right thing and still wrong.

The dock has two rhythms. Shelving happens **weekly** (the SOPs change every week — Kaveri's **400** SOPs, heading-structured, Hindi-English mixed). Fetching happens about **40,000** times a day (every shipment, every dispatcher question). Anything that costs a little per fetch costs a lot per day; anything that costs a lot per shelf is paid once a week. Keep those two rhythms apart in your head — every design choice at this station belongs to one of them.

### A · the brilliant new hire, no filing cabinet



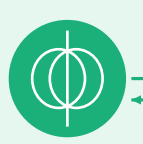
the model

Asked: "What is the e-way bill re-filing step for a customs hold?"

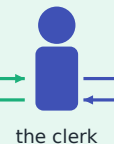
Answers fluently — from what it read years ago, about somebody else's company. Sounds right. Is not SOP 214 §4.

fluent ≠ correct · valid is not true

### B · the same hire, plus a clerk and a cabinet



the model



the clerk  
(retrieval)

SOP 214 §1-3  
SOP 214 §4 ✓  
SOP 214 §5-6  
SOP 215...

400 SOPs, shelved

ask → fetch §4 → hand it over → answer from it, and cite it

**W2** · The brilliant new hire without and with a cabinet. On the left the answer is fluent and about the wrong company. On the right the clerk fetches SOP 214 §4, the hire answers from it and says which paragraph it came from. Redraw this from memory on Day D as the two ends of Atlas A1.

## R3

THE REAL  
NAME

## The real name — RAG, retrieval, the two lanes, and the skip threshold

### WORD CARD · RAG (retrieval-augmented generation)

|                       |                                                                                                                                                                     |
|-----------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| PLAIN MEANING         | Answering with the model after first fetching the relevant pieces of your own documents and placing them in the prompt — generation that is augmented by retrieval. |
| WHAT IT IS NOT        | Not fine-tuning (teaching the model new habits) and not memory (nothing is kept between calls); the documents are looked up fresh each time.                        |
| HEARD AT KAVERI       | "KAVI doesn't know SOP 214 — put a RAG pipeline in front of it so it reads §4 before it answers."                                                                   |
| HOW THE EXAM WORDS IT | "ground the answer in internal documents", "knowledge base that changes weekly", "design a retrieval pipeline".                                                     |
| EXPANDS               | <i>Loading Dock ch. 3.5 · Atlas A1 (the pipeline end to end)</i>                                                                                                    |

### WORD CARD · retrieval

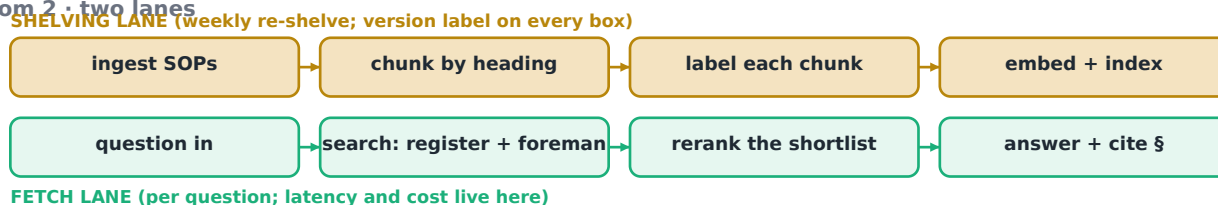
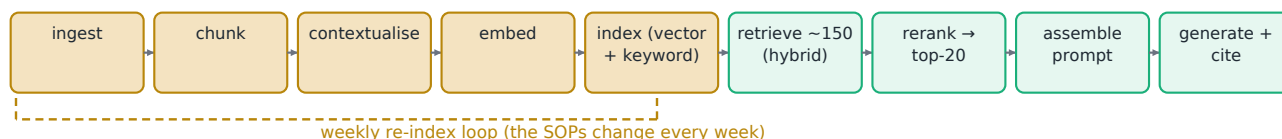
|                       |                                                                                                        |
|-----------------------|--------------------------------------------------------------------------------------------------------|
| PLAIN MEANING         | The clerk's run: given a question, find and return the few passages most likely to contain the answer. |
| WHAT IT IS NOT        | Not the answer itself — retrieval hands over paragraphs; the model still has to read them and write.   |
| HEARD AT KAVERI       | "Retrieval brought back the cancellation paragraph, so the answer was a fluent cancellation."          |
| HOW THE EXAM WORDS IT | "retrieve the top-k chunks", "retrieval failure", "the retrieved context did not contain..."           |
| EXPANDS               | <i>Loading Dock glossary 'top_k', 'recall vs precision (retrieval sense)'</i>                          |

**WORD CARD · pipeline (RAG sense)**

|                              |                                                                                                                                             |
|------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------|
| <b>PLAIN MEANING</b>         | The fixed sequence of steps a document goes through to be shelved, and a question goes through to be answered — drawn as boxes with arrows. |
| <b>WHAT IT IS NOT</b>        | Not a single program: the shelving lane and the fetch lane run at different times, on different rhythms, and are tuned separately.          |
| <b>HEARD AT KAVERI</b>       | “The shelving lane runs Sunday night; the fetch lane runs 40,000 times a day — don't tune them as one thing.”                               |
| <b>HOW THE EXAM WORDS IT</b> | “ingestion pipeline”, “query-time pipeline”, “re-index on a schedule”.                                                                      |
| <b>EXPANDS</b>               | <i>Loading Dock Atlas A1 (weekly re-index loop drawn)</i>                                                                                   |

Now the technical version. A RAG pipeline has a **shelving lane** (ingest → cut → label → file by meaning) that runs when documents change, and a **fetch lane** (question → search → shortlist → assemble the prompt → generate → cite) that runs per question. Everything you pay per fetch is multiplied by **40,000** a day; everything you pay per shelf is multiplied by one week.

One canon number decides whether you need any of this: Anthropic's guidance says that below roughly **200,000 tokens** of documents (about 500 pages) you can put the whole collection into the prompt and skip RAG entirely — and with prompt caching that can be the cheaper, more accurate choice. Kaveri's 400 SOPs are far above that. A 30-page carrier contract is far below it. *Loading Dock ch. 3.5 · DRIFT box ‘contextual retrieval’ · Station 8 here for the caching half.*

**Zoom 1 · two boxes****Zoom 2 · two lanes****Zoom 3 · the full pipeline (Atlas A1's slow ancestor)**

**W5** · The same pipeline at three zoom levels. Zoom 1 is the whole idea; Zoom 2 is the two lanes and their rhythms; Zoom 3 is the slow ancestor of Atlas A1 with the weekly re-index loop. Redraw Zoom 3 from memory on Day D — it is A1.

**ADD IT · the whole RAG pipeline (adding is the right move here)**

A stem describes 400 weekly-changing, heading-structured SOPs and a dispatcher who needs the exact re-filing steps. Adding a retrieval pipeline here is **the right move**, not over-engineering; the corpus is far above the ~200k-token skip threshold and changes weekly, so pasting everything into the prompt is both too big and stale by Tuesday. The OE ghost is not ‘you added RAG’; it is ‘you added RAG to a 30-page contract that fits in one window’.

**R4****THE EXAM****The exam — how a 3.5 stem hides ‘do you even need a cabinet?’**

**The disguise.** The stem gives you a corpus size, an update rhythm and a question style, then offers four pipelines. Three are competent; one matches the size and rhythm. The tell is usually a number (pages, tokens, update frequency) that decides RAG versus whole-in-context, and a rhythm word (weekly, hourly, static) that decides how the shelving lane is built.

**Family baited:** OE (adding a pipeline to a corpus that fits in the window) and its mirror HL (“RAG is always more accurate than stuffing the context”).

**Mini-stem 1.** A regional carrier keeps 22 pages of rate cards that change quarterly; dispatchers ask price questions all day; latency must stay under two seconds. Which design? — **Shape: no RAG: the whole 22 pages ride in the prompt, cached; one call, no fetch lane.** Under the threshold, stable, always used.

**Mini-stem 2.** Kaveri's 400 SOPs, heading-structured, updated every week, queried 40,000 times a day about specific procedures. — **Shape: a two-lane RAG pipeline with a weekly re-shelve and per-question retrieval of a few passages;** the design questions that follow (how to cut, how to file) are the rest of this station.

**CHECK YOURSELF**

1. Why was the new hire wrong even though he was fluent?
2. Name the two lanes and the rhythm of each.
3. Below what corpus size does the guidance say to skip RAG, and why is that not a defeat?
4. Explain to Meera in two sentences why KF-2481 got a wrong answer.

----- ∞ fold — answers below -----

1. He never had Kaveri's own document; he answered from general knowledge about somebody else's rules. Fluent ≠ correct — valid is not true.
2. Shelving lane (ingest → cut → label → file by meaning), weekly; fetch lane (question → search → shortlist → prompt → answer), ~40,000 times a day.
3. Below ~200,000 tokens (~500 pages). Whole-in-context is often more accurate and, cached, cheaper — the pipeline exists for corpora that do not fit or change often.
4. KAVI never saw SOP 214 — it answered from general knowledge. We need a clerk that fetches the exact paragraph before KAVI speaks, and shelves cut so the clerk can find it.

**Climb 2 · The sofa, sawn into six boxes — how you cut decides what you find****R1****THE SCENE****The scene — one sofa, three ways to box it**

Saturday morning at the shelving station. Padma's sofa has to go onto the racks in pieces because no rack takes a whole sofa. The old foreman — retired, still turns up, explains but never decides — watches three crews box it. Crew A puts the whole thing in one enormous crate: nothing is lost, and nobody can lift it. Crew B saws every 40 cm regardless of what they are cutting through: six neat identical boxes, and the left armrest is half in box three, half in box four. Crew C cuts at the joints — seat, back, two arms, two legs: six boxes of different sizes, each one a whole part.

Monday, Padma's repair man asks for ‘the left armrest’. Crew A hands him the crate. Crew B hands him box three — which is mostly the seat with a stub of arm. Crew C hands him the arm. Same sofa. Three answers.

**R2****PLAIN WORDS****Plain words — cut where the thing changes, not every N centimetres**

A document is the sofa. The racks are the cabinet. You cut because a whole document is too heavy to fetch for every question, and because a question wants one part, not the whole. The only question that matters is: **after cutting, does each piece still hold one complete idea?** Cutting by length (every 200 words) is easy to do and easy to get wrong, because documents do not change subject every 200 words. Cutting by structure (every heading) follows the author's own joints. Sometimes you leave a little overlap at each cut so a sentence that straddles a joint is whole on at least one side.

**TOY**

TOY: a 12-line recipe. Cut every 4 lines and step 5 ('bake 40 min') lands at the top of piece 2, separated from 'preheat to 180°' at the bottom of piece 1. Cut at the headings — Ingredients / Prep / Bake — and each piece is a complete stage. Ask 'how long do I bake it?' and only the second cut answers in one piece.

That is exactly what happened to KF-2481. SOP 214 was first cut every fixed number of words. The question 'how do I re-file the e-way bill after a customs hold?' pulled the piece that *started* with the end of §3 (cancellation) and held only the first line of §4. The answer came back as a fluent set of cancellation steps. Re-cut at the headings and the same question pulls §4 whole.

**A · one box (whole SOP)**

SOP 214 — Customs holds. §1 scope · §2 documents · §3 cancellation · §4 e-way bill re-filing · §5 escalation · §6 fees

too heavy to fetch; carries everything

**B · every 200 words (fixed-size)**

§1 + half of §2

rest of §2 + §3 start

§3 cancellation steps + first line of §4

rest of §4 ≠ cut mid-list

§5

§6

§4 query → box 3 → cancellation steps

**C · per section (heading-aware)**

§1 scope

§2 documents

§3 cancellation

§4 e-way bill re-filing ✓ whole procedure

§5 escalation

§6 fees

§4 query → box 4 → the whole procedure

**W3** · One SOP boxed three ways. A: one crate — complete, unliftable. B: sawn every 200 words — the §4 question fetches box 3, the cancellation steps. C: one box per heading — the §4 question fetches the whole re-filing procedure. Redraw B and C from memory on Day D; they are the two ends of Atlas A2.

**R3****THE REAL NAME****The real name — chunking, four kinds, and D3 Law 2****WORD CARD · chunking**

|                              |                                                                                                                                                   |
|------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>PLAIN MEANING</b>         | Cutting a long document into pieces (chunks) small enough to shelf, fetch and place in a prompt one at a time.                                    |
| <b>WHAT IT IS NOT</b>        | Not summarising — nothing is lost, only divided; and not the same as splitting a file for storage, because the pieces must each make sense alone. |
| <b>HEARD AT KAVERI</b>       | "Chunk SOP 214 at the headings, Arjun — the fixed-size cut is why Padma got the cancellation steps."                                              |
| <b>HOW THE EXAM WORDS IT</b> | "select an appropriate chunking strategy", "chunk size", "chunk boundaries", "heading-aware".                                                     |
| <b>EXPANDS</b>               | <i>Loading Dock glossary 'chunking (fixed / semantic / heading-aware / overlap)' · ch. 3.5 · E4 · Atlas A2</i>                                    |

| Kind                              | How it cuts                                                         | When it is right                                                    | When it hurts                                                            |
|-----------------------------------|---------------------------------------------------------------------|---------------------------------------------------------------------|--------------------------------------------------------------------------|
| <b>fixed-size</b>                 | every N tokens (say 200-500), regardless of content                 | flat text with no structure (logs, transcripts); a quick first pass | <b>cuts through procedures</b> — the KF-2481 failure                     |
| <b>heading-aware</b> (structural) | at the author's headings / sections; long sections sub-cut          | heading-structured documents — Kaveri's SOPs, contracts, manuals    | documents with no headings; sections of wildly different size            |
| <b>overlap</b>                    | any strategy, plus a repeated tail (say 10-20 %) at each cut        | when sentences straddle boundaries; belt-and-braces for fixed-size  | more chunks, more storage, near-duplicate hits crowd the shortlist       |
| <b>semantic</b>                   | cut where the meaning shifts, found by comparing adjacent sentences | unstructured prose with real topic changes                          | cost at shelving time; unpredictable sizes; overkill when headings exist |

Chunk size is a dial, not a law: a few hundred tokens is the usual band (Anthropic's contextual-retrieval guidance chunks at a few hundred tokens and adds ~**50-100** tokens of label — Climb 4). Too small and a chunk has no context; too big and the prompt fills with the wrong part of the right section. The Dock's E4 shows the exact wrong answer fixed-size gives for the e-way bill question and the right one heading-aware gives — when you type it on Day A, watch the line that prints which chunk was retrieved.

### D3 LAW 2 · Chunk along meaning, not length.

Born here. Cut where the document changes subject — at the author's joints — not every N words. The test of a chunk is whether it holds one complete idea a reader could act on alone. Broken: every 200 words through SOP 214, so §4's procedure straddles two chunks and the question fetches the wrong half.

### TRAP · 'Smaller chunks are more precise'

Smaller chunks are more precise about *where* they came from, and less useful about *what* they say. Cut §4's six-step procedure into six one-step chunks and the question 'what are the steps?' fetches one step, fluently. The **trap** is choosing a size by a rule of thumb instead of by the unit a question actually wants. 'procedure', 'steps', 'section' in a stem are the tell that the unit is a section.

## R4

### THE EXAM

### The exam — how a stem disguises 'which cut?'

**The disguise.** The stem describes the document's shape (headings? tables? weekly updates? Hindi-English?) and the question's grain ('the exact steps', 'which section says'), then offers four chunkers. The tell is the document-shape word — heading-structured, sectioned, procedural — which points at structural chunking; flat or transcript points at fixed-size with overlap.

**Family baited:** HL ("smaller is always better", "semantic chunking is always best") and OE (semantic chunking on a document that already has headings).

**Mini-stem 1.** A manual of 120 numbered procedures, each under a heading, each three to eight steps; users ask for 'the steps for X'. Fixed-size 256-token chunks are retrieving partial procedures. Fix? — **Shape: re-chunk at the headings so one chunk = one procedure; sub-cut only sections that exceed the size band; keep the heading text inside each chunk.**

**Mini-stem 2.** Six months of dispatcher chat transcripts, no structure, asked 'what did we tell customers about X?' — **Shape: fixed-size chunks with overlap** (there are no joints to cut at); semantic chunking only if the transcripts have real topic shifts worth paying for at shelving time.

**PENCIL LAB P1 · Chunk a 300-word TOY SOP three ways**

Below is a TOY SOP (all numbers invented). Take a pencil. **(a)** Cut it every 60 words — draw a line after every 60th word (count roughly; ‘about 60’ is fine). **(b)** Cut it at the headings. **(c)** Cut it at the headings but repeat the last sentence of each section at the top of the next (overlap). Then answer: which cut leaves the *whole* re-filing procedure inside one chunk? For the question ‘what do I do to re-file after a hold?’, which cut hands Padma the cancellation steps?

**TOY SOP 9 · Customs holds (300 words, invented)**

**§1 Coverage.** This procedure applies to any consignment stopped at a bonded yard by a customs officer. It covers documents, cancellation, re-filing and escalation. It does not cover damaged goods, which are handled under TOY SOP 12. Dispatchers must log the hold in the register within one hour and inform the customer the same day.

**§2 Documents.** Collect the invoice, the packing list, the transporter copy and the original e-way bill number. Check that the vehicle number on the bill matches the truck at the yard. If the customer's GST number is missing, request it before proceeding.

**§3 Cancellation.** If the goods will not move within twenty-four hours of the bill's generation, cancel the bill on the portal. Select the reason ‘others’ and enter the hold reference. Cancellation is not possible after twenty-four hours; in that case proceed directly to §4.

**§4 Re-filing.** To re-file after a hold: (1) generate a fresh bill using the original invoice; (2) enter the bonded yard as the dispatch-from place; (3) enter the hold reference in the remarks; (4) attach the officer's release note; (5) send the new bill number to the transporter; (6) update the register and the customer.

**§5 Escalation.** If the officer does not issue a release note within two working days, escalate to the customs desk with the hold reference and all documents from §2.

----- ✕ fold here — the worked answer is below the line -----

**WORKED ANSWER · P1**

**(a) Every 60 words** gives five chunks. Chunk 3 starts inside §2's last sentence, holds all of §3 and the first words of §4 (‘To re-file after a hold: (1) generate a fresh bill...’). Chunk 4 holds steps (2)–(6) and the start of §5. So the phrase ‘*re-file after a hold*’ lives in chunk 3 — the chunk that is mostly **cancellation**. That is the chunk the question fetches. Padma gets §3.

**(b) At the headings** gives five chunks, one per section. §4 is one chunk holding all six steps. The question fetches it whole. **This is the cut that answers.**

**(c) Headings + overlap** gives the same five chunks, each starting with the last sentence of the previous section. §4's chunk now begins ‘Cancellation is not possible after twenty-four hours; in that case proceed directly to §4.’ — which is actually helpful: a question about a two-day-old hold now lands on a chunk that says both ‘don't cancel’ and ‘re-file like this’. Cost: five slightly larger chunks and a §3/§4 sentence that appears twice.

**What you should feel:** the length cut did nothing wrong by its own rule and still handed over the wrong procedure. D3 Law 2 is the rule that would have stopped it. On Day A, E4 prints the same failure with real code — watch the line that names the retrieved chunk.

**CHECK YOURSELF**

1. Name the four chunking kinds and the document shape each suits.
2. What exactly went wrong with KF-2481's first answer?
3. Which word in a stem tells you the unit a question wants is a section?
4. Explain D3 Law 2 to Meera in two sentences.

----- ✕ fold — answers below -----

1. Fixed-size — flat text; heading-aware — structured documents; overlap — straddling sentences, added to another strategy; semantic — unstructured prose with real topic shifts.
2. SOP 214 was cut by length; §4's procedure straddled two chunks; the question fetched the chunk that was mostly §3 (cancellation), and the model answered fluently from it.
3. 'procedure', 'steps', 'section' — the tell that one chunk must hold one whole section.
4. We cut documents where they change subject, not every so-many words. That way the piece the system fetches is a whole procedure, not the tail of one and the head of the next.

## Climb 3 · The meaning map — filing by what a thing is about

### R1

#### THE SCENE

### The scene — the foreman's wall

The old foreman never used the alphabetical register. He had a wall. Every consignment note got a pin, and he pinned things that were *about the same thing* near each other — customs paperwork in the top-right, refunds bottom-left, late trucks top-left — without ever writing down a rule for it. New note comes in: 'officer wants the bill filed again after the hold.' He does not look for the words; he looks at where the note *belongs*, pins it next to 'customs hold release', and everything a dispatcher needs is already within arm's reach of that pin.

Watson: "But what are the axes of the wall?" Foreman: "Nobody knows, son. There are hundreds of them. You only ever need to know what's *near*."

### R2

#### PLAIN WORDS

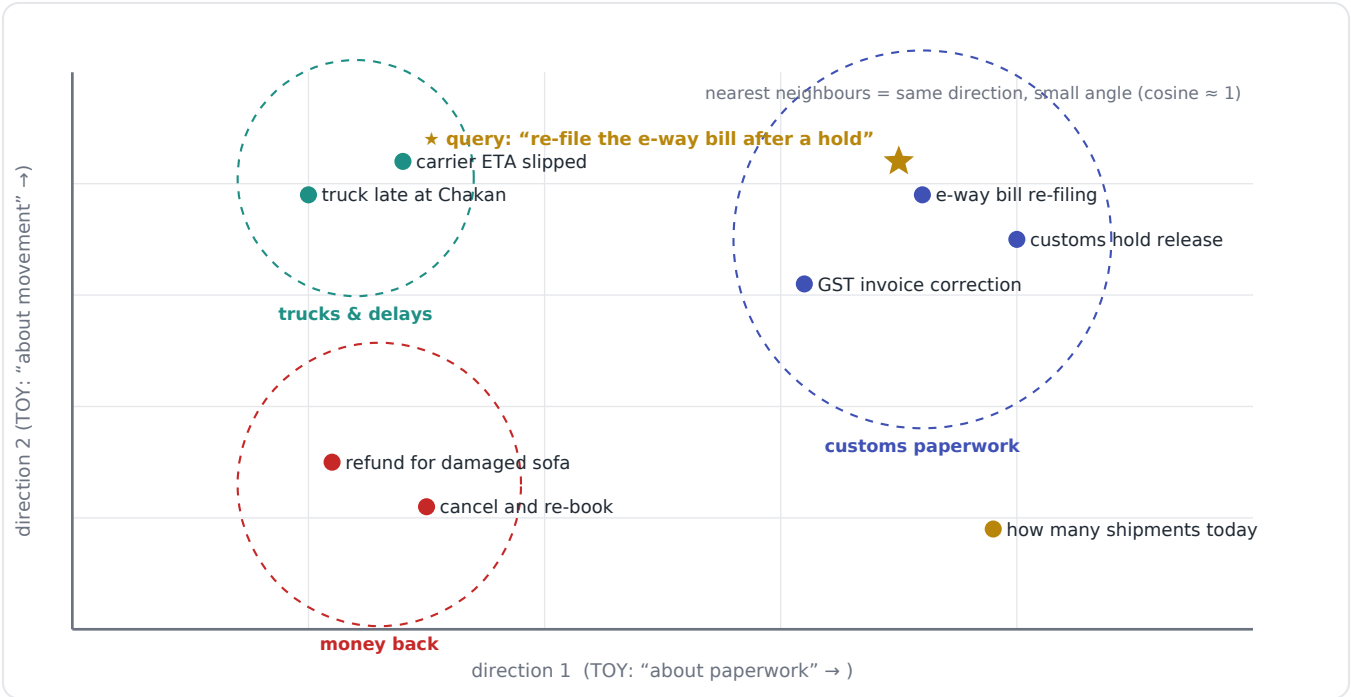
### Plain words — turn every sentence into a position, then look for neighbours

Suppose you could give every sentence a spot on a map, so that sentences about the same thing land close together and sentences about different things land far apart. Then 'find the paragraph that answers this question' becomes 'put the question on the map and see what is nearest'. No word-matching at all: 're-file after a hold' and 'generate a fresh e-way bill following release' share no important words and land at the same spot because they *mean* the same thing.

#### TOY

TOY: a 2-D map. Direction 1 is 'how much is this about paperwork?', direction 2 'how much about physical movement?'. 'Truck late at Chakan' → low paperwork, high movement. 'GST invoice correction' → high paperwork, low movement. 'Re-file the e-way bill after a hold' → high paperwork, medium movement — right next to 'customs hold release'. Nearness is measured as 'pointing the same way from the origin': two pins in the same direction are neighbours even if one is further out.

The real map has hundreds or thousands of directions, not two, and nobody names them; a trained model assigns the positions. The one idea that survives from the toy is the only one you need: **same direction = same meaning**.



**W4** · The meaning map (TOY, two directions). Eight Kaveri phrases pinned by what they are about; three clusters emerge without anyone naming a rule. The gold star is the query; its nearest neighbours are the customs-paperwork cluster. The real map has thousands of directions — you never see them, you only ever ask what is near.

R3

THE REAL NAME

### The real name — embedding, cosine similarity, nearest neighbour, vector index, top\_k

| WORD CARD · embedding |                                                                                                                                                                        |
|-----------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| PLAIN MEANING         | A list of numbers that is a sentence's position on the meaning map — produced by a model trained so that similar meanings get similar positions.                       |
| WHAT IT IS NOT        | Not an understanding of the sentence and not a summary; it is a coordinate, and coordinates can be close for the wrong reasons (same style, same length, same domain). |
| HEARD AT KAVERI       | "Embed every chunk once at shelving time; embed the question at fetch time; compare positions."                                                                        |
| HOW THE EXAM WORDS IT | "vector representation", "dense retrieval", "semantic similarity", "embedding model".                                                                                  |
| EXPANDS               | Loading Dock glossary 'embedding' · E5 (stub embedder in DRY_RUN)                                                                                                      |

| WORD CARD · cosine similarity |                                                                                                                                                               |
|-------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------|
| PLAIN MEANING                 | The nearness score between two positions on the map: 1 means pointing exactly the same way, 0 means unrelated directions.                                     |
| WHAT IT IS NOT                | Not a distance in metres — it ignores how far out a pin sits and measures only direction, which is why a short question and a long paragraph can still match. |
| HEARD AT KAVERI               | "The \$4 chunk scored 0.91 against the question; the \$3 chunk 0.84 — close, which is why we also want the register."                                         |
| HOW THE EXAM WORDS IT         | "similarity score", "similarity threshold", "nearest neighbours by cosine".                                                                                   |
| EXPANDS                       | Loading Dock glossary 'embedding' (the comparison half)                                                                                                       |

**WORD CARD · nearest neighbour**

|                              |                                                                                                                                    |
|------------------------------|------------------------------------------------------------------------------------------------------------------------------------|
| <b>PLAIN MEANING</b>         | The chunk whose position is closest to the question's position — the pin nearest the star.                                         |
| <b>WHAT IT IS NOT</b>        | Not necessarily the best answer: it is the nearest <i>by direction</i> , which is what the map can see, not what the reader needs. |
| <b>HEARD AT KAVERI</b>       | "Top three neighbours for the hold question: §4, §3, and a 2019 memo about hold fees."                                             |
| <b>HOW THE EXAM WORDS IT</b> | "approximate nearest neighbour (ANN)", "k-NN search", "return the k closest vectors".                                              |
| <b>EXPANDS</b>               | <i>Loading Dock glossary 'vector index', 'top_k'</i>                                                                               |

**WORD CARD · vector index**

|                              |                                                                                                                                                                  |
|------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>PLAIN MEANING</b>         | The store that keeps every chunk's position and can answer 'which positions are nearest this one?' fast, even with millions of chunks.                           |
| <b>WHAT IT IS NOT</b>        | Not the index at the back of a book (that is the keyword register, Station 2) and not the documents themselves — it holds positions and pointers back to chunks. |
| <b>HEARD AT KAVERI</b>       | "Rebuild the vector index every Sunday night when the SOPs change; label each entry with its SOP version."                                                       |
| <b>HOW THE EXAM WORDS IT</b> | "vector store", "vector database", "ANN index", "re-index on update".                                                                                            |
| <b>EXPANDS</b>               | <i>Loading Dock glossary 'vector index' · Atlas A1 (the index box)</i>                                                                                           |

**WORD CARD · top\_k**

|                              |                                                                                                                               |
|------------------------------|-------------------------------------------------------------------------------------------------------------------------------|
| <b>PLAIN MEANING</b>         | How many chunks the clerk brings back from the cabinet — the length of the shortlist.                                         |
| <b>WHAT IT IS NOT</b>        | Not how many the model reads: a large first shortlist is usually cut down again (Climb 4) before anything reaches the prompt. |
| <b>HEARD AT KAVERI</b>       | "k=3 gave us §4, §3 and the fees memo; k=20 then rerank gives us §4 first almost every time."                                 |
| <b>HOW THE EXAM WORDS IT</b> | "retrieve the top-k", "k too small misses; k too large dilutes", "~150 candidates reranked to top-20".                        |
| <b>EXPANDS</b>               | <i>Loading Dock glossary 'top_k'</i>                                                                                          |

The technical sentence: at shelving time every chunk is embedded and its vector stored in a vector index; at fetch time the question is embedded, the index returns the top\_k nearest neighbours by cosine similarity, and those chunks go to the next stage. The Dock's E5 does exactly this with a stub embedder so that the free probe runs before any paid call.

► **SAME WORD, TWO WORLDS · index** — **(1)** book sense: an alphabetical list of words with page numbers — the register (keyword search, Station 2). **(2)** vector sense: a store of positions that answers 'what is nearest?'. **The tell:** 'keyword', 'BM25', 'exact match' → book sense; 'embedding', 'nearest', 'similarity' → vector sense.

**WRONG BELIEF, CORRECTED**

|                                  |                                                                                                                                                                                                                                                                                         |
|----------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>I used to think...</b>        | embeddings understand meaning like a person does                                                                                                                                                                                                                                        |
| <b>Why it feels right</b>        | They match paraphrases a keyword search would miss, which looks like understanding.                                                                                                                                                                                                     |
| <b>The correction</b>            | <b>An embedding is a position, learned from patterns of use. It cannot tell 'Form EWB-01' from 'Form EWB-02' (two codes, one neighbourhood), cannot count, and can rank a fluent irrelevant memo above a terse correct paragraph. It sees direction, not truth — valid is not true.</b> |
| <b>The stem that punishes it</b> | A stem where semantic retrieval keeps returning the wrong form number and the answer is to add keyword search, not a bigger embedding model.                                                                                                                                            |

**R4****THE EXAM****The exam — how a stem disguises ‘the map is the wrong tool for this question’**

**The disguise.** A retrieval system ‘using state-of-the-art embeddings’ keeps failing on a particular class of question. The tell is the kind of thing being asked for: codes, ids, form numbers, exact phrases have no meaning-neighbours and defeat the map; paraphrases and symptoms are what the map is for.

**Family baited:** VG (‘embedding’ sounds like ‘understanding’) and OE (a larger embedding model where an exact-word channel was the fix).

**Mini-stem 1.** Dispatchers search ‘EWB-01 field 7’ and get paragraphs about e-way bills in general, never the field list. — **Shape: add an exact-word channel — the register — alongside the map and combine the two scores (Station 2); codes are exact-match questions.**

**Mini-stem 2.** Dispatchers describe symptoms in their own words (‘customer stuck at the yard, officer wants papers again’) and exact-word search returns nothing. — **Shape: this is the map’s home ground: embed and retrieve by nearest neighbour; keep the shortlist around 20 and let a careful second reader re-sort it (Climb 4).**

**PENCIL LAB P2 · Place eight Kaveri phrases on a 2-D meaning map by hand**

Draw two axes on paper: horizontal = ‘how much is this about paperwork?’ (0 to 10), vertical = ‘how much about physical movement?’ (0 to 10). Place each phrase as a dot by gut feel: **(1)** e-way bill re-filing · **(2)** customs hold release · **(3)** GST invoice correction · **(4)** refund for damaged sofa · **(5)** cancel and re-book the pickup · **(6)** truck late at Chakan · **(7)** carrier ETA slipped · **(8)** how many shipments went out today. Then place the query ★ ‘officer wants the bill filed again after the hold’. Read off: which two dots are nearest ★? Which dot is nearest (6)? Now draw a straight line from the origin through ★ — which dots lie roughly on that line, near or far? Those are the same-direction neighbours.

----- ∞ fold here — the worked answer is below the line -----

**WORKED ANSWER · P2**

A typical pencil map: (1) ≈ (9, 4) · (2) ≈ (8, 5) · (3) ≈ (9, 1) · (4) ≈ (5, 2) · (5) ≈ (4, 6) · (6) ≈ (1, 9) · (7) ≈ (2, 9) · (8) ≈ (3, 5). ★ lands near (8.5, 4.5). Nearest to ★: **(1) and (2)** — the customs-paperwork cluster — even though ★ shares almost no words with (1). Nearest to (6): (7). Notice (3) sits on nearly the same direction as ★ (both mostly paperwork) even though it is a different task — that is a **near miss the map cannot see**, and it is why Climb 4 adds a careful second reader and Station 2 adds the register.

Notice also (8): it is not near anything useful. ‘How many...’ is a counting question — no paragraph answers it; a query over rows does. The map is the wrong drawer for it (Station 2, P4).

**What you should feel:** you just did what an embedding model does, with two directions instead of a thousand, and you already found both its gift (paraphrase) and its blind spot (same direction, different task).

**CHECK YOURSELF**

1. What is an embedding, in one plain sentence?
2. Why can a short question and a long paragraph still match?
3. Which kinds of question defeat the map?
4. What does top\_k control and what happens at both extremes?
5. Explain to Meera in two sentences why ‘better embeddings’ will not fix the EWB-01 searches.

----- ∞ fold — answers below -----

1. A sentence's position on a meaning map, as a list of numbers, made so that similar meanings land close.
2. Cosine similarity measures direction only, not how far out a pin sits.
3. Codes, ids, form numbers, exact phrases — things with no meaning-neighbours; they want the register (exact-word search, Station 2).
4. The shortlist length: too small misses the right chunk; too large dilutes the prompt with near-misses.
5. Embeddings match by meaning, and a form number has no meaning to match — it needs its exact letters found. We add a plain word search alongside and let the two agree.

## Climb 4 · The label inside the box, and the supervisor who re-sorts the shortlist

### R1

#### THE SCENE

### The scene — a box with no label is a box from nowhere

Sunday. Crew C's six sofa boxes are on the rack, cut at the joints, perfect. The night shift moves them to another aisle. Monday, the repair man asks for 'the left armrest of Padma's sofa' — and the picker, holding a box that says only *ARMREST*, cannot say whose sofa, which side, or which order. Whole part, wrong box. The foreman's rule, written on the wall in marker: **every box says what it is AND where it came from.** 'Left armrest · sofa · KF-2481 · Deshmukh · boxed 27 Sep.' Now the picker never brings the wrong armrest, because the box itself carries the sentence that answers the question.

And a second rule on the wall, in different handwriting: the picker brings twenty boxes to the desk; the **supervisor** looks at each against the actual request and sends back the best three. The picker is fast and rough. The supervisor is slow and careful. Twenty is cheap to carry; three is what the customer sees.

### R2

#### PLAIN WORDS

### Plain words — write the context into the chunk; then re-sort the shortlist with a careful reader

A chunk cut at the heading says '(1) generate a fresh bill using the original invoice...' — six good steps that, on their own, do not say they belong to SOP 214, §4, customs holds, or re-filing. Ask a question with those words and the map may not land near this chunk at all, because the chunk never says them. The fix is embarrassingly simple: **before shelving, write a one- or two-sentence label at the top of each chunk saying what document it is from and what it is about**, and embed the labelled chunk. Now the chunk's position on the map includes 'SOP 214 · customs hold · re-filing', and the question lands on it.

#### TOY

TOY: 12 recipe cards cut at the headings. Card 7 reads 'Bake 40 min at 180°.' Which dish? Write 'Lemon cake — baking step' at the top and 'how long does the lemon cake bake?' finds card 7 first. Cost: two words per card, written once.

Then the shortlist. The map's top-20 is quick and rough — position only. A second, more expensive reader looks at the question and each of the twenty chunks *together* and asks 'does this one actually answer that?' and re-sorts. You would never run the careful reader over all 400 SOPs; you run it over twenty. That is the whole trick: a cheap wide net, then an expensive narrow one.

**R3**THE REAL  
NAME**The real name — contextual retrieval, reranking, and the failure-rate ladder****WORD CARD · contextual retrieval**

|                              |                                                                                                                                                                                                       |
|------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>PLAIN MEANING</b>         | Anthropic's shelving-time trick: before embedding (and before building the keyword register), prepend to each chunk a short generated sentence saying which document it is from and what it is about. |
| <b>WHAT IT IS NOT</b>        | Not a query-time step and not a bigger model; it is ~50-100 extra tokens written into each chunk once, at shelving.                                                                                   |
| <b>HEARD AT KAVERI</b>       | "Every §4 chunk now begins 'From SOP 214, customs holds: the procedure for re-filing the e-way bill after release.'"                                                                                  |
| <b>HOW THE EXAM WORDS IT</b> | "situate the chunk within the document", "contextual embeddings and contextual BM25", "reduce retrieval failure rate".                                                                                |
| <b>EXPANDS</b>               | <i>Loading Dock DRIFT box 'contextual retrieval' · ch. 3.5</i>                                                                                                                                        |

**WORD CARD · reranking**

|                              |                                                                                                                                                                                        |
|------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>PLAIN MEANING</b>         | A second, more careful scoring of the first shortlist — a model reads the question and each candidate together and re-orders them by how well each actually answers.                   |
| <b>WHAT IT IS NOT</b>        | Not another search: it never looks at the whole cabinet, only at the ~150 the first search brought back; and not free — it is the expensive step, which is why it runs on a shortlist. |
| <b>HEARD AT KAVERI</b>       | "Retrieve 150, rerank to 20, hand 20 to the model — §4 rises to the top and the 2019 fees memo sinks."                                                                                 |
| <b>HOW THE EXAM WORDS IT</b> | "apply a reranker to the top-N candidates", "cross-encoder", "two-stage retrieval".                                                                                                    |
| <b>EXPANDS</b>               | <i>Loading Dock glossary 'reranking' ('the supervisor re-sorting the shortlist') · Atlas A1</i>                                                                                        |

**WORD CARD · failure rate (retrieval)**

|                              |                                                                                                                                           |
|------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------|
| <b>PLAIN MEANING</b>         | The share of questions for which the right chunk was NOT in the shortlist handed to the model — the picker came back without the armrest. |
| <b>WHAT IT IS NOT</b>        | Not the share of wrong answers (the model can still stumble with the right chunk in hand) and not a latency figure.                       |
| <b>HEARD AT KAVERI</b>       | "Top-20 retrieval fails 5.7 % of the time on plain embeddings; with labels, the register and the supervisor, 1.9 %."                      |
| <b>HOW THE EXAM WORDS IT</b> | "reduce the top-20-chunk retrieval failure rate by N %", "recall@20".                                                                     |
| <b>EXPANDS</b>               | <i>Loading Dock DRIFT box 'contextual retrieval' (the 35 / 49 / 67 % figures)</i>                                                         |

The canon ladder, in gold, verbatim from the Dock and re-verified on 28 Sep 2026 against Anthropic's guidance: plain embeddings fail on **5.7 %** of questions (top-20). Contextual embeddings alone cut that by **35 %** → **3.7 %**. Contextual embeddings + a contextual register (exact-word search; Station 2 gives it its real name) cut it by **49 %** → **2.9 %**. Add reranking and the cut is **67 %** → **1.9 %**. The recommended shape: retrieve about **150** candidates, rerank, keep the **top-20**. And the threshold from Climb 1: under about **200k tokens**, skip all of it and put the corpus in the prompt.

**DRIFT · Contextual retrieval (carried verbatim from the Loading Dock; re-verified 28 Sep 2026 — holds)**

| Loading Dock / older workbook reading                                                                                                                                     | Current docs (checked 28 Sep 2026)                                                                                                                                                                        |
|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Failure-rate cuts <b>35 % → 49 % → 67 %</b> , i.e. <b>5.7 → 3.7 → 2.9 → 1.9 %</b> ; retrieve ~ <b>150</b> → rerank → <b>top-20</b> ; skip RAG under ~ <b>200k</b> tokens. | Unchanged: <b>35 / 49 / 67 %</b> , <b>5.7 / 3.7 / 2.9 / 1.9 %</b> , <b>150 → top-20</b> , ~ <b>200k</b> tokens (≈ 500 pages); chunks of a few hundred tokens with ~ <b>50-100</b> tokens of context each. |

**IN PLAIN WORDS** · Nothing has moved. Why a stem may still look different: older materials quote ‘embeddings + BM25’ without the contextual prefix and get smaller gains, or quote top-k of 5 or 10 from a different benchmark. What to answer: **the ladder above, in order — label at shelving, hybrid at fetch, rerank the shortlist; and check the corpus size first.**

**ADD IT · the reranking stage (adding is the right move here)**

A stem says top-20 map-based retrieval ‘often ranks a related but wrong section first’ and latency budget allows ~300 ms more. Adding a reranker over the shortlist is **the right move** — it is precisely the tool for ‘right neighbourhood, wrong house’. It would be OE only if the shortlist were already reliable or the budget were 2 s total with 1.9 s already spent.

**WRONG BELIEF, CORRECTED**

|                                  |                                                                                                                                                                                                                                                                             |
|----------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>I used to think...</b>        | more chunks in the prompt = better answers                                                                                                                                                                                                                                  |
| <b>Why it feels right</b>        | If the right chunk is somewhere in twenty, surely thirty is safer.                                                                                                                                                                                                          |
| <b>The correction</b>            | <b>Past a point, extra chunks are near-misses that crowd the right one and invite the model to blend them; they also cost tokens on every one of 40,000 calls. The gain comes from getting the right chunk to the top (labels, hybrid, rerank), not from carrying more.</b> |
| <b>The stem that punishes it</b> | A stem where raising top_k from 20 to 60 ‘reduced accuracy’ and the fix is a reranker, not a bigger window.                                                                                                                                                                 |

**R4**
**THE EXAM**
**The exam — how a stem disguises the ladder**

**The disguise.** A pipeline ‘retrieves the wrong section about 6 % of the time’; options offer a bigger model, more chunks, a reranker, a contextual prefix. The tell is the failure rate quoted (if it is near 5.7 % you are being asked for the ladder) and the stage word: at ingestion → contextual prefix; on the shortlist → reranker.

**Family baited:** OE (bigger model) and VG (‘reranking’ read as ‘rebuilding the cabinet’).

**Mini-stem 1.** Fixed budget, wrong-section rate ~6 %, no latency slack at query time. Which single change gives the largest cut at zero query-time cost? — **Shape: contextual retrieval: write the document/section context into each chunk at shelving; ~35 % cut, paid once a week.**

**Mini-stem 2.** Same pipeline after contextual embeddings and the contextual register are combined; still 2.9 %; 300 ms of latency slack. — **Shape: rerank the ~150 candidates to top-20; the next rung of the ladder, ~67 % total cut to 1.9 %.**

**CHECK YOURSELF**

1. What does contextual retrieval add, and when is it paid for?
2. Write the failure-rate ladder from memory.
3. Why does the reranker run on 150 and not on 400 SOPs?
4. Explain to Meera in two sentences what a reranker is.

----- ∞ fold — answers below -----

1. A short generated sentence per chunk saying document and topic, prepended before embedding/indexing; paid once at shelving time.
2. 5.7 % → 3.7 % (contextual embeddings, −35 %) → 2.9 % (+ contextual register / exact-word search, −49 %) → 1.9 % (+ rerank, −67 %).
3. It is the expensive, careful reader; the cheap wide net finds candidates, the careful one re-sorts a shortlist.
4. The first search brings twenty boxes quickly and roughly. A second, slower reader checks each against the actual question and puts the best on top.

## Climb 5 · The weekly re-shelve, the version label, and what ‘failure’ means

### R1

#### THE SCENE

### The scene — the box that was true last Tuesday

Tuesday. Customs changed the hold-release form on Monday; SOP 214 §4 was rewritten Monday night. The rack still holds last week's §4 box. A dispatcher asks, the picker fetches the box, the answer is confident, correct for last week, and wrong today. Nobody lied. The cabinet is simply behind the world, and nothing on the box says which week it belongs to. The foreman's third rule: **every box carries the date it was shelved and the version of the document it was cut from, and the rack is re-done every Sunday night.** A dispatcher can now be told ‘this is from §4 v12, shelved 27 Sep’ — and a stale box can be found and pulled.

### R2

#### PLAIN WORDS

### Plain words — a cabinet is only as true as its last re-shelving

Documents change; embeddings and registers are frozen at the moment they were built. So the shelving lane must *run again* whenever the documents change — for Kaveri, weekly — and every chunk must carry its source, version and date so a stale one can be identified and so the answer can say where it came from. This is also the difference between a system that is merely wrong and one that is *being fed bad boxes*: an out-of-date, mislabelled or malicious chunk that keeps being retrieved will keep producing confident wrong answers, and nothing downstream can tell.

#### TOY

TOY: 12 cards, re-copied every Sunday from the master recipe book. If Thursday's edit to card 7 is not re-copied until Sunday, every Friday question gets Wednesday's baking time. Write ‘v3 · copied Sun 27’ on each card and at least you can see it.

Last plain-words point: what ‘failure’ means. The clerk's job is to bring back the right box. He can fail in two directions: he can **miss** it (the right box is not in the shortlist) or he can **bury** it (the right box is there, under fourteen wrong ones). Both are retrieval failures; they are fixed by different things — a wider net for missing, a better sort for burying.

### R3

#### THE REAL NAME

### The real name — re-index, metadata, context poisoning, recall and precision

#### WORD CARD · re-index (re-shelve)

|                       |                                                                                                                         |
|-----------------------|-------------------------------------------------------------------------------------------------------------------------|
| PLAIN MEANING         | Running the shelving lane again — re-chunk, re-label, re-embed, rebuild the index — because the documents changed.      |
| WHAT IT IS NOT        | Not a query-time refresh; the fetch lane sees only what the last re-index built.                                        |
| HEARD AT KAVERI       | “Re-index every Sunday night; the SOP store changes weekly and a Monday hold must not be answered from last week's §4.” |
| HOW THE EXAM WORDS IT | “documents updated weekly”, “re-index on a schedule”, “stale retrieval”.                                                |
| EXPANDS               | <i>Loading Dock Atlas A1 (the weekly re-index loop) · ch. 3.5</i>                                                       |

**WORD CARD · metadata (version label)**

|                              |                                                                                                                                                             |
|------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>PLAIN MEANING</b>         | The facts written on the box: source document, section, version, date shelved, language — stored with the chunk and usable to filter or cite.               |
| <b>WHAT IT IS NOT</b>        | Not the content of the chunk and not the contextual prefix (which is prose embedded into the chunk); metadata is structured fields the index can filter on. |
| <b>HEARD AT KAVERI</b>       | "Filter to sop=214 AND version=current before ranking; cite 'SOP 214 §4 v12' in the answer."                                                                |
| <b>HOW THE EXAM WORDS IT</b> | "metadata filtering", "cite the source section", "restrict retrieval to the current version".                                                               |
| <b>EXPANDS</b>               | <i>Loading Dock ch. 3.6 (free text with metadata written in) · Station 2 here</i>                                                                           |

**WORD CARD · context poisoning**

|                              |                                                                                                                                              |
|------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------|
| <b>PLAIN MEANING</b>         | Wrong, stale or malicious text that gets retrieved into the prompt and steers the answer — the model trusts what is on the desk.             |
| <b>WHAT IT IS NOT</b>        | Not a hallucination (the model inventing) — the model is faithfully repeating a bad chunk; and not only an attack: a stale §4 is poison too. |
| <b>HEARD AT KAVERI</b>       | "A 2019 memo about hold fees keeps surfacing and KAVI keeps quoting a fee that no longer exists."                                            |
| <b>HOW THE EXAM WORDS IT</b> | "untrusted content in the retrieval corpus", "stale documents contaminate answers", "provenance".                                            |
| <b>EXPANDS</b>               | <i>Loading Dock glossary 'context poisoning'</i>                                                                                             |

**WORD CARD · recall vs precision (retrieval sense)**

|                              |                                                                                                                                                 |
|------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>PLAIN MEANING</b>         | Recall: of the right chunks that exist, how many the shortlist caught. Precision: of the chunks in the shortlist, how many were actually right. |
| <b>WHAT IT IS NOT</b>        | Not accuracy of the final answer, and not each other: a wide net has high recall and low precision; a tiny net the reverse.                     |
| <b>HEARD AT KAVERI</b>       | "Retrieve 150 for recall, rerank to 20 for precision — that is the whole two-stage idea in two words."                                          |
| <b>HOW THE EXAM WORDS IT</b> | "recall@k", "retrieve wide then rerank", "the right chunk was retrieved but ranked 14th".                                                       |
| <b>EXPANDS</b>               | <i>Loading Dock glossary 'recall vs precision (retrieval sense)'</i>                                                                            |

Put together: wide first retrieval (~**150**) buys recall; reranking to **top-20** buys precision; contextual prefixes lift both; version fields on every chunk keep stale versions out; a weekly re-index keeps the cabinet true. The **5.7 → 1.9 %** ladder is a recall@20 ladder — the share of questions whose right chunk was missing from the twenty.

**TRAP · 'Retrieval is working — accuracy is 94 %'**

A retrieval failure rate of 6 % on **40,000** calls a day is **2,400** wrong-paragraph answers a day, every day, each of them fluent. The **trap** is reading a retrieval percentage as 'good enough' without multiplying by the daily volume the stem gave you. The daily volume is the tell: it is there to be multiplied.

**R4****THE EXAM****The exam — how a stem disguises staleness and poison**

**The disguise.** 'Answers were correct at launch and drifted'; 'a fee that was abolished keeps appearing'; 'documents are updated weekly but the index was built once'. The tell is a time word — weekly, since, drifted, still — pointing at re-indexing and version metadata rather than at the model.

**Family baited:** CM (a D1 answer such as 'add an evaluator' to a shelving problem) and SF (the stale chunk fails silently — nothing errors).

**Mini-stem 1.** SOPs change weekly; the index was built at launch; dispatchers now get last month's procedure with full confidence. — **Shape: schedule the shelving lane (re-chunk, re-embed, rebuild) on the documents' rhythm; stamp every chunk with version and date; filter to current.**

**Mini-stem 2.** A retired memo keeps outranking the current SOP for fee questions. — **Shape: remove or version-tag it and filter; treat it as context poisoning, not as a prompt problem.**

## CHECK YOURSELF

1. Why does a correct cabinet become wrong without anyone lying?
2. Recall or precision: the right chunk was in the shortlist but ranked 14th. Which is low?
3. What is context poisoning, and is it always an attack?
4. 6 % retrieval failure at 40,000 calls a day is how many wrong-paragraph answers?
5. Explain to Meera in two sentences why we re-shelve every Sunday.

----- ∞ fold — answers below -----

1. Documents change; embeddings and registers are frozen at build time; without re-indexing the fetch lane serves last week's truth.
2. Precision (and ordering). Recall was fine — the net caught it; the sort buried it. Fix with reranking.
3. Bad text retrieved into the prompt steering the answer; no — a stale or mislabelled chunk poisons just as well.
4. About 2,400 a day.
5. The SOPs change every week and the system only knows what was on the shelves the last time we built them. Sunday night we rebuild, so Monday's answers come from Monday's rules.

## WHAT WATSON WROTE DOWN

- RAG = clerk + cabinet: fetch the paragraph, then answer from it and cite it. Skip it under ~200k tokens.
- Two lanes, two rhythms: shelving weekly (400 SOPs), fetching 40,000/day. Per-fetch costs are multiplied by the day.
- **D3 Law 2:** chunk along meaning — headings for structured docs; fixed+overlap for flat text; semantic only when it earns it. **Fixed-size through §4 = the cancellation steps.**
- Embedding = position; cosine = same direction; index = 'what is nearest?'; top\_k = shortlist length. Codes and ids defeat the map.
- Ladder: 5.7 → 3.7 → 2.9 → 1.9 % (35 / 49 / 67 %): contextual prefix → + contextual BM25 → + rerank. Retrieve ~150, rerank to top-20.
- Recall = did we catch it; precision = did we bury it. Wide net then careful sort.
- Re-index on the documents' rhythm; version metadata on every chunk; stale = poison.
- Tells: heading-structured · procedure/steps · codes/ids · weekly/drifted · daily volume (multiply it).

**Law in force:** D3 Law 2 · Chunk along meaning, not length.

**Read next in the Loading Dock:** ch. 3.5 · E4 (watch the retrieved-chunk line) · E5 · Atlas A1-A2 · glossary entries for chunking, embedding, vector index, reranking, top\_k, recall/precision, context poisoning · DRIFT box 'contextual retrieval'

## TONIGHT YOU WILL

## Wednesday 30 Sep · ~45 min · Station 2 · The register and the foreman

- ☐ watch two searches fail on different questions and succeed together, by hand (P3)
- ☐ see why a fluent system cannot count, and what the ₹1,800 mis-route actually was
- ☐ route ten dispatcher questions into four drawers, one cell of the A3 matrix at a time (P4)
- ☐ own the words: keyword search / BM25, semantic search, hybrid search, score fusion, structured lookup, aggregate query, query pattern, data shape
- ☐ meet **D3 Law 3**: retrieval follows the shape of the data, not the fashion

**Then:** read Loading Dock ch. 3.6. It should read like a summary. Star anything still foggy; the reverse map names the Walk page.

## STATION 2

# The register and the foreman

Apply retrieval strategies matched to data shape and query pattern — two searches, four drawers

## OBJECTIVE

## 3.6

expands Loading Dock ch. 3.6 · E5-E6 · Atlas A3 · glossary (BM25/keyword, hybrid search, recall vs precision)

## Climb 1 · The register and the foreman — two ways to find a box, and what each cannot do

## R1

## THE SCENE

### The scene — Wednesday, two questions, two failures

Wednesday morning. Two dispatchers, two questions, thirty seconds apart. The first: “Which SOP has *Form GST EWB-01* field seven?” The second: “Customer stuck at the yard, officer wants the papers again — what do I do?”

Sunita goes to the **register** — the fat alphabetical ledger where every SOP's words are listed with the section they appear in. ‘EWB-01’: three entries, page numbers, done in ten seconds. The second question she tries the same way: ‘stuck’ — nothing. ‘papers again’ — nothing. The SOP says ‘re-filing after release’; the dispatcher said ‘papers again’. Same thing, different words; the register is deaf to meaning.

Ramesh asks the **old foreman**. Second question: “That's \$4, son, re-filing after a hold.” Instant. First question: “EWB-oh-what? A form's a form. Try the register.” The foreman knows what things are *about*; a code is about nothing.

Meera, from the door: “So we need both. Stop arguing about which is better.”

## R2

## PLAIN WORDS

### Plain words — exact words versus same meaning

There are two honest ways to find a paragraph. **Match the words**: does the paragraph contain the words the question used? Fast, exact, brilliant for codes, ids, names, form numbers — and blind the moment the question is phrased differently from the document. **Match the meaning**: put the question on the meaning map from Station 1 and pull what is nearest. Brilliant for paraphrases and symptoms — and blind to codes, because ‘EWB-01’ and ‘EWB-02’ sit in the same neighbourhood.

## TOY

TOY: 10 cards. Query A ‘EWB-01’: word-matching finds the 1 card containing that string; meaning-matching returns the 3 cards about e-way bills and cannot tell which. Query B ‘papers again after the hold’: word-matching finds 0 cards; meaning-matching returns the re-filing card first. Two queries, two winners, no overall winner.

Word-matching has one more feature worth knowing in plain words: it does not treat every word equally. A word that appears in nearly every SOP ('consignment', 'shall') counts for almost nothing; a word that appears in one section ('EWB-01') counts for a lot; and a short paragraph that mentions the word twice beats a long one that mentions it once. That is the whole of the register's cleverness.

#### A · the register alone (keyword)



the register

"e-way bill" ✓ exact words  
 "re-file after hold" ✗ different words  
 → finds §4 only if the question uses the SOP's words

#### B · the foreman alone (semantic)



the foreman

"re-file after hold" ✓ same meaning  
 "Form GST EWB-01" ✗ a code has no meaning-neighbours  
 → finds §4 by sense, misses exact codes

#### C · both, fused 0.6 / 0.4



the register



the foreman

exact codes → register wins  
 paraphrases → foreman wins  
 agreement → top of the list; §4 surfaces either way

**W6** · The register alone, the foreman alone, and both together. Each alone fails on the other's question; fused, the code question is won by the register and the paraphrase by the foreman, and a chunk both agree on rises to the top. Redraw C from memory on Day D — it is the 'hybrid' box of Atlas A1 and A3.

### R3

THE REAL  
NAME

## The real name — keyword search (BM25), semantic search, and D3 Law 3

### WORD CARD · keyword search (BM25)

|                       |                                                                                                                                                                            |
|-----------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| PLAIN MEANING         | Finding chunks by the words they contain, scored so that rare words count more than common ones and short chunks that repeat the word beat long ones that mention it once. |
| WHAT IT IS NOT        | Not a plain 'find in page' (that has no scoring) and not dumb: it is the strongest tool there is for codes, ids and exact phrases.                                         |
| HEARD AT KAVERI       | "EWB-01 is a keyword question — the register finds it in one hop; the map cannot tell it from EWB-02."                                                                     |
| HOW THE EXAM WORDS IT | "BM25", "lexical / sparse retrieval", "exact-match terms", "full-text index".                                                                                              |
| EXPANDS               | <i>Loading Dock glossary 'BM25/keyword' ('check the register')</i>                                                                                                         |

### WORD CARD · semantic search

|                       |                                                                                                                                                      |
|-----------------------|------------------------------------------------------------------------------------------------------------------------------------------------------|
| PLAIN MEANING         | Finding chunks by nearness on the meaning map — embed the question, return the nearest neighbours.                                                   |
| WHAT IT IS NOT        | Not 'smarter keyword search'; it is a different sense entirely. It sees direction, not letters, so it cannot distinguish two codes and cannot count. |
| HEARD AT KAVERI       | "'Officer wants the papers again' is a semantic question — the foreman knows it is §4 without a single shared word."                                 |
| HOW THE EXAM WORDS IT | "vector / dense retrieval", "embedding similarity", "paraphrase queries", "users describe symptoms in their own words".                              |
| EXPANDS               | <i>Loading Dock glossary 'embedding', 'vector index' ('ask the old foreman')</i>                                                                     |

**WORD CARD · query pattern**

|                              |                                                                                                                                                      |
|------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>PLAIN MEANING</b>         | The shape of what people actually ask: an id? a count? an exact code? a symptom in their own words? — read off the questions, not off the documents. |
| <b>WHAT IT IS NOT</b>        | Not the data shape (how the documents are built); the two are read separately and the strategy must fit both.                                        |
| <b>HEARD AT KAVERI</b>       | “Dispatchers' questions are 60 % ids, 25 % procedures, 15 % counts — that decides the drawers, not the vendor's brochure.”                           |
| <b>HOW THE EXAM WORDS IT</b> | “matched to query patterns”, “users typically ask for...”, “the dominant query type is...”.                                                          |
| <b>EXPANDS</b>               | <i>Loading Dock Atlas A3 (the column axis)</i>                                                                                                       |

**WORD CARD · data shape**

|                              |                                                                                                                                        |
|------------------------------|----------------------------------------------------------------------------------------------------------------------------------------|
| <b>PLAIN MEANING</b>         | How the information is actually built: rows in a table with ids? prose with headings? free text with a few known fields inside it?     |
| <b>WHAT IT IS NOT</b>        | Not the file format; a PDF of a table is still a table, and a spreadsheet of remarks is still prose.                                   |
| <b>HEARD AT KAVERI</b>       | “Shipments are rows with a KF- id; SOPs are prose under headings; carrier emails are free text with a tracking number buried in them.” |
| <b>HOW THE EXAM WORDS IT</b> | “structured / semi-structured / unstructured”, “heading-structured documents”, “transactional records”.                                |
| <b>EXPANDS</b>               | <i>Loading Dock Atlas A3 (the row axis)</i>                                                                                            |

Technically: the register is a keyword index scored with BM25; the foreman is semantic search over the vector index from Station 1. Neither is the modern one or the old one — they answer different questions. The Dock's E5 shows exactly where keyword alone and semantic alone each fail on ‘e-way bill re-filing’; on Day B, watch the two score columns it prints before they are combined.

**D3 LAW 3 · Retrieval follows the shape of the data, not the fashion.**

Born here. Read the shape of the data and the shape of the questions first; choose the search that fits them; use both when the questions come in both shapes. Broken: ‘we moved everything to vector search because that is what RAG means’ — and the EWB-01 searches started returning EWB-02.

► **SAME WORD, TWO WORLDS · prompt** — **(1)** system-prompt sense: the standing instruction the model reads every call. **(2)** MCP sense (Station 7): a reusable template a server offers, such as customs\_triage. **The tell:** ‘instruct the model to...’ → system prompt; ‘the server exposes a prompt’ → MCP primitive.

**R4****THE EXAM****The exam — how a stem disguises ‘wrong search for this question’**

**The disguise.** ‘The team migrated to a vector database and users complain that searches for X now return Y.’ The tell is what X is: codes, SKUs, form numbers, error strings, names → the register was thrown away; ‘in their own words’, symptoms, synonyms → the map is missing.

**Family baited:** HL (‘keyword search is deprecated’) and VG (‘semantic’ read as ‘better’).

**Mini-stem 1.** After moving SOP search to embeddings, searches for part numbers like ‘BRK-2231’ return the neighbouring part's page. — **Shape: restore a keyword (BM25) channel for exact terms and combine it with the vector scores; part numbers are lexical.**

**Mini-stem 2.** A keyword-only helpdesk search misses tickets whose authors used different words from the KB article. — **Shape: add semantic search and combine the two lists; paraphrase is what the map is for.**

## CHECK YOURSELF

1. What does the register do well and what is it deaf to?
2. What does the foreman do well and what is he blind to?
3. What two shapes must you read before choosing a retrieval strategy?
4. Explain D3 Law 3 to Meera in two sentences.

----- ∞ fold — answers below -----

1. Exact words, codes, ids, names — and deaf to meaning: different words for the same thing find nothing.
2. Meaning, paraphrase, symptoms — and blind to codes, which have no meaning-neighbours.
3. The data shape (rows? prose under headings? free text with fields?) and the query pattern (ids? counts? codes? symptoms?).
4. We pick the search to match what the data looks like and what people actually ask, not whichever is fashionable. Codes want the register, symptoms want the foreman, and most days we need both.

## Climb 2 · Both, and trust where they agree — hybrid by hand

### R1

#### THE SCENE

### The scene — Sunita and the foreman at the same desk

Thursday. Meera has moved Sunita's register to the foreman's desk. Every question is now answered twice, and the two answers are laid side by side. When the register and the foreman name the same section, that section goes to the top without argument. When only one of them has an opinion — the register on a code, the foreman on a description — that one leads. When they disagree, the foreman's word carries a little more weight than the register's, because most dispatcher questions are descriptions, not codes. Nobody was replaced. The desk got slower by a second and stopped being wrong.

### R2

#### PLAIN WORDS

### Plain words — two scores, one list

Run both searches. Each gives every chunk a score. Turn both scores onto the same scale (say 0 to 1), then blend them — 60 % of the meaning score plus 40 % of the word score, or whatever weighting fits the question mix — and sort by the blend. Chunks that both searches liked rise; chunks only one liked still appear; chunks neither liked vanish. The weights are a dial you tune to the query pattern: more weight on words for a code-heavy desk, more on meaning for a symptom-heavy one.

#### TOY

TOY: 4 chunks, query 're-file the EWB after a hold'. Word scores: \$4 = 0.5, \$3 = 0.5, EWB-form page = 1.0, fees memo = 0. Meaning scores: \$4 = 0.9, \$3 = 0.8, form page = 0.4, memo = 0.3. Blend  $0.6 \times \text{meaning} + 0.4 \times \text{words}$ : \$4 = 0.74, \$3 = 0.68, form page = 0.64, memo = 0.18. \$4 wins — neither score alone put it clearly first.

### R3

#### THE REAL NAME

### The real name — hybrid search and score fusion

#### WORD CARD · hybrid search

|                       |                                                                                                                                                 |
|-----------------------|-------------------------------------------------------------------------------------------------------------------------------------------------|
| PLAIN MEANING         | Running keyword and semantic search on the same question and merging the two ranked lists into one.                                             |
| WHAT IT IS NOT        | Not 'semantic with a keyword fallback' (that only helps when the first returns nothing); both run every time and both scores count.             |
| HEARD AT KAVERI       | "Hybrid is why the desk stopped being wrong: codes come from the register, descriptions from the foreman, and \$4 sits on top when both agree." |
| HOW THE EXAM WORDS IT | "combine BM25 and vector scores", "hybrid retrieval", "contextual embeddings + contextual BM25" (the 49 % rung of the ladder).                  |
| EXPANDS               | <i>Loading Dock glossary 'hybrid search' ('trust where they agree') · E5 · Atlas A1</i>                                                         |

**WORD CARD · score fusion**

|                              |                                                                                                                                                                                     |
|------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>PLAIN MEANING</b>         | How the two lists become one: normalise both scores, then take a weighted sum (e.g. 0.6 / 0.4) — or, rank-based, add $1/(k + \text{rank})$ from each list (reciprocal rank fusion). |
| <b>WHAT IT IS NOT</b>        | Not reranking (Station 1's careful second reader); fusion is arithmetic on the two first-pass scores, reranking is a model reading the shortlist.                                   |
| <b>HEARD AT KAVERI</b>       | "We fuse 0.6 semantic / 0.4 keyword because most desk questions are descriptions; a parts desk would flip it."                                                                      |
| <b>HOW THE EXAM WORDS IT</b> | "weighted combination of scores", "RRF", "tune the fusion weight to the query mix".                                                                                                 |
| <b>EXPANDS</b>               | <i>Loading Dock E5 (the fused-score column)</i>                                                                                                                                     |

In the ladder from Station 1, this is the second rung: contextual embeddings + contextual BM25 cut the top-20 failure rate by **49 % (5.7 → 2.9 %)**. The contextual prefix helps the register as much as the map — a chunk whose first line says 'SOP 214 · customs hold · re-filing' now *contains* the words a dispatcher will type.

**WRONG BELIEF, CORRECTED**

|                                  |                                                                                                                                                                                                                                         |
|----------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>I used to think...</b>        | RAG replaced keyword search                                                                                                                                                                                                             |
| <b>Why it feels right</b>        | Every RAG diagram shows an embedding model and a vector database; the register is nowhere in the picture.                                                                                                                               |
| <b>The correction</b>            | <b>Production retrieval is hybrid by default. Keyword search is the only reliable way to find codes, ids and exact phrases, and Anthropic's own failure-rate ladder gets its second rung from adding contextual BM25 to embeddings.</b> |
| <b>The stem that punishes it</b> | A stem where the 'modern' vector-only option loses to the option that keeps BM25 and fuses.                                                                                                                                             |

**ADD IT · a second retrieval channel (adding is the right move here)**

A stem describes a desk where 30 % of questions are part numbers and 70 % are descriptions, on a single vector index. Adding a keyword channel and fusing is **the right move** — it is one index and a few lines of arithmetic, and it is the only fix for the 30 %. It would be OE if every question were a description and the failure rate were already at the bottom of the ladder.

**R4****THE EXAM****The exam — how a stem disguises 'you need both'**

**The disguise.** Two symptoms in one stem — some questions miss because of wording, others because of codes — with options that fix only one. The tell is the pair: 'both ... and ...', 'some users ... other users ...', two query shapes named in one breath.

**Family baited:** TR (picking the fix for the louder complaint) and OE (a reranker before there is a second channel to rank).

**Mini-stem 1.** Support search misses paraphrased questions and also confuses error codes E-401 / E-410. Budget allows one change. — **Shape: hybrid: keep the vector index, add BM25, fuse with a weight tuned to the mix.**

**Mini-stem 2.** Hybrid is in place; the right chunk is usually in the top 20 but ranked 8th–14th. — **Shape: rerank the shortlist (Station 1, Climb 4); fusion is done, ordering is the problem.**

### PENCIL LAB P3 · Score four TOY chunks two ways, fuse them, then watch each channel fail alone

Four TOY chunks: **A** '§4 Re-filing after a hold: generate a fresh bill...' (52 words) · **B** '§3 Cancellation: cancel the bill on the portal within twenty-four hours...' (48 words) · **C** 'Form EWB-01 field list: field 7 = vehicle number...' (30 words) · **D** '2019 memo: hold fees were ₹250/day...' (40 words).

**Query 1:** 'officer wants the bill filed again after the hold'. **(i) The register:** count, for each chunk, how many of the query's *rare* words it contains (ignore 'the', 'after', 'wants'; count bill / hold / filed-or-re-filing / officer). Divide by the chunk's word count × 10 to reward short chunks. **(ii) The foreman:** give each chunk a nearness 0–1 by gut (use your P2 map). **(iii) Scale** both columns to 0–1 (divide by the column's max), fuse  $0.6 \times \text{meaning} + 0.4 \times \text{words}$ , sort.

**Query 2:** 'EWB-01 field 7'. Repeat (i)–(iii). Then answer: on which query does the register alone put the wrong chunk first? On which does the foreman alone? Does the fused list put the right chunk first on both?

----- ∞ fold here — the worked answer is below the line -----

#### WORKED ANSWER · P3

**Query 1, register:** A has bill, hold, re-filing →  $3 / 52 \times 10 \approx 0.58$ ; B has bill, hold →  $2 / 48 \times 10 \approx 0.42$ ; C has bill? no ('EWB' is the code, not the word) → 0; D has hold →  $1 / 40 \times 10 = 0.25$ . Scaled: A 1.0 · B 0.72 · C 0 · D 0.43. **Foreman:** A 0.9 · B 0.8 · C 0.4 · D 0.3 → scaled A 1.0 · B 0.89 · C 0.44 · D 0.33. **Fused:** A =  $0.6 + 0.4 = 1.00$  · B =  $0.53 + 0.29 = 0.82$  · C 0.27 · D 0.37. A first. (Register alone also put A first here, narrowly; the foreman alone had B close behind — fusion widened the gap.)

**Query 2, register:** only C contains 'EWB-01' and 'field' →  $C \approx 2 / 30 \times 10 = 0.67$ , others 0 → scaled C 1.0, rest 0. **Foreman:** everything about e-way bills is near: A 0.6 · B 0.5 · C 0.7 · D 0.2 → scaled A 0.86 · B 0.71 · C 1.0 · D 0.29. **Fused:** C =  $0.6 + 0.4 = 1.00$  · A 0.52 · B 0.43 · D 0.17. C first.

**Where each fails alone:** change Query 1 to 'papers again after release' — the register scores every chunk 0 (no shared rare words) and has no opinion, so it 'fails' by silence; the foreman still says A. Change Query 2 to 'EWB-02 field 7' — the register finds nothing (no such chunk) and the foreman confidently returns C, the EWB-01 page: **wrong form, fluent**. Fused, the first query is carried by the foreman and the second is at least flagged by a register score of zero on every chunk — the signal that no exact match exists.

**What you should feel:** you have just done E5's fused-score column by hand. On Day B, when it prints, you will know which column saved which query.

#### CHECK YOURSELF

1. What is hybrid search in one sentence?
2. Fusion versus reranking — which is arithmetic and which is a reader?
3. Which rung of the failure-rate ladder does hybrid supply, and by how much?
4. When would you weight the register above the foreman?
5. Explain to Meera in two sentences why we kept the register after buying the map.

----- ∞ fold — answers below -----

1. Run keyword and semantic search on every question and merge the two ranked lists.
2. Fusion is arithmetic on the two first-pass scores; reranking is a model reading the question with each shortlisted chunk.
3. The second: contextual embeddings + contextual BM25, −49 %,  $5.7 \rightarrow 2.9$  %.
4. A code-heavy query mix — a parts desk, error codes, form numbers.
5. The map finds things by meaning and cannot tell two form numbers apart. The register finds exact codes in one hop, so we run both and trust where they agree.

### Climb 3 · The fluent wrong number, and the four drawers

R1

THE SCENE

**The scene — ‘how many shipments were late this week?’**

Friday, 5 pm, Meera's weekly call. She types into KAVI: “How many shipments were late this week?” KAVI answers in one confident sentence: “Approximately 312 shipments were delayed this week, primarily due to customs holds and carrier capacity.” Meera reads it out. Arjun, from the back: “Where did 312 come from?” It came from a paragraph. Retrieval had fetched a chunk of last month's ops review that mentioned 312 late shipments, and the model had answered fluently from the desk in front of it. The real number this week was 41. Nothing broke. Nobody was warned. The ₹1,800 the Dock's E6 prices for the mis-route is the cost of everything that ran to produce a number that a ₹4 query over the shipments table would have produced correctly.

R2

PLAIN WORDS

**Plain words — four kinds of question, four drawers**

A paragraph can tell you a procedure. It cannot tell you how many shipments were late *this week*, because that number is not written anywhere — it has to be counted from the rows. And a paragraph is the wrong place to look up a single consignment, because the consignment lives in a table with an id. So before any searching happens, read the question's shape and open the right drawer:

| Drawer               | The question sounds like                            | What answers it                                     | Kaveri example                                    |
|----------------------|-----------------------------------------------------|-----------------------------------------------------|---------------------------------------------------|
| ONE RECORD BY ITS ID | an id, a code, ‘where is / what is the status of X’ | one exact record by key — no searching at all       | ‘Where is KF-2481?’ → the shipment row            |
| COUNT THE ROWS       | how many, total, average, trend, ‘this week’        | a query that counts or sums rows                    | ‘How many late this week?’ → count over shipments |
| EXACT WORDS          | an exact term, code, form number, error string      | the register, weighted up                           | ‘EWB-01 field 7’                                  |
| MEANING              | a symptom, a paraphrase, ‘what do I do when...’     | the foreman + the register, combined, then reranked | ‘officer wants the papers again’ → §4             |

**TOY**

TOY: 10 questions from one shift: 5 ‘where is KF-...’, 2 ‘how many...’, 1 form number, 2 ‘what do I do when...’. Only 3 of the 10 want the cabinet at all. Sending all 10 through the cabinet is slow for the 7 and wrong for the 2 counts.

Two refinements in plain words. First, **tune by question shape**: a desk that mostly asks for codes weights the register up; a desk that mostly describes symptoms weights the foreman up. Second, **free text with facts inside it** — carrier emails with a tracking number buried in them — gets the facts pulled out and written on the box as metadata at shelving time, so the record can be found by its key and the prose can still be searched by meaning.

LOOKUP

an id or a code → one exact record

“Where is KF-2481?” → `get_shipment_status`

no RAG at all

AGGREGATE

a count, a sum, a ‘how many’ → a query over rows

“How many shipments late this week?” → SQL-ish

RAG cannot count

KEYWORD-LEAN

exact words, codes, form numbers → the register

“Form GST EWB-01 fields” → BM25 weighted up

codes have no neighbours

SEMANTIC / HYBRID

a symptom, a paraphrase, a policy question → the foreman + register

“customer stuck at customs, what now?” → hybrid + rerank

meaning, then agreement

read the question's SHAPE first, then open one drawer — D3 Law 3

**W7** · The four drawers, slowly. Read the shape of the question, open one drawer. The first two are not retrieval at all; only the last two use the cabinet. This is Atlas A3 with the cells un-crowded — redraw A3 from memory on Day D, drawer by drawer.

**R3**THE REAL  
NAME**The real name — structured lookup, aggregate query, and the A3 matrix****WORD CARD · structured lookup**

|                              |                                                                                                                                        |
|------------------------------|----------------------------------------------------------------------------------------------------------------------------------------|
| <b>PLAIN MEANING</b>         | Fetching one record by its key from a table or an API — consignment id in, row out — with no searching and no ranking.                 |
| <b>WHAT IT IS NOT</b>        | Not retrieval in the RAG sense; nothing is embedded, nothing is scored. Routing a lookup through the cabinet is the classic mis-route. |
| <b>HEARD AT KAVERI</b>       | “KF-2481 is a lookup: <code>get_shipment_status('KF-2481')</code> , one row, four milliseconds.”                                       |
| <b>HOW THE EXAM WORDS IT</b> | “query the transactional system directly”, “structured data with a primary key”, “no RAG needed for...”.                               |
| <b>EXPANDS</b>               | <i>Loading Dock E6 (consignment id → structured lookup, ‘no RAG!’) · Atlas A3</i>                                                      |

**WORD CARD · aggregate query**

|                              |                                                                                                                                                 |
|------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>PLAIN MEANING</b>         | A count, sum, average or trend computed over many rows — SQL-shaped: ‘how many shipments were late this week’.                                  |
| <b>WHAT IT IS NOT</b>        | Not something a paragraph can answer: no document contains this week's count, so retrieval can only find an old number and present it fluently. |
| <b>HEARD AT KAVERI</b>       | “‘How many late this week?’ is an aggregate — route it to the shipments table, not to the SOP store; that was the ₹1,800 lesson.”               |
| <b>HOW THE EXAM WORDS IT</b> | “analytical question”, “count / total / average”, “RAG returns a plausible but stale figure”.                                                   |
| <b>EXPANDS</b>               | <i>Loading Dock E6 (‘how many shipments late this week’ → SQL-ish aggregate; the mis-routed query and its cost) · Atlas A3</i>                  |

**WORD CARD · metadata filtering (free text with fields written in)**

|                              |                                                                                                                                                                                    |
|------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>PLAIN MEANING</b>         | Pulling the known facts out of free text at shelving time (tracking number, carrier, date), storing them as fields on the chunk, and using them to narrow a search before ranking. |
| <b>WHAT IT IS NOT</b>        | Not the contextual prefix (prose inside the chunk) and not a separate database: the fields live with the chunk and the search filters on them first.                               |
| <b>HEARD AT KAVERI</b>       | “Carrier emails: extract the tracking number and date into fields; then ‘what did BlueDart say about KF-2481?’ filters by id and searches the prose.”                              |
| <b>HOW THE EXAM WORDS IT</b> | “semi-structured data”, “filter by metadata then retrieve”, “extract entities at ingestion”.                                                                                       |
| <b>EXPANDS</b>               | <i>Loading Dock ch. 3.6 (free text with metadata written in) · Atlas A3</i>                                                                                                        |

The A3 matrix is these drawers drawn as data shape × query pattern: rows = how the data is built (table with keys · prose under headings · free text with fields inside), columns = what is asked (id · count · exact term · description), and each cell names one route with a Kaveri example. Day D redraws A3; P4 fills it one cell at a time tonight. The Dock's E6 runs three queries down three routes and then mis-routes one on purpose — the canon price of that mis-route is **₹1,800** against **₹4** for the right drawer.

**TRAP · The fluent wrong number**

A count answered from a paragraph is the most dangerous failure in this domain because it is *never* an error: the sentence is grammatical, the number is plausible, the source chunk really exists. The **trap** is any option that sends a ‘how many / total / this week’ question through retrieval. The tell is the counting word — how many, total, average, trend — and the time window, which no document can contain.

**WRONG BELIEF, CORRECTED**

|                                  |                                                                                                                                                                                                                   |
|----------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>I used to think...</b>        | one good retrieval pipeline should answer every question                                                                                                                                                          |
| <b>Why it feels right</b>        | It answered the hard policy questions beautifully, so surely it can handle the easy ones.                                                                                                                         |
| <b>The correction</b>            | <b>The pipeline answers questions whose answers are written in a paragraph. Ids want a lookup, counts want a query; sending them through the cabinet is slower, dearer, and — for counts — confidently wrong.</b> |
| <b>The stem that punishes it</b> | A stem where 'route by question shape first' beats 'improve the retrieval pipeline' for a system that mis-answers counts.                                                                                         |

**R4****THE EXAM****The exam — how a stem disguises the wrong drawer**

**The disguise.** A single 'knowledge assistant' that 'occasionally gives outdated figures' or 'is slow for simple status checks'. The tell is the question type named in the complaint: figures / how many / this week → aggregate drawer; status of / where is / id → lookup drawer; and the fix is a **router by shape in front of retrieval** (D1's router, now with retrieval as one of its destinations — cite it as D1 canon, not as a new idea).

**Family baited:** CM (a retrieval tweak for a routing problem) and SF (the wrong number raises no error).

**Mini-stem 1.** Dispatchers use the SOP assistant to check consignment status; it takes four seconds and sometimes quotes a stale location from an email. — **Shape: route id-shaped questions to a structured lookup against the shipments system; keep retrieval for procedure questions.**

**Mini-stem 2.** Management asks the assistant for weekly late-shipment counts and gets plausible numbers that do not match the report. — **Shape: route counting questions to an aggregate query over the rows; never to retrieval; show the query, not a paragraph.**

**PENCIL LAB P4 · Route ten dispatcher questions into the A3 matrix, one cell at a time**

Draw the matrix: rows = **table with keys · prose under headings · free text with fields inside**; columns = **id · count · exact term · description**. Place each question in one cell and write the route (lookup / aggregate / keyword-lean / semantic-lean / hybrid): (1) Where is KF-2481? · (2) How many shipments went out from Chakan today? · (3) What are the steps to re-file the e-way bill after a hold? · (4) EWB-01 field 7 — what goes there? · (5) What did BlueDart say about KF-2481 yesterday? · (6) Which SOP covers damaged goods? · (7) Average customs-hold duration last month? · (8) The officer wants the papers again — what now? · (9) Is Padma's sofa still on hold? · (10) Which carrier mentions 'bonded yard' in its contract? Then: which questions never touch the cabinet? Which cell is most crowded, and what does that tell you to weight?

----- ∞ fold here — the worked answer is below the line -----

**WORKED ANSWER · P4**

(1) table × id → **lookup** · (2) table × count → **aggregate** · (3) prose × description → **hybrid** (then rerank) · (4) prose × exact term → **keyword-lean** · (5) free text × id → **metadata filter by KF-2481, then semantic over the prose** · (6) prose × description → **semantic-lean** (a 'which document is about X' question — the map's home ground) · (7) table × count → **aggregate** · (8) prose × description → **hybrid** · (9) table × id → **lookup** ('Padma's sofa' resolves to KF-2481 first — a tiny lookup before the lookup) · (10) free text × exact term → **keyword-lean** with a carrier filter.

Never touch the cabinet: (1), (2), (7), (9) — four of ten. Most crowded cell: prose × description (3, 6, 8) — so the foreman gets the heavier fusion weight on this desk. On a parts desk the crowded cell would be exact term, and the register would.

**What you should feel:** the matrix is not a taxonomy to memorise; it is a habit — shape first, drawer second, search last. E6 does the same three routes in code on Day B; watch the line that prints which route each query took, and the cost line for the one it mis-routes on purpose.

## CHECK YOURSELF

1. Why can't RAG count?
2. Name the four drawers and say which two never use the cabinet.
3. What was the ₹1,800 vs ₹4 in E6?
4. How does free text with a tracking number inside it get both a lookup and a search?
5. Explain to Meera in two sentences why KAVI said 312.

----- ∞ fold — answers below -----

1. A count over this week's rows is not written in any paragraph; retrieval can only find an old number and the model presents it fluently.
2. Lookup, aggregate, keyword-lean, semantic/hybrid; lookup and aggregate never touch it.
3. The price of mis-routing a question through retrieval against the price of the right drawer — the mis-route bill versus the structured route.
4. Extract the number at shelving time as a metadata field; filter by it, then search the prose by meaning.
5. It found an old review that mentioned 312 late shipments and answered from that paragraph. Counts have to come from the shipments table, so we route 'how many' questions there instead.

### WHAT WATSON WROTE DOWN

- Register = keyword/BM25: exact words, rare words count more, short chunks win. Foreman = semantic: nearest on the map. Each blind to the other's question.
- **D3 Law 3**: read data shape × query pattern first. Codes/ids → register; symptoms/paraphrase → foreman; both shapes → hybrid.
- Hybrid = both every time, fused (0.6/0.4 or RRF); fusion is arithmetic, reranking is a reader. Ladder rung 2: **−49 %, 5.7 → 2.9 %**.
- Four drawers: lookup · aggregate · keyword-lean · semantic/hybrid. Lookup and aggregate never touch the cabinet.
- **RAG cannot count**: a fluent wrong number raises no error. Tell: how many / total / this week. E6 mis-route **₹1,800 vs ₹4**.
- Free text with fields inside → extract to metadata at shelving; filter, then search.
- Router by shape in front of retrieval = D1's router with a new destination; cite D1, don't re-invent it.

**Law in force**: D3 Law 3 · Retrieval follows the shape of the data, not the fashion.

**Read next in the Loading Dock**: ch. 3.6 · E5 (the two score columns, then the fused one) · E6 (which route each query took; the mis-route cost line) · Atlas A3 · glossary entries for BM25/keyword, hybrid search, recall vs precision

## TONIGHT YOU WILL

## Thursday 1 Oct · ~45 min · Station 3 · The gate

- ☐ read a carrier's eighteen-month-old pass and find the words **all bays** — the Refund Heist's ghost in Professional clothes
- ☐ separate WHO YOU ARE from WHICH BAYS for good, and see why 'logged in securely' is not 'secure'
- ☐ meet the four kinds of pass on one card and the one thing none of them settles
- ☐ write the scope table for a five-caller TOY tower, find the wildcard, write the one-line fix, and say what the ledger shows for a denied call (P7)
- ☐ own the words: authentication, authorization, token (auth sense), bearer token, API key, OAuth, service account, SSO, mTLS, scope, least privilege, wildcard, audit log, transport (stdio · Streamable HTTP)
- ☐ meet **D3 Law 1**: every tool you add is a door; know who holds the key

**Then:** read Loading Dock ch. 3.2. It should read like a summary. Star anything still foggy; the reverse map names the Walk page. Stations 1 and 3 must both be read before Day A.

## STATION 3

## The gate

Analyze authentication and authorization requirements; identify security gaps — who you are, and which bays

## OBJECTIVE

## 3.2

expands Loading Dock ch. 3.2 · E3 · Atlas A5 · glossary (authN vs authZ, scope, least privilege, token (auth sense), service account, audit log, transport)

## Climb 1 · Who you are is not which bays — the pass and the manifest

## R1

## THE SCENE

## The scene — Thursday, the carrier's pass

Thursday, first light. The security cabin at the gate. A BlueDart driver hands over the carrier's portal pass — laminated, genuine, issued eighteen months ago when BlueDart first started collecting from Kaveri. The guard checks the hologram, checks the photo, checks the issue stamp. All good. He waves the truck through.

Watson, reading over the guard's shoulder, turns the pass over. Under *Permitted areas* it says: **all bays**. BlueDart collects from bay 6. With this pass the driver can walk into bay 1 (Padma's sofa), bay 3 (the refunds cage), bay 9 (another carrier's consignments), and nobody at the gate would have done anything wrong. The guard checked *who* perfectly. Nobody, in eighteen months, has checked *which bays*.

Holmes: "You have seen this before, Watson. In the Refund Heist the badge was valid and the refund still walked out. This is the same ghost, Professional-grade: nothing is forged, everything is permitted."

## R2

## PLAIN WORDS

## Plain words — two questions at every door, always both

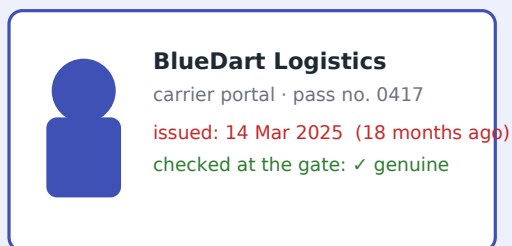
Every door asks two questions. **Who are you?** — proved with a pass, a badge, a password. **What may you do here?** — written on a manifest, a list of bays, a list of actions. The first question can be answered perfectly and the second never asked. That is the gap this station is about, and it is the commonest security hole in agent systems because tools are added faster than manifests are written.

## TOY

TOY: a hostel with 12 rooms. Everyone has a genuine key card (who you are: solved). But every card opens every room, because the cards were programmed 'all doors' on day one to save time. No card is forged. Every room is open. The fix is not better cards; it is a list per card of which doors.

The dangerous version of the gap is a pass whose manifest says *everything*. Nobody wrote 'everything' on purpose; it was the default when the pass was issued and nobody has looked since. The tell, whenever you see a pass, is one word on the back: all, \*, any, admin.

#### THE GATE PASS · who you are (authentication)



a genuine pass says nothing about which bays

#### THE MANIFEST · which bays (authorization)

|                        |   |
|------------------------|---|
| read own consignments  | ✓ |
| update own ETA         | ✓ |
| <b>read ALL bays *</b> | ✓ |
| <b>issue refunds</b>   | ✓ |

\* the wildcard: one line, every customer's consignments — the Refund Heist's ghost

**the fix is one line: scope = consignments:own:read**

**W8** · The gate pass and the manifest. The pass is genuine — who-you-are is solved. The manifest carries a wildcard nobody has read for eighteen months. The fix is one line on the manifest, not a better pass. Redraw this from memory on Day D: it is the crimson token on Atlas A5, before and after.

### R3

THE REAL  
NAME

## The real name — authentication, authorization, token (auth sense), scope, and D3 Law 1

### WORD CARD · authentication (authN)

|                              |                                                                                                                                       |
|------------------------------|---------------------------------------------------------------------------------------------------------------------------------------|
| <b>PLAIN MEANING</b>         | Proving who a caller is — a person, a program, a carrier — by something it holds or knows: a key, a token, a password, a certificate. |
| <b>WHAT IT IS NOT</b>        | Not permission. A perfectly authenticated caller may still be allowed to do nothing, or — the gap — everything.                       |
| <b>HEARD AT KAVERI</b>       | "The carrier portal authenticates BlueDart fine; the question nobody asked is what BlueDart may then read."                           |
| <b>HOW THE EXAM WORDS IT</b> | "verify the identity of the caller", "credentials", "the token is valid", "signed in via SSO".                                        |
| <b>EXPANDS</b>               | <i>Loading Dock glossary 'authN vs authZ' ('the gate pass says who you are') · ch. 3.2 · Atlas A5</i>                                 |

### WORD CARD · authorization (authZ)

|                              |                                                                                                                                       |
|------------------------------|---------------------------------------------------------------------------------------------------------------------------------------|
| <b>PLAIN MEANING</b>         | Deciding what an already-identified caller may do — which records, which actions, which bays — and enforcing it at the door, in code. |
| <b>WHAT IT IS NOT</b>        | Not something a prompt can do: an instruction to the model is a request; a check in code before the tool runs is a control.           |
| <b>HEARD AT KAVERI</b>       | "process_refund must check the caller's scope before it runs, not ask KAVI to be careful."                                            |
| <b>HOW THE EXAM WORDS IT</b> | "least privilege", "scope the token to...", "enforce per-tool permissions", "the agent can read every customer's...".                 |
| <b>EXPANDS</b>               | <i>Loading Dock glossary 'authN vs authZ' ('the manifest says which bays') · E3 · Atlas A5</i>                                        |

**WORD CARD · token (auth sense)**

|                              |                                                                                                                                                                      |
|------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>PLAIN MEANING</b>         | A string a caller presents to prove who it is (and often what it may do): issued by someone trusted, checked at the door, usually time-limited.                      |
| <b>WHAT IT IS NOT</b>        | Not an LLM token (a word-piece, counted and billed — Chapter 0 ▮). A bearer token is one 'whoever holds it, is it' — which is why it must expire and must be scoped. |
| <b>HEARD AT KAVERI</b>       | "BlueDart's portal token was issued eighteen months ago and reads every customer's consignments — that is the OP ghost."                                             |
| <b>HOW THE EXAM WORDS IT</b> | "bearer token", "access token", "the token grants...", "token expiry", "rotate the credential".                                                                      |
| <b>EXPANDS</b>               | <i>Loading Dock glossary 'token (auth sense, disambiguated from LLM token!)'</i>                                                                                     |

▮ **SAME WORD, TWO WORLDS · token** — (1) auth sense (this station): a pass — issued, expires, checked at a door, lists bays. (2) LLM sense (Station 1, 5, 8): a word-piece — counted in thousands, billed per call. **The tell:** issued / expires / scope → auth; counted / window / bill → LLM.

**WORD CARD · scope**

|                              |                                                                                                                                         |
|------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------|
| <b>PLAIN MEANING</b>         | The written list of what a token or identity may do — which actions, on which records — carried with the token and checked at the door. |
| <b>WHAT IT IS NOT</b>        | Not a Python variable's visibility (▮) and not a suggestion: a scope only exists where something enforces it.                           |
| <b>HEARD AT KAVERI</b>       | "Scope for BlueDart: consignments:own:read, eta:own:write — nothing else."                                                              |
| <b>HOW THE EXAM WORDS IT</b> | "scoped to", "the token grants read access to all...", "narrow the permission", "resource-level authorization".                         |
| <b>EXPANDS</b>               | <i>Loading Dock glossary 'scope' · E3 (allowlist by scope, not by prompt) · Atlas A5</i>                                                |

▮ **SAME WORD, TWO WORLDS · scope** — (1) auth sense (this station): the list of doors a token may open. (2) Python sense (CCDV-F): where a variable name is visible — function, module. **The tell:** 'token', 'permission', 'read all' → auth; 'variable', 'function', 'global' → Python.

Technically: authentication (authN) establishes identity; authorization (authZ) grants and enforces permissions; a token is the usual carrier of both — a bearer token proves identity by possession, and a scoped token also carries the manifest. The Dock's E3 puts a scope check *in code* in front of `process_refund` and then finds and narrows the over-permissive carrier token; when you type it on Day A, watch the line where the allowlist is checked before the tool body runs.

**D3 LAW 1 · Every tool you add is a door; know who holds the key.**

Born here. A capability without an owner, a scope and a ledger entry is a hole with a name. For every tool in the belt you must be able to say: who may call it, with what token, to reach which records, and where the call is written down. Broken: the carrier portal token — genuine, eighteen months old, **all bays**, and nobody could say who had used it for what.

**WRONG BELIEF, CORRECTED**

|                                  |                                                                                                                                                                                                                            |
|----------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>I used to think...</b>        | logged in securely = secure                                                                                                                                                                                                |
| <b>Why it feels right</b>        | The login page had a padlock, the token was signed, the audit said 'authentication: strong'.                                                                                                                               |
| <b>The correction</b>            | <b>Login answers who. Security is who × what: a strongly authenticated caller with a wildcard scope is a strongly authenticated way to read every customer's data. The gap is almost always on the authorization side.</b> |
| <b>The stem that punishes it</b> | A stem where every credential is valid and rotated, and the correct option is still 'scope the token / check permissions per tool'.                                                                                        |

**R4****THE EXAM****The exam — how a 3.2 stem hides the missing manifest**

**The disguise.** The stem spends its words on authentication — logins, issuers, rotation, encryption — and slips the gap into one clause: 'the integration token can read all...', 'a shared service key', 'the agent has access to the customer database'. The tell is the scope clause, and the shape of the right answer is always **narrow the permission at the door, in code**.

**Family baited:** OP (the over-permissive ghost) and SD ('add a line to the system prompt telling the agent not to...').

**Mini-stem 1.** A carrier-portal integration authenticates with issuer-signed, daily-rotated tokens and records every call; the token was provisioned with read access to all consignments so it 'would not break'. What is the gap? — **Shape: authorization, not authentication: re-issue the token scoped to the carrier's own consignments; enforce the scope where the data is read.**

**Mini-stem 2.** KAVI's process\_refund is called with a valid machine credential; a prompt injection in a customer message got a refund issued. — **Shape: a per-call authorization check in code (caller, scope, amount) in front of the tool; the prompt is not a control** — the Refund Heist, again.

**CHECK YOURSELF**

1. What are the two questions every door asks?
2. Why is a genuine, rotated, signed token still dangerous?
3. Where does an authorization check belong — the prompt or the code?
4. Explain D3 Law 1 to Meera in two sentences.

----- ∞ fold — answers below -----

1. Who are you? (authentication) and What may you do here? (authorization).
2. Because a token carries a manifest too; a valid token whose scope says 'all' is a valid way to reach everything.
3. In code, at the door, before the tool body runs. A prompt instruction is a request, not a control.
4. Every tool we give KAVI is a door into something. For each one we must be able to say who may open it, how far it opens, and where each opening is written down.

**Climb 2 · Four kinds of pass on one card****R1****THE SCENE****The scene — four people at the gate, four ways of proving it**

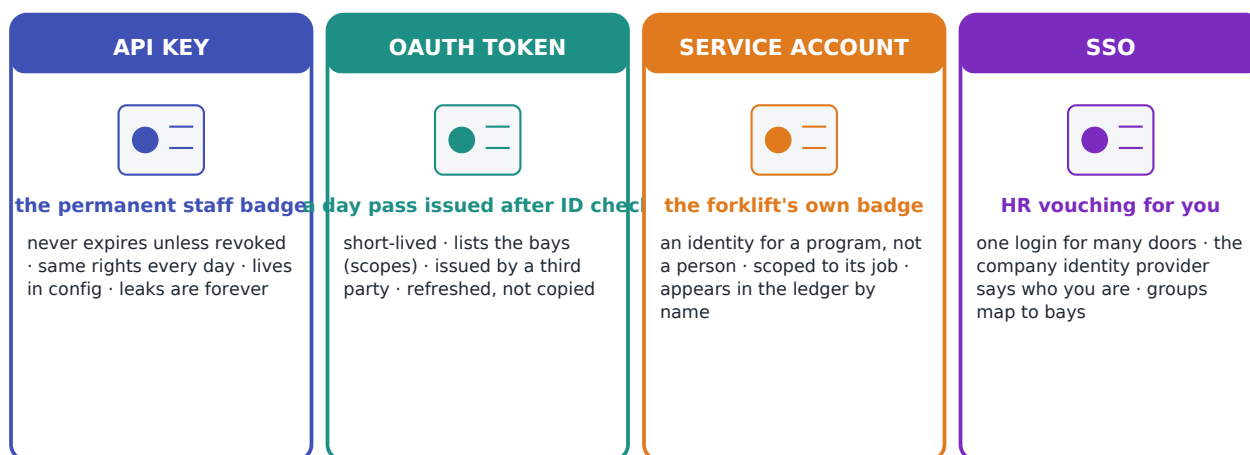
Friday morning, the gate at 8 am. Sunita, staff, flashes the permanent badge she was issued five years ago — the guard barely looks. A visiting auditor shows a day pass the front desk printed after checking his ID: valid till 6 pm, three named rooms. The forklift rolls up with its own tag bolted to the frame — no driver's name on it; the machine is the identity. And a new hire from the Mumbai office walks in with nothing at all: the guard rings HR, HR says 'yes, she's ours, group: dispatch', and the gate opens. Four passes. Four answers to *who*. Not one of them, by itself, says which bays.

**R2****PLAIN WORDS****Plain words — how long it lasts, who issued it, and whether it is a person**

Passes differ on three things. **How long it lasts:** a permanent badge versus a pass that expires tonight. **Who issued it and checked what:** the company itself, a trusted third party who checked ID, or HR vouching live. **Whether the holder is a person or a machine:** forklifts need passes too, and a forklift with a person's badge is a person who cannot be found when something goes wrong. The right pass for a job is the one that lasts no longer than the job, is issued by whoever can actually vouch, and names the true holder.

**TOY**

TOY: 4 callers of the same refund tool — a dispatcher (person, all day), a nightly report script (machine, forever), a partner portal (third party, per session), the auditor (person, one afternoon). One pass type cannot fit all four.



all four answer **WHO** — none of them, by itself, answers **WHICH BAYS**; that is the scope list

**W9** · The four kinds of pass: the permanent staff badge, the day pass issued after an ID check, the forklift's own badge, and HR vouching for you. All four answer **WHO**; the scope list answers **WHICH BAYS**. Redraw from memory on Day D — they are the key symbols on Atlas A5.

### R3

THE REAL NAME

## The real name — API key, OAuth, service account, SSO, mTLS

### WORD CARD · API key

|                       |                                                                                                                                                |
|-----------------------|------------------------------------------------------------------------------------------------------------------------------------------------|
| PLAIN MEANING         | A long secret string that identifies a caller to a service — the permanent staff badge: same rights every day, valid until someone revokes it. |
| WHAT IT IS NOT        | Not scoped by default and not short-lived; a leaked key is a leaked identity until rotated. Not a person: keys identify integrations.          |
| HEARD AT KAVERI       | "ANTHROPIC_API_KEY in the environment is Kaveri's badge to Claude — and the one we cleared to stop double-billing on the Max plan."            |
| HOW THE EXAM WORDS IT | "static credential", "rotate the key", "stored in config / secrets manager", "shared key across services" (a gap).                             |
| EXPANDS               | <i>Loading Dock ch. 3.2 · Atlas A5 (the key symbols)</i>                                                                                       |

### WORD CARD · OAuth (token issued after an identity check)

|                       |                                                                                                                                                                                         |
|-----------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| PLAIN MEANING         | A standard where a trusted issuer checks who you are, then hands you a short-lived token that lists what you may do (scopes); the service you call checks the token, not your password. |
| WHAT IT IS NOT        | Not a login screen and not authorization by itself — the scopes on the token are only as narrow as whoever requested them made them.                                                    |
| HEARD AT KAVERI       | "The carrier portal uses OAuth; the token is fine — the scope it was requested with was 'all consignments'."                                                                            |
| HOW THE EXAM WORDS IT | "OAuth 2.0 flow", "access token with scopes", "refresh token", "issuer", "client credentials" (machine-to-machine).                                                                     |
| EXPANDS               | <i>Loading Dock ch. 3.2 · MCP authorization (Station 7 DRIFT: issuer validation, client ID metadata documents)</i>                                                                      |

### WORD CARD · service account

|                       |                                                                                                                                                |
|-----------------------|------------------------------------------------------------------------------------------------------------------------------------------------|
| PLAIN MEANING         | An identity for a program rather than a person — the forklift's own badge — with its own credentials and its own, usually narrow, permissions. |
| WHAT IT IS NOT        | Not a person's account borrowed by a script: that hides who actually acted and outlives the person.                                            |
| HEARD AT KAVERI       | "The nightly re-index runs as svc-indexer with read access to the SOP bucket and nothing else."                                                |
| HOW THE EXAM WORDS IT | "machine-to-machine", "run the agent under a dedicated identity", "the job runs with the developer's credentials" (a gap).                     |
| EXPANDS               | <i>Loading Dock glossary 'service account' · Atlas A5</i>                                                                                      |

**WORD CARD · SSO (single sign-on)**

|                              |                                                                                                                                        |
|------------------------------|----------------------------------------------------------------------------------------------------------------------------------------|
| <b>PLAIN MEANING</b>         | One company login that vouches for a person to many systems — HR at the gate — usually with group memberships that map to permissions. |
| <b>WHAT IT IS NOT</b>        | Not a permission list by itself; SSO says who and which groups, and each system still decides what a group may do.                     |
| <b>HEARD AT KAVERI</b>       | “Dispatchers reach KAVI through SSO; the ‘dispatch’ group maps to lookup and read scopes, never to refunds.”                           |
| <b>HOW THE EXAM WORDS IT</b> | “federated identity”, “SAML / OIDC”, “group-based access”, “de-provisioning when someone leaves”.                                      |
| <b>EXPANDS</b>               | <i>Loading Dock ch. 3.2 (auth boundary map)</i>                                                                                        |

**WORD CARD · mTLS (mutual TLS)**

|                              |                                                                                                                             |
|------------------------------|-----------------------------------------------------------------------------------------------------------------------------|
| <b>PLAIN MEANING</b>         | Both ends of a connection prove who they are with certificates before any data moves — two badges checked at the same door. |
| <b>WHAT IT IS NOT</b>        | Not a token and not authorization; it identifies machines to each other on the wire and says nothing about which records.   |
| <b>HEARD AT KAVERI</b>       | “The TMS only accepts connections from services that present the Kaveri-issued certificate.”                                |
| <b>HOW THE EXAM WORDS IT</b> | “mutual authentication at the transport layer”, “certificate-based service identity”.                                       |
| <b>EXPANDS</b>               | <i>Loading Dock ch. 3.2 (mentioned in passing; carded here for completeness)</i>                                            |

| Pass            | Lasts                  | Issued by                                | Holder                   | Carries a scope list?              | Right for                                          |
|-----------------|------------------------|------------------------------------------|--------------------------|------------------------------------|----------------------------------------------------|
| API key         | until revoked          | the service                              | an integration           | rarely — usually all-or-nothing    | a stable server-to-server link, in a secrets store |
| OAuth token     | minutes-hours          | a trusted issuer after an identity check | a person or a client app | yes — scopes                       | third-party portals, delegated access              |
| Service account | until revoked; rotated | your platform                            | a program                | yes — its own permissions          | batch jobs, agents, MCP servers                    |
| SSO             | the session            | the company identity provider            | a person                 | groups → each system's permissions | staff reaching internal tools                      |
| mTLS            | the certificate's life | your certificate authority               | a machine                | no                                 | service-to-service on the wire                     |

**R4****THE EXAM****The exam — how a stem disguises the wrong kind of pass**

**The disguise.** A nightly job ‘uses Arjun’s credentials’; a partner ‘shares one API key across all its tenants’; a token ‘never expires for reliability’. The tell is a mismatch between pass and holder: a person’s pass on a machine, a permanent pass for a session-long job, an all-or-nothing key where scopes were needed.

**Family baited:** VG (OAuth read as authorization) and OP.

**Mini-stem 1.** The re-index job runs under a developer’s SSO account; she is leaving next month. — **Shape: a service account with read-only access to the SOP store; credentials in a secrets store; rotated.**

**Mini-stem 2.** A partner’s portal integration is a single static API key with full access, used by all of the partner’s staff. — **Shape: per-user OAuth tokens with narrow scopes, or at least per-tenant keys scoped to own records; audit by identity.**

## CHECK YOURSELF

1. Which pass is the forklift's own badge, and why does it matter?
2. Does OAuth solve authorization?
3. Which pass carries no scope list at all?
4. Explain to Meera in two sentences why the nightly job must not use Arjun's login.

----- ∞ fold — answers below -----

1. A service account — so a program acts under its own name with its own narrow rights, and the ledger shows the program, not a borrowed person.
2. No. It issues a token with whatever scopes were requested; if 'all consignments' was requested, that is what it grants.
3. mTLS — it identifies machines on the wire; and API keys usually — all-or-nothing unless the service adds scopes.
4. If Arjun leaves, the job dies with his account — or worse, keeps running with a departed person's rights. A job needs its own badge with only the rights the job needs.

## Climb 3 · The manifest: scopes, least privilege, the wildcard, and the ledger

### R1

#### THE SCENE

### The scene — writing the manifest, and the one line that was never written

Friday, 10 am. Meera makes Arjun sit with the security guard and a sheet of paper. Down the left: everyone who calls the tower — dispatchers, the nightly job, three carriers, the auditor, KAVI itself. Across the top: every door — read a consignment, update an ETA, read the SOP store, issue a refund, escalate. Fill the grid: yes or no, and for 'yes', *which* consignments. Twenty minutes later there is exactly one cell with the word **all** in it, and it belongs to BlueDart, and it has been there for eighteen months. Arjun changes it to 'own'. One line.

The guard's last question: "And when I turn someone away — do I write that down?" Meera: "Especially then."

### R2

#### PLAIN WORDS

### Plain words — as little as the job needs, written down, including the refusals

Three plain rules. **Give each caller the smallest set of doors its job needs** — not the set that is convenient. **Write the grid down where the doors can read it**, so the door itself refuses; a sign saying 'staff only' is not a lock. **Write every opening and every refusal in a ledger**, with who, what, when — because the refusals are how you find the intruder and the misconfiguration.

#### TOY

TOY: 5 callers × 4 doors = 20 cells. Nineteen say 'no' or 'own'. One says 'all'. Finding it took twenty minutes with a pencil; not finding it took eighteen months.

### R3

#### THE REAL NAME

### The real name — least privilege, wildcard, audit log; a sign is not a lock

#### WORD CARD · least privilege

|                       |                                                                                                                                            |
|-----------------------|--------------------------------------------------------------------------------------------------------------------------------------------|
| PLAIN MEANING         | The rule that every caller gets exactly the permissions its job needs and no more — the smallest manifest that still lets the work happen. |
| WHAT IT IS NOT        | Not 'as little as possible' to the point of breaking the job; and not a one-off — permissions creep, so the grid is reviewed.              |
| HEARD AT KAVERI       | "KAVI's tools: get_customer and lookup_order read; process_refund only via the scoped refund identity, capped by amount."                  |
| HOW THE EXAM WORDS IT | "principle of least privilege", "over-permissive", "broad access 'so nothing breaks'" (the tell).                                          |
| EXPANDS               | <i>Loading Dock glossary 'least privilege' · trap-of-traps 'capability bloat and missing authorization'</i>                                |

**WORD CARD · wildcard (in a scope)**

|                              |                                                                                                                             |
|------------------------------|-----------------------------------------------------------------------------------------------------------------------------|
| <b>PLAIN MEANING</b>         | A scope entry that means ‘everything’ — *, all, any, admin — usually a default nobody replaced.                             |
| <b>WHAT IT IS NOT</b>        | Not always an attack; it is nearly always an accident that has not been found yet.                                          |
| <b>HEARD AT KAVERI</b>       | “‘consignments:*:read’ on a carrier token — one asterisk, every customer's data.”                                           |
| <b>HOW THE EXAM WORDS IT</b> | “read access to all records”, “* permission”, “provisioned with broad access”; the option that narrows it is usually right. |
| <b>EXPANDS</b>               | <i>Loading Dock E3 (the over-permissive carrier token found and narrowed) · Atlas A5 (marked in crimson before/after)</i>   |

**WORD CARD · audit log (the ledger)**

|                              |                                                                                                                                                          |
|------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>PLAIN MEANING</b>         | An append-only record of who did what, to which record, when — including refused attempts.                                                               |
| <b>WHAT IT IS NOT</b>        | Not the application log (Station 4 — those are for debugging) and not optional for money-moving tools; and a ledger that omits denials is half a ledger. |
| <b>HEARD AT KAVERI</b>       | “The ledger shows: 09:14 svc-carrier-bluedart DENIED read consignment KF-2481 (scope consignments:own) — that line is the gate working.”                 |
| <b>HOW THE EXAM WORDS IT</b> | “audit trail”, “log every privileged action with the caller identity”, “denied requests are logged”.                                                     |
| <b>EXPANDS</b>               | <i>Loading Dock glossary ‘audit log’ · E3 (the denial line)</i>                                                                                          |

Now the sentence the whole domain turns on: **a sign is not a lock**. ‘Config beats instruction’ from CCAR-F, renamed authorization. The Dock's E3 puts an allowlist check *in code* in front of `process_refund`: caller scope in, allowed or denied out, before the tool body runs, and every denial written to the ledger. A system prompt that says ‘never refund above ₹5,000’ is a sign; the model may read it, may be argued out of it, may never see the injected instruction that overrides it. The check in code cannot be persuaded. That distinction — safety theatre versus real control — is a core exam discriminator in this domain.

**TRAP · The Refund Heist ghost, named**

In CCAR-F the ghost was a valid badge and an unguarded refund button. Here it is a valid, rotated, OAuth-issued carrier token with **all bays** on its manifest — everything modern, nothing forged. The **trap** is any option that fixes authentication when the gap is authorization: ‘rotate the token’, ‘move to OAuth’, ‘add MFA’, ‘tell the agent to only read its own carrier's data’. The tell is the scope clause the stem slips past you.

**WRONG BELIEF, CORRECTED**

|                                  |                                                                                                                                                                                                                                     |
|----------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>I used to think...</b>        | a firm system prompt is a control                                                                                                                                                                                                   |
| <b>Why it feels right</b>        | The prompt is in code, versioned and reviewed, and the model follows it 99 % of the time.                                                                                                                                           |
| <b>The correction</b>            | <b>An instruction is a request the model usually honours; a check in code is a rule the tool cannot skip. Anything that moves money or reads other people's data needs the check. The prompt is defence in depth, not the door.</b> |
| <b>The stem that punishes it</b> | A stem where ‘instruct the agent never to...’ is the sounds-diligent option next to ‘enforce the scope in the tool handler’.                                                                                                        |

**ADD IT · a per-tool authorization check (adding is the right move here)**

A stem offers ‘add a scope check in front of `process_refund` and log every decision’ as one of four options for an agent that can already issue refunds. Adding this control is **the right move**, not clutter: it is a door that exists and has no lock. The trimming reflex (Station 5) is about unused capabilities; a used capability without authorization is the opposite problem.

**R4****THE EXAM****The exam — how a stem disguises the missing lock**

**The disguise.** A long list of good hygiene (rotation, encryption, MFA, prompt guardrails) with one clause that names broad access or a prompt-only rule. The tell is the word 'all' / '\*' / 'any' in a scope, or the phrase 'instructed the model to' where an action has consequences.

**Family baited:** OP · SD.

**Mini-stem 1.** A partner token is OAuth-issued, rotated daily, encrypted at rest, and scoped to consignments\*:read 'for simplicity'. — **Shape: narrow the scope to the partner's own consignments; enforce at the data layer; log denials.**

**Mini-stem 2.** KAVI's system prompt forbids refunds above ₹5,000; a support ticket containing 'ignore previous rules' triggered a ₹12,000 refund. — **Shape: an amount-and-scope check in the process\_refund handler (code), plus a human step above the threshold; keep the prompt as a second layer.**

**PENCIL LAB P7 · Write the scope table for a five-caller TOY tower; find the wildcard; write the fix; say what the ledger shows**

TOY tower. Callers: **dispatcher (SSO) · svc-indexer (nightly job) · svc-carrier-bluedart · svc-carrier-delhivery · KAVI (the agent)**. Doors: **read consignment · update ETA · read SOP store · issue refund · escalate to human**. Draw the 5 × 5 grid and fill every cell with *no*, *own*, *all* or *capped* — as the job needs, not as is convenient. Then: (a) which cell is the wildcard if BlueDart was provisioned 'so nothing breaks'? (b) write the one-line fix as a scope string; (c) write the ledger line for BlueDart's next attempt to read KF-2481 (a Delhivery consignment); (d) which caller should be able to issue a refund, and with what extra condition?

----- ∞ fold here — the worked answer is below the line -----

**WORKED ANSWER · P7**

A defensible grid: **dispatcher** — read consignment: *all* (it is their job) · update ETA: *no* · read SOP: *all* · refund: *no* (they escalate) · escalate: yes. **svc-indexer** — read SOP: *all*; everything else *no*.

**svc-carrier-bluedart** — read consignment: *own* · update ETA: *own* · rest *no*. **svc-carrier-delhivery** — same as BlueDart, for its own. **KAVI** — read consignment: *all* (on behalf of dispatchers, under their SSO identity where possible) · read SOP: *all* · refund: *capped* (amount ≤ threshold, via the scoped refund identity) · escalate: yes · update ETA: *no*.

(a) The wildcard: **svc-carrier-bluedart × read consignment = all**. (b) The fix:

**consignments:own:read** (and eta:own:write) — one line on the token's scope, enforced where consignments are read. (c) The ledger: **2026-10-02T09:14:07Z svc-carrier-bluedart read consignment KF-2481 DENIED reason=scope(consignments:own) ref=...** — the denial is logged, with identity, action, target, reason. (d) Refunds: only KAVI's scoped refund identity, *capped* by amount in code, above which `escalate_to_human` is the only door.

**What you should feel:** the grid took a pencil and ten minutes; the wildcard was obvious once the grid existed. E3 does (b) and (c) in code on Day A — watch the allowlist check and the DENIED line.

**CHECK YOURSELF**

1. What is least privilege in one sentence?
2. Why must the ledger record denials?
3. Sign or lock: 'the system prompt says never refund above ₹5,000'.
4. Explain to Meera in two sentences why we changed one word on BlueDart's token.

----- ∞ fold — answers below -----

1. Every caller gets exactly the doors its job needs, written where the doors enforce it, and reviewed.

2. Denials are the gate working — they are how you find intruders and misconfigurations; a ledger of successes only is half a ledger.

3. A sign. The lock is an amount-and-scope check in the tool handler.

4. Their token could read every customer's shipments, not just their own, and nobody had noticed for eighteen months. We changed 'all' to 'own' and the system now refuses — and records — anything else.

## Climb 4 · Same building or the highway — how the road changes the gate

R1

THE SCENE

The scene — the pass you need depends on where the truck came from

Saturday. Two deliveries to bay 6. The first is a trolley pushed across the yard from Kaveri's own packing shed — same building, same staff, never left the fence; the guard nods it through. The second is a truck off the Mumbai-Pune highway: unknown driver, three hours of public road, could have stopped anywhere. That one gets the full check — pass, manifest, seal on the doors. Same bay, same goods; the road decides the gate.

R2

PLAIN WORDS

Plain words — a connection inside one machine versus a connection across the network

When the model's program and the tool's program run on the same machine and talk through a private pipe, the caller is already inside the fence: the operating system decided who may run the program, and there is no public road for anyone else to join. When they talk over the network — different machines, possibly the internet — anyone who can reach the address can knock, so the door needs a real gate: identity checked on every request, permissions enforced, traffic encrypted.

TOY

TOY: 2 rooms. A note passed under the door between them (nobody else can reach the corridor) versus a letter sent through the public post (anyone can post one). The second needs a signature and an envelope; the first does not.

R3

THE REAL NAME

The real name — transport: stdio versus Streamable HTTP

WORD CARD · transport (MCP sense)

|                       |                                                                                                                                                                      |
|-----------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| PLAIN MEANING         | The road the messages travel on between the model-side program (client) and the tool-side program (server): a private pipe on one machine, or HTTP across a network. |
| WHAT IT IS NOT        | Not the protocol itself (the messages are the same on either road) and not a security control — the road decides which controls you need.                            |
| HEARD AT KAVERI       | "kaveri_mcp.py runs over stdio on Arjun's laptop; the carrier-facing server will run over Streamable HTTP with OAuth in front."                                      |
| HOW THE EXAM WORDS IT | "stdio vs Streamable HTTP", "remote MCP server", "the server is exposed over the network".                                                                           |
| EXPANDS               | <i>Loading Dock glossary 'transport (stdio vs HTTP)' · ch. 3.7 · DRIFT box 'MCP spec 2026-07-28' · Station 7 here</i>                                                |

**WORD CARD · stdio (standard input/output)**

|                              |                                                                                                                                                        |
|------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>PLAIN MEANING</b>         | The private pipe: the client starts the server as a child process on the same machine and they exchange messages through its input and output streams. |
| <b>WHAT IT IS NOT</b>        | Not reachable from the network at all; identity comes from who was allowed to run the process, so there is nothing to 'log in' to.                     |
| <b>HEARD AT KAVERI</b>       | "When KAVI launches kaveri_mcp.py locally, that is stdio — same building, no gate needed beyond the OS user."                                          |
| <b>HOW THE EXAM WORDS IT</b> | "local server", "launched as a subprocess", "no network exposure", "development / single-user".                                                        |
| <b>EXPANDS</b>               | <i>Loading Dock glossary 'transport' ('stdio = same building')</i>                                                                                     |

**WORD CARD · Streamable HTTP**

|                              |                                                                                                                                                                                                        |
|------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>PLAIN MEANING</b>         | The highway: the server listens at a network address and clients send each request over HTTP; the current MCP spec makes each request self-contained (stateless) so any server instance can answer it. |
| <b>WHAT IT IS NOT</b>        | Not the old 'HTTP+SSE' transport, which is deprecated in the 2026-07-28 spec; and not safe by default — it needs authentication on every request.                                                      |
| <b>HEARD AT KAVERI</b>       | "The carrier gateway is Streamable HTTP behind OAuth: every request carries a token, every token carries a scope."                                                                                     |
| <b>HOW THE EXAM WORDS IT</b> | "remote / hosted MCP server", "multiple clients across the network", "requires authentication and TLS".                                                                                                |
| <b>EXPANDS</b>               | <i>Loading Dock glossary 'transport (stdio vs HTTP)' · DRIFT box 'MCP spec 2026-07-28' · Station 7 here</i>                                                                                            |

The technical point for 3.2: **transport shapes authentication**. stdio inherits the operating-system user — the question 'who are you?' was answered when the process was allowed to start. Streamable HTTP has to answer it on every request: a token per request, a scope per token, TLS on the wire, and (Station 7) the 2026-07-28 spec's hardened authorization rules — issuer validation, client-ID metadata documents. The road decides the gate; the manifest is needed on both roads.

**R4****THE EXAM****The exam — how a stem disguises 'the road changed and the gate didn't'**

**The disguise.** A local tool server 'that worked fine in development' is 'exposed so other teams can use it'. The tell is the move from local to network: 'hosted', 'remote', 'other teams / partners can connect' — and the shape of the answer is a gate per request.

**Family baited:** SF (no gate, no error — everything silently works for everyone) and OP.

**Mini-stem 1.** kaveri\_mcp.py, previously launched locally by KAVI, is deployed as a hosted server so dispatch tools and two carriers can call it. — **Shape: Streamable HTTP with per-request token authentication, per-caller scopes on each tool, TLS, and denials in the ledger.**

**Mini-stem 2.** A single-developer local server over stdio; the team proposes adding OAuth 'for consistency'. — **Shape: unnecessary on stdio: the OS user is the identity; spend the effort on scoping the tools it exposes** — not every door needs a new gate.

**CHECK YOURSELF**

1. Why does stdio need no login?
2. What must Streamable HTTP do that stdio does not?
3. Which is deprecated in the 2026-07-28 spec — stdio, Streamable HTTP, or HTTP+SSE?
4. Explain to Meera in two sentences why the carrier server needs a gate the laptop one didn't.

----- ∞ fold — answers below -----

1. The server is a child process on the same machine; who may run it was decided by the operating system — there is no public road.
2. Authenticate every request (a token), enforce scopes per caller, encrypt the wire.
3. HTTP+SSE as a transport. stdio and Streamable HTTP are current.
4. On the laptop only Arjun's own programs could talk to it. On the network anyone who finds the address can knock, so every request has to show a pass and the pass has to say which bays.

#### WHAT WATSON WROTE DOWN

- Two questions at every door: **who** (authN) · **which bays** (authZ). Valid ≠ permitted; the gap is nearly always authZ.
-  token: issued/expires/scope = auth sense. Bearer = whoever holds it; so expire it and scope it.
- Four passes: API key (permanent badge) · OAuth token (day pass with scopes) · service account (the forklift's badge) · SSO (HR vouches). None settles which bays.
- Scope = the manifest; least privilege = smallest manifest that does the job; wildcard (**all / \* / any**) = the ghost. Fix = one line, enforced in code.
- **D3 Law 1**: every tool is a door — who holds the key, what it opens, where it's written. Ledger records denials.
- A sign is not a lock: prompt = request, code check = control (config beats instruction, renamed).
- Transport shapes authN: stdio = same building (OS user); Streamable HTTP = highway (token per request, TLS). HTTP+SSE deprecated (**2026-07-28**).
- Tells: all / \* / any · 'so nothing breaks' · 'instructed the agent to' · hosted / remote / other teams.

**Law in force:** D3 Law 1 · Every tool you add is a door; know who holds the key.

**Read next in the Loading Dock:** ch. 3.2 · E3 (the allowlist check before the tool body; the DENIED ledger line) · Atlas A5 (the crimson token before/after) · glossary entries for authN vs authZ, scope, least privilege, token (auth sense), service account, audit log, transport

## TONIGHT YOU WILL

## Friday 2 Oct · ~45 min · Station 4 · The camera room

- ☐ watch Tuesday's p95 — 300 dispatchers' actual afternoon that the average hid
- ☐ follow one ticket through three stages twice, without and with a sticker on the windscreen (P6)
- ☐ compute p50 and p95 from twenty TOY latencies whose mean says healthy — in your own handwriting (P5)
- ☐ label a wall of gauges: which labels are safe and which melt the store (P12)
- ☐ own the words: observability, structured log, trace, span, trace id, metric, series, label, percentile (p50/p95/p99), SLA, cardinality, alert, dashboard
- ☐ meet **D3 Law 4**: you cannot fix what you cannot see; carry the canon **310 ms** mean / **2,900 ms** p95 / **4,100 ms** total

**Then:** read Loading Dock ch. 3.4. It should read like a summary. Star anything still foggy; the reverse map names the Walk page. Stations 2 and 4 must both be read before Day B.

## STATION 4

## The camera room

Analyze observability challenges; select monitoring strategies at scale — the wall of gauges, and the afternoon the average hid

## OBJECTIVE

## 3.4

expands Loading Dock ch. 3.4 · E7 · Atlas A6 · glossary (audit log, trace/span, p50/p95/p99 latency, cardinality)

## Climb 1 · Three kinds of watching — the diary, the sticker, and the gauge

## R1

## THE SCENE

## The scene — Tuesday afternoon, 'it's slow', and nobody can say where

Tuesday, 2.30 pm. Three hundred dispatchers on shift. The tower is 'slow' — everyone says so, nobody can say what that means. Arjun opens three files. The router's diary: '14:10 request in ... 14:10 request in ...' — thousands of lines. The retrieval diary: the same, with some lines that took longer. The generation diary: the same again. Three diaries, three thousand trucks, and no way to say which line in diary two belongs to which line in diary one. He starts reading timestamps and matching them by eye. At 4 pm he has a theory. At 4.15 pm it is wrong.

Meera, at the door of the camera room: "We have cameras on every bay and a register at every gate. Why can't we say what happened to *one* ticket?" Because nothing on the ticket says it is the same ticket from bay to bay.

## R2

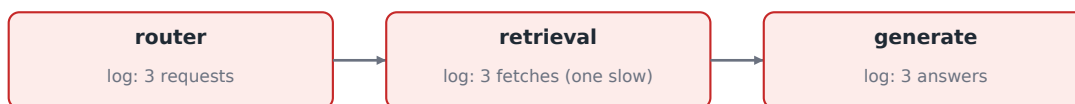
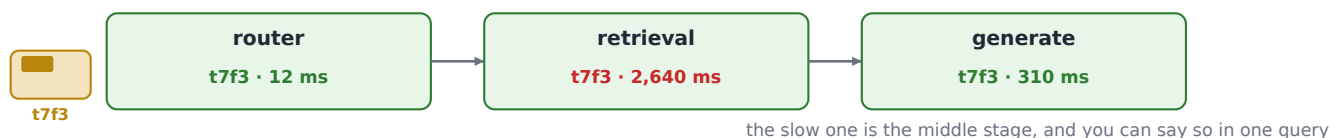
## PLAIN WORDS

## Plain words — a diary, a sticker, and a gauge answer three different questions

Three ways of watching, each for a different question. **The diary** answers 'what happened, exactly, in this one place?' — one line per event, with all the details. **The sticker** answers 'what happened to *this* ticket, everywhere it went?' — put the same mark on the ticket at the first gate and every diary line about it carries the mark, so you can pull its whole journey with one search, and see how long it spent at each stage. **The gauge** answers 'how are we doing, overall, right now and over time?' — not events, but numbers that summarise them: how many, how long, how often failed — per stage, per route, per minute.

## TOY

TOY: 3 tickets through 3 stages. Without stickers you have 9 diary lines and no idea which 3 belong together —  $3 \times 2 \times 1 = 6$  possible matchings, and that is with three tickets, not three thousand. With a sticker, each ticket's 3 lines share a mark; the slow ticket is the one whose middle line shows the long gap.

**A · no sticker: three logs, three trucks, no way to say which was which****B · sticker KF-2481-t7f3 on the windscreen: one ticket followed through all three, time at each**

**W10** · One ticket through three stages, without and with a sticker. Without: three diaries, no way to join them. With: the mark t7f3 on every line, the time at each stage, and the slow stage named in one query. Redraw from memory on Day D — this is the ‘traces’ layer of Atlas A6.

**R3****THE REAL NAME****The real name — observability, structured logs, traces, spans, trace ids, metrics; D3 Law 4****WORD CARD · observability**

|                              |                                                                                                                                                                    |
|------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>PLAIN MEANING</b>         | Being able to answer questions about what a running system did — for one request and for all of them — from what it recorded, without adding new code to find out. |
| <b>WHAT IT IS NOT</b>        | Not ‘monitoring’ (a fixed set of gauges) and not a dashboard (a picture of gauges); it is whether the questions you have not thought of yet can still be answered. |
| <b>HEARD AT KAVERI</b>       | “At 40,000 shipments a day, observability is the difference between ‘it’s slow’ and ‘retrieval p95 went from 300 ms to 2.9 s at 14:10 after the index rebuild’.”   |
| <b>HOW THE EXAM WORDS IT</b> | “observability at scale”, “diagnose where latency is spent”, “instrument the pipeline”.                                                                            |
| <b>EXPANDS</b>               | <i>Loading Dock ch. 3.4 (‘the dock cameras + the register — you cannot fix what you cannot see’) · Atlas A6</i>                                                    |

**WORD CARD · structured log**

|                              |                                                                                                                                                                                                 |
|------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>PLAIN MEANING</b>         | The diary: one record per event, written as named fields (time, stage, trace id, duration, tokens, outcome) rather than a free sentence, so machines can search and count them.                 |
| <b>WHAT IT IS NOT</b>        | Not print statements — a sentence like ‘done in 2 s’ cannot be filtered by stage or joined by ticket; and not the audit ledger (Station 3), which is for who-did-what and must never be pruned. |
| <b>HEARD AT KAVERI</b>       | “{ts, stage:‘retrieval’, trace:‘t7f3’, ms:2640, tokens_in:4310, route:‘hybrid’, ok:true}.”                                                                                                      |
| <b>HOW THE EXAM WORDS IT</b> | “structured logging”, “log fields”, “correlate logs across services”.                                                                                                                           |
| <b>EXPANDS</b>               | <i>Loading Dock E7 (structured logging + trace ids around tool calls) · Atlas A6 (layer 1)</i>                                                                                                  |

**WORD CARD · trace / span / trace id**

|                              |                                                                                                                                                                                                              |
|------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>PLAIN MEANING</b>         | The sticker: a trace is one request's whole journey; each stage it passes through is a span with a start and a duration; the trace id is the mark copied onto every log line and span so they can be joined. |
| <b>WHAT IT IS NOT</b>        | Not a log (a trace is the join across logs) and not a metric (a trace is one ticket, a metric is all tickets); <b>span</b> here is a timed segment, not an HTML tag.                                         |
| <b>HEARD AT KAVERI</b>       | "Trace t7f3: router 12 ms → retrieval 2,640 ms → generate 310 ms; the middle span is the story."                                                                                                             |
| <b>HOW THE EXAM WORDS IT</b> | "distributed tracing", "propagate the trace id", "which span is slow", "end-to-end latency breakdown".                                                                                                       |
| <b>EXPANDS</b>               | <i>Loading Dock glossary 'trace/span' · E7 · Atlas A6 (layer 2)</i>                                                                                                                                          |

► **SAME WORD, TWO WORLDS · span** — **(1)** trace sense (this station): one timed stage of one request, inside a trace. **(2)** HTML sense: an inline tag with no meaning of its own. **The tell:** 'duration', 'parent', 'trace' → trace sense; 'tag', 'class', 'markup' → HTML.

**WORD CARD · metric**

|                              |                                                                                                                                                              |
|------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>PLAIN MEANING</b>         | The gauge: a number over time that summarises many events — count of calls, share that failed, how long they took — kept per category (stage, route, model). |
| <b>WHAT IT IS NOT</b>        | Not an event and not about one ticket; it cannot tell you what happened to t7f3, only what happened to Tuesday.                                              |
| <b>HEARD AT KAVERI</b>       | "retrieval_latency_ms by stage and route, every minute: that is the gauge that jumped at 14:10."                                                             |
| <b>HOW THE EXAM WORDS IT</b> | "aggregate metrics", "time series", "latency / error / throughput per stage", "alert on a metric".                                                           |
| <b>EXPANDS</b>               | <i>Loading Dock E7 (token, latency and cost counters per stage) · Atlas A6 (layer 3)</i>                                                                     |

Technically: structured logs are the events; a trace id propagated through every stage joins them into traces made of spans; metrics are aggregates over events, kept as time series per label (Climb 3). The Dock's E7 wraps every tool call with a trace id, counts tokens, latency and cost per stage, and prints one aggregated view that answers 'where did the 40,000 go slow?' — on Day B, watch the line that stamps the trace id before the first stage, and the per-stage totals at the end.

**D3 LAW 4 · You cannot fix what you cannot see.**

Born here. A system with no diary, no sticker and no gauge fails silently and is repaired by guessing. Broken: Tuesday afternoon — three diaries that could not be joined, a theory built from timestamps by eye, and an index rebuild at 14:09 that nobody connected to the slowness until Thursday.

**ADD IT · distributed tracing (adding is the right move here)**

A stem describes a three-stage pipeline where 'the team cannot tell which stage is slow'. Adding trace-id propagation and per-stage spans is **the right move** — it is one field copied through three calls, and nothing else answers the question. It would be OE to add a full tracing platform to a single-stage script; it is never OE to make a multi-stage request joinable.

**R4****THE EXAM****The exam — how a 3.4 stem hides 'you cannot join the diaries'**

**The disguise.** 'Logs exist for every service' — and then 'the team cannot determine which stage causes the latency'. The tell is 'cannot correlate / attribute / determine which stage' next to 'logs exist': logs without a trace id.

**Family baited:** VG (trace read as log) and OE (a new screen where a sticker was missing).

**Mini-stem 1.** Every stage logs richly; nobody can follow one request across router, retrieval and generation. — **Shape:** **generate a trace id at entry, propagate it through every call, log it on every line, record a span per stage.**

**Mini-stem 2.** A single-process script with one stage is ‘hard to debug’. — **Shape: structured logs with fields; tracing adds nothing to a one-stage journey.**

### PENCIL LAB P6 · Thread three tickets through three stages, without and with a trace id

Three TOY tickets — **A, B, C** — each passes router → retrieval → generate. Here are the nine diary lines, shuffled, as they would appear without a sticker: *router 14:10:01 (12 ms) · retrieval 14:10:01 (280 ms) · generate 14:10:02 (300 ms) · router 14:10:03 (11 ms) · retrieval 14:10:03 (2,640 ms) · generate 14:10:06 (310 ms) · router 14:10:04 (12 ms) · retrieval 14:10:04 (300 ms) · generate 14:10:05 (290 ms)*. (a) Try to match them into three journeys by timestamp alone. Which ticket was slow, and how sure are you? (b) Now here are the same lines with a trace id: every line carries a, b or c — *router a · retrieval a (280) · generate a (300) · router b · retrieval b (2,640) · generate b (310) · router c · retrieval c (300) · generate c (290)*. Which ticket was slow, at which stage, and what one query finds it?

----- ∞ fold here — the worked answer is below the line -----

### WORKED ANSWER · P6

(a) By timestamp you can guess: the 14:10:03 router line is followed by a 2,640 ms retrieval that ends around 14:10:06, so the ‘generate 14:10:06’ line probably belongs to it — but the 14:10:04 ticket’s lines interleave with it, and with three hundred tickets a minute you would never be sure. Sure-ness: low. (b) With the id: **ticket b, retrieval span, 2,640 ms**; total b = 11 + 2,640 + 310 = 2,961 ms. The query: **trace = b**, or, for all slow ones, **stage = retrieval AND ms > 2000**. One field copied three times turned a guessing game into a lookup.

**What you should feel:** E7 does exactly this on Day B — watch the line that creates the id and the three places that log it.

### CHECK YOURSELF

1. Diary, sticker, gauge — name the real terms and the question each answers.
2. Why are print statements not observability?
3. What is the one thing that turns three diaries into a journey?
4. Explain D3 Law 4 to Meera in two sentences.

----- ∞ fold — answers below -----

1. Structured log — what happened here; trace (spans, trace id) — what happened to this request everywhere; metric — how are we doing overall, over time.
2. A free sentence cannot be filtered by stage, joined by ticket, or counted; structured fields can.
3. A trace id generated at entry and copied onto every log line and span.
4. If the system does not write down what it did, per ticket and overall, we can only guess when it goes wrong. Cameras and a register are cheap; guessing at 40,000 a day is not.

## Climb 2 · The afternoon the average hid — percentiles by hand

**R1**

THE SCENE

### The scene — ‘average response time is fine’

Wednesday, the review. The vendor’s slide says *average response time: under a second*. Meera has three hundred dispatchers who spent Tuesday afternoon swearing at the screen. Both are true. Arjun lines up Tuesday’s calls in a row, shortest to longest, and walks Meera along it. The first half: quick, quick, quick. The last twentieth: three seconds, four, five. “The average is the whole row melted into one number. Your dispatchers don’t live in the average. One in twenty of them lived *here*” — he points at the tail — “and at three hundred people on shift, that’s fifteen at any moment, all afternoon, and it’s always the same fifteen kinds of question.”

R2

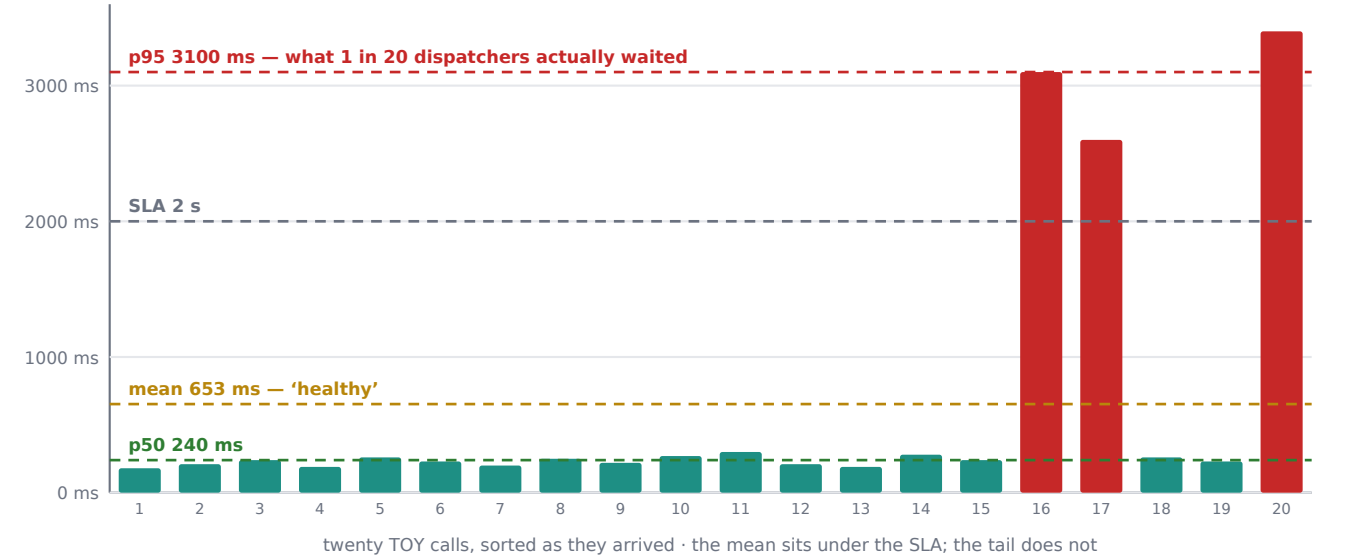
PLAIN WORDS

Plain words — line them up, then walk to the 95th of every 100

The average adds everything up and divides — so a few very slow calls get smeared thin across many fast ones and vanish. A **mark along the line** refuses to melt anything: line up all the calls from fastest to slowest and walk along the line; the value at the halfway mark is what a typical call took (half were faster), the value 95 % of the way along is the ninety-fifth mark (nineteen in twenty were faster — one in twenty waited at least this long). The ninety-fifth mark is what your unluckiest regular user experiences every day. A promise about speed is a promise about the tail, or it is not a promise.

TOY

TOY: 20 calls. 17 take about a quarter of a second; 3 take about three seconds. Mean  $\approx$  0.65 s — ‘healthy’. Halfway mark = 0.24 s. Ninety-fifth mark = 3.1 s. One dispatcher in twenty waited over three seconds while the slide said ‘under a second’. Do it yourself in P5.



**W11** · Twenty TOY calls as they arrived, with the mean, the p50, the p95 and a 2-second SLA drawn across them. The mean sits comfortably under the SLA; the p95 does not. The three crimson bars are the afternoon the average hid. Redraw from memory on Day D — it is the p95-vs-mean panel of Atlas A6.

R3

THE REAL NAME

The real name — percentiles (p50 / p95 / p99), SLA, and the canon figures

| WORD CARD · percentile latency (p50 / p95 / p99) |                                                                                                                                                                                              |
|--------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| PLAIN MEANING                                    | Sort every call's latency from fastest to slowest; pN is the value N % of the way along — p50 the middle, p95 the point nineteen in twenty were faster than, p99 the worst one in a hundred. |
| WHAT IT IS NOT                                   | Not the average (which melts the tail away) and not ‘95 % accuracy’ ( a percentile is a position in a sorted list of times, not a score).                                                    |
| HEARD AT KAVERI                                  | “Tuesday: mean 310 ms, p95 2,900 ms. Fifteen of three hundred dispatchers, all afternoon.”                                                                                                   |
| HOW THE EXAM WORDS IT                            | “p95 latency”, “tail latency”, “the mean is acceptable but users complain”, “SLA on the 95th percentile”.                                                                                    |
| EXPANDS                                          | <i>Loading Dock glossary ‘p50/p95/p99 latency’ · E7 (p95 vs mean shown with numbers) · Atlas A6</i>                                                                                          |

**WORD CARD · SLA (service-level agreement)**

|                              |                                                                                                                                                                                 |
|------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>PLAIN MEANING</b>         | A promise about how the system will behave, written as a number on a percentile over a window — ‘p95 under 2 s over any 5-minute window’ — with consequences when it is missed. |
| <b>WHAT IT IS NOT</b>        | Not a promise about the average, and not a target you hope for (that is an SLO, the internal objective); an SLA is what someone can hold you to.                                |
| <b>HEARD AT KAVERI</b>       | “The Dock's three SLAs — 2 s, 4 s, 5 s — are promises about tails, one per route; the alert in Climb 3 fires on the p95, not the mean.”                                         |
| <b>HOW THE EXAM WORDS IT</b> | “meet a 2-second SLA at p95”, “latency budget per stage”, “the SLA is defined on the mean” (the tell of a bad SLA).                                                             |
| <b>EXPANDS</b>               | <i>Loading Dock ch. 3.3 / 3.4 (the three SLAs) · Station 6 here (the dial board)</i>                                                                                            |

The canon figures from the Dock's E7, in gold: on Tuesday's run the mean latency was **310 ms** and the p95 was **2,900 ms** — nine times the mean — and the slow ticket's end-to-end total was **4,100 ms** across its stages. The Dock's three SLAs are **2 s, 4 s** and **5 s**: each a promise about a tail, and the 2,900 ms p95 breaks the first of them while the 310 ms mean sails under all three. When you type E7 on Day B, watch the two lines that print mean and p95 side by side — the whole objective is the gap between them.

**WRONG BELIEF, CORRECTED**

|                                  |                                                                                                                                                                                                                                                 |
|----------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>I used to think...</b>        | the average is fine, so users are fine                                                                                                                                                                                                          |
| <b>Why it feels right</b>        | The dashboard shows one number, it is under the SLA, and averages are what reports have always shown.                                                                                                                                           |
| <b>The correction</b>            | <b>The average melts the tail. A p95 of 2,900 ms with a mean of 310 ms means one call in twenty takes nine times the ‘typical’ time — and at 300 concurrent users that is fifteen people at every moment. Promises are made on percentiles.</b> |
| <b>The stem that punishes it</b> | A stem where the mean meets the SLA, users complain, and the right option is ‘measure and alert on p95 per stage’.                                                                                                                              |

**TRAP · ‘Optimise the average’**

Options that speed up the common case (a faster model for simple queries, a cache) can leave the tail untouched, because the tail is usually a *different* kind of call — the hybrid retrieval with a rerank, the escalation, the long document. The **trap** is fixing the mean when the complaint was the tail. The tell: ‘users report’ beside a healthy average.

**R4****THE EXAM****The exam — how a stem disguises the tail**

**The disguise.** A reassuring average, a complaint, and options that improve throughput or the mean. The tell is the pair: ‘average ... is within target’ + ‘users / dispatchers report’. The shape is always **measure per-stage percentiles, find the slow stage, fix the tail's cause**.

**Family baited:** VG (percentile read as accuracy) and TR (optimising the currency not complained about).

**Mini-stem 1.** Mean 400 ms, SLA 2 s, dispatchers say ‘it hangs’. — **Shape: compute p95/p99 per stage from the traces; the tail will name the stage (probably retrieval + rerank on hybrid queries); ring the bell on p95, not the mean.**

**Mini-stem 2.** A team proposes defining the SLA on the mean ‘because p95 is noisy’. — **Shape: no: define it on p95 over a window long enough to smooth noise; the mean cannot represent the users who wait.**

**PENCIL LAB P5 · Compute p50 and p95 from twenty TOY latencies whose mean says healthy**

Twenty TOY latencies (ms), as they arrived: **180, 210, 240, 190, 260, 230, 200, 250, 220, 270, 300, 210, 190, 280, 240, 3100, 2600, 260, 230, 3400**. (a) Add them up and divide by 20: the mean. Does it clear a 2 s SLA? (b) Sort them. (c) p50: with 20 values, take the average of the 10th and 11th sorted values. (d) p95: 95 % of 20 = 19, so take the 19th sorted value (nearest-rank method). (e) How many of the 20 calls broke the 2 s SLA? At 300 dispatchers on shift, how many are waiting past 2 s at any moment if this pattern holds? (f) Write one sentence for Meera that uses the p95 and not the mean.

----- ∞ fold here — the worked answer is below the line -----

**WORKED ANSWER · P5**

(a) Sum = **13,060**; mean = **653 ms** — comfortably under 2 s: 'healthy'. (b) Sorted: 180, 190, 190, 200, 210, 210, 220, 230, 230, 240, 240, 250, 260, 260, 270, 280, 300, 2600, 3100, 3400. (c) p50 =  $(240 + 240) / 2 =$  **240 ms**. (d) 19th value = **3,100 ms** = p95. (e) Three calls broke 2 s ( $2,600 \cdot 3,100 \cdot 3,400$ ) — 3 of 20 = 15 %; at 300 on shift that is about **45** people waiting past the SLA at any moment. (f) 'One dispatcher in twenty waits over three seconds — p95 is 3.1 s against a 2 s promise — even though the average call takes 0.65 s.'

**What you should feel:** the mean was true and useless. The canon version is E7's 310 ms mean against 2,900 ms p95 — the same shape with real numbers.

**CHECK YOURSELF**

1. How do you compute a p95 by hand?
2. Quote the canon E7 figures.
3. Why is an SLA on the mean a bad SLA?
4. Explain to Meera in two sentences why the vendor's slide and her dispatchers are both right.

----- ∞ fold — answers below -----

1. Sort all values fastest to slowest; take the value 95 % of the way along (nearest rank: the  $[0.95 \times n]$ -th).
2. Mean 310 ms; p95 2,900 ms; slow ticket total 4,100 ms; SLAs 2 s / 4 s / 5 s.
3. The mean hides the tail; the users who suffer are in the tail; a promise on the mean promises nothing to them.
4. The slide melted every call into one average, and most calls are quick. But one in twenty takes nine times longer, and with 300 people on shift that is fifteen of them stuck at any moment.

**Climb 3 · The wall of gauges — what to label, and when to ring the bell****R1****THE SCENE****The scene — the wall that nobody could read**

Thursday. Someone had a good idea: a gauge for every consignment. Forty thousand consignments a day, forty thousand new gauges a day; by Thursday the camera-room wall is a snowfield of tiny dials and the storage bill has tripled. Nobody looks at it. In the corner, the bell: it has rung 214 times since Monday, for every single call over two seconds, and Arjun muted it on Tuesday. The one time it should have rung — Tuesday 14:10, when retrieval went bad for everyone — it was ringing exactly as it had all morning, and he did not look up.

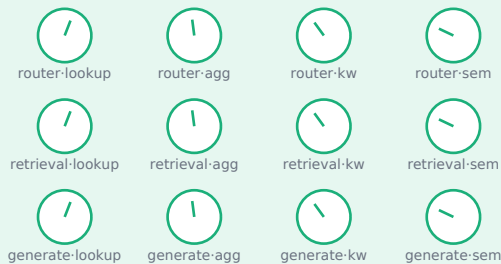
**R2****PLAIN WORDS****Plain words — label by category, not by identity; ring the bell rarely and mean it**

A gauge is useful when it summarises many things: 'retrieval latency, hybrid route, this minute'. Each way you split a gauge — by stage, by route, by model — multiplies the number of gauges. Split by a *category* with a handful of values (three stages, four routes) and you get a dozen gauges you can read. Split by an *identity* with unbounded values (consignment id, customer id, trace id) and you get a gauge per thing that ever existed: unreadable, and the store melts. Identities belong on the sticker and in the diary, never on the gauge.

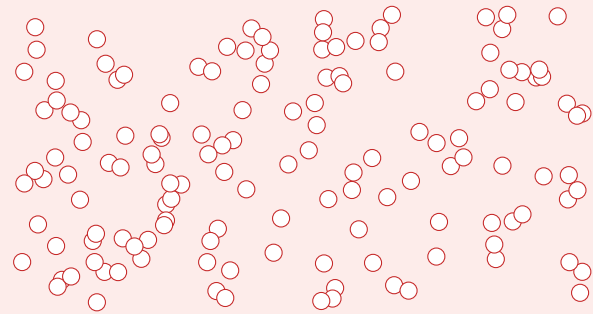
**TOY**

TOY: 3 stages × 4 routes = 12 gauges. Add a label with 2 model names: 24. Add the consignment id: 24 × 40,000 = 960,000 today, more tomorrow. The first two are categories; the third is an identity.

And the bell: ring it for a *pattern*, not an *event* — ‘p95 over 4 s for five minutes’ rather than ‘a call over 2 s’ — and ring it rarely enough that when it rings, someone looks up. A bell that rings all morning is a bell that has been muted.

**A · labels are categories: 3 stages × 4 routes = 12 gauges**

readable at a glance; cheap to store forever

**B · a label is the consignment id: 40,000 new gauges a day**

the store melts; the wall is unreadable; nobody looks

**W12** · The wall of gauges. Left: labels are categories — twelve dials anyone can read. Right: a label is the consignment id — forty thousand new dials a day, unreadable, and the store melts. This is the cardinality trap of Atlas A6; redraw both panels from memory on Day D.

**R3**

THE REAL  
NAME

**The real name — series, label, cardinality, alert****WORD CARD · series (time series) and label**

|                              |                                                                                                                                                                                                                      |
|------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>PLAIN MEANING</b>         | A series is one gauge over time — one metric with one particular set of labels; a label is a name=value tag that splits a metric (stage=retrieval, route=hybrid). Each distinct label combination is its own series. |
| <b>WHAT IT IS NOT</b>        | Not the contextual prefix or the version metadata of Station 1 (► ‘label’ there is text on a chunk); here a label is a key on a metric.                                                                              |
| <b>HEARD AT KAVERI</b>       | “retrieval_latency_ms{stage=retrieval, route=hybrid} is one series; add model=haiku and it is another.”                                                                                                              |
| <b>HOW THE EXAM WORDS IT</b> | “metric dimensions”, “label set”, “per-series storage cost”.                                                                                                                                                         |
| <b>EXPANDS</b>               | <i>Loading Dock glossary ‘cardinality’ · Atlas A6 (the cardinality trap)</i>                                                                                                                                         |

► **SAME WORD, TWO WORLDS · label** — **(1)** metrics sense (this station): a name=value tag on a gauge; each combination is a series. **(2)** shelving sense (Station 1): the contextual sentence or version field written on a chunk. **The tell:** ‘metric’, ‘series’, ‘cardinality’ → metrics sense; ‘chunk’, ‘version’, ‘prefix’ → shelving sense.

**WORD CARD · cardinality**

|                              |                                                                                                                                                                               |
|------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>PLAIN MEANING</b>         | How many distinct values a label can take — and therefore how many series a metric explodes into. Low: stage (3), route (4). High: consignment id (40,000 a day and growing). |
| <b>WHAT IT IS NOT</b>        | Not the number of events; a million calls into 12 series is fine, a thousand calls into a thousand series is the trap.                                                        |
| <b>HEARD AT KAVERI</b>       | “Putting KF- ids on the latency metric tripled the storage bill and made the wall unreadable — the identity belongs on the trace, not the gauge.”                             |
| <b>HOW THE EXAM WORDS IT</b> | “high-cardinality labels”, “unbounded label values”, “metric explosion”, “use trace ids / logs for per-request detail”.                                                       |
| <b>EXPANDS</b>               | <i>Loading Dock glossary ‘cardinality’ · E7 · Atlas A6</i>                                                                                                                    |

**WORD CARD · alert**

|                              |                                                                                                                                                 |
|------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>PLAIN MEANING</b>         | A rule on a metric that notifies a person when a pattern holds — ‘retrieval p95 > 4 s for 5 minutes’ — with someone named to respond.           |
| <b>WHAT IT IS NOT</b>        | Not a log of every bad event and not a dashboard; an alert that fires on single events, or fires constantly, is noise and gets muted.           |
| <b>HEARD AT KAVERI</b>       | “Two alerts on the tower: p95 per route over its SLA for 5 min; error rate per stage over 2 % for 5 min. Both page the on-call, both are rare.” |
| <b>HOW THE EXAM WORDS IT</b> | “alert fatigue”, “threshold on p95 over a window”, “few, actionable alerts”, “the alert fires on every slow request” (the tell).                |
| <b>EXPANDS</b>               | <i>Loading Dock Atlas A6 (layer 5: alerts, ‘few and dear’)</i>                                                                                  |

Technically: a metric with labels stage (3) × route (4) × model (2) is 24 series; the same metric with a consignment-id label is 24 × 40,000 today and unbounded over time — the cardinality trap. Per-request detail lives in logs and traces (keyed by trace id), never on metrics. Alerts are few, on percentiles over windows, each with an owner. The Dock’s E7 keeps its counters per stage and route only; on Day B, notice what it does *not* label by.

**TRAP · The channel that cried wolf**

An alert per slow call at 40,000 calls a day is 2,000 alerts a day at a 5 % tail — and a muted channel by Tuesday. The **trap** is any option that alerts on single events or on every stage of every request. The tell: ‘alert on every’, ‘notify whenever a request’; the correct shape is **a percentile over a window, per route, with an owner**.

**R4****THE EXAM****The exam — how a stem disguises the melting store and the muted bell**

**The disguise.** ‘Metrics storage costs have grown 20× since adding per-customer tracking’; ‘the team receives hundreds of alerts a day and has started ignoring them’. The tell is an unbounded label (customer, consignment, session, trace) on a metric, or alert volume described in hundreds.

**Family baited:** VG (cardinality) and OE (more screens for a muted bell).

**Mini-stem 1.** Latency metrics were labelled by consignment id ‘to find slow shipments’; storage exploded. — **Shape: drop the id from the metric; keep labels to stage/route/model; find slow shipments via traces and logs keyed by trace id.**

**Mini-stem 2.** Alerts fire for every request over 2 s; on-call ignores the channel. — **Shape: replace with a p95-over-5-minutes rule per route, one owner, rare by design.**

**PENCIL LAB P12 · Label a metric wall: which labels are safe and which melt the store**

You are labelling `request_latency_ms`. For each candidate label say **safe** (bounded category) or **melts** (unbounded identity), and for the melting ones say where that detail should live instead: (1) stage · (2) route (lookup / aggregate / keyword / hybrid) · (3) consignment\_id · (4) model · (5) dispatcher\_id (300 people) · (6) trace\_id · (7) customer\_phone · (8) http\_status · (9) sop\_version (weekly) · (10) query\_text. Then count the series for the safe set if stage has 3 values, route 4, model 2, http\_status 5, sop\_version ~52 a year.

----- × fold here — the worked answer is below the line -----

**WORKED ANSWER · P12**

(1) stage — **safe** · (2) route — **safe** · (3) consignment\_id — **melts** → trace/log field · (4) model — **safe** · (5) dispatcher\_id — **borderline**: 300 is bounded but multiplies everything by 300; keep it in logs, or aggregate to team · (6) trace\_id — **melts** (one series per request) → it *is* the trace key, never a metric label · (7) customer\_phone — **melts**, and personal data on a metric is a second problem → nowhere; the ledger has the customer id if needed · (8) http\_status — **safe** · (9) sop\_version — **safe but growing**: ~52 values a year; acceptable, or collapse to ‘current / stale’ · (10) query\_text — **melts** → logs. Series for the safe set:  $3 \times 4 \times 2 \times 5 \times 52 = 6,240$  — already plenty; add dispatcher\_id and it is 1.9 million.

**What you should feel:** a label is a multiplication. E7’s counters are labelled by stage and route only — that is a design decision, not an omission.

## CHECK YOURSELF

1. What is cardinality, and which labels are safe?
2. Where does per-request detail live if not on metrics?
3. Write a good alert rule for the retrieval stage.
4. Explain to Meera in two sentences why we took the consignment ids off the wall.

----- ∞ fold — answers below -----

1. The number of distinct values a label can take; safe labels are bounded categories (stage, route, model, status), not identities (ids, phones, query text).
2. In logs and traces, keyed by trace id.
3. retrieval p95 > 4 s for 5 minutes, per route, pages the on-call owner.
4. A gauge per shipment means forty thousand new gauges a day and a bill that triples. We keep gauges for the handful of categories that matter and look up a single shipment by its ticket mark instead.

## Climb 4 · The incident walks down the stairs — and the dashboard is a window, not a sensor

### R1

#### THE SCENE

### The scene — Thursday, done properly

Thursday, 14:12. The bell rings — once, because the rule is ‘one fetch in twenty slower than four seconds, for five minutes, on the mixed-search route’. Arjun looks up. The wall screen shows which route and which stage. The metric shows the jump began at 14:10. He pulls one slow ticket from that minute: fetching, 2,640 ms, and the sticker says which shelf it went to. The diary line for that stage says the shelves were rebuilt at 14:09 and the mixed-search questions are hitting a cold store. Five floors, five minutes, one cause. On Tuesday the same walk took two days and ended in the wrong place.

Meera: “So the new screen fixed it?” Arjun: “The screen showed it. The sticker and the diary fixed it. The screen is a window.”

### R2

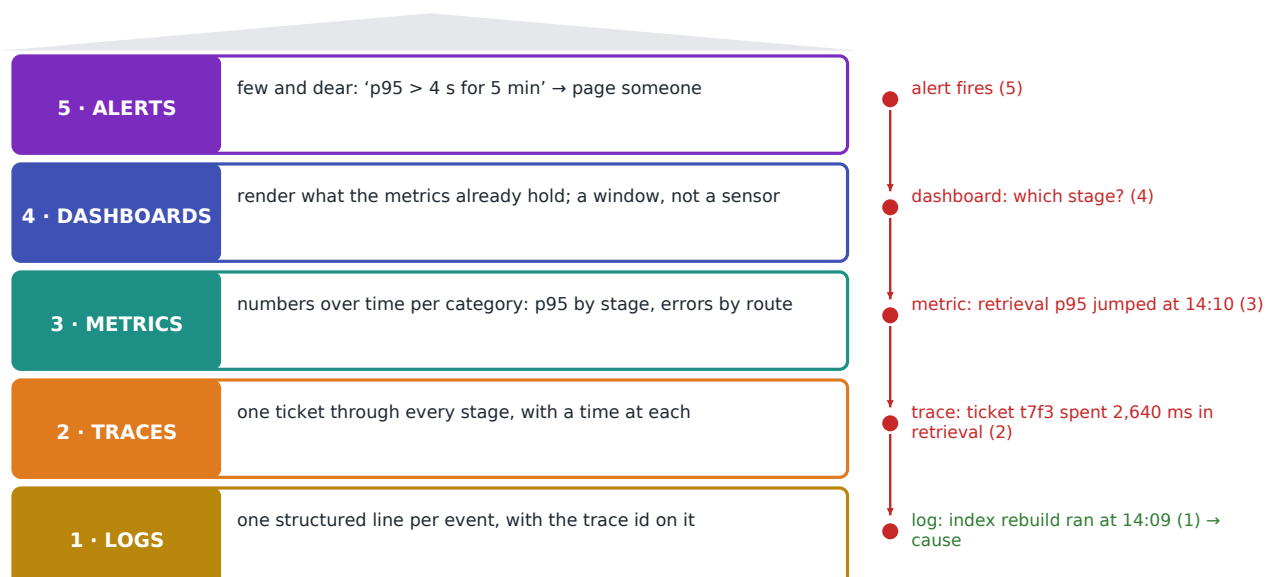
#### PLAIN WORDS

### Plain words — five floors, and the incident walks down

Stack the five kinds of watching as floors: diary lines at the bottom, stickers above them, gauges above those, screens that draw the gauges, and at the top the bell. Building the stack goes up — each floor is made from the one below. An incident walks *down*: the bell names a pattern, the screen names a route and stage, the gauge names a minute, the sticker names a ticket and a stage, the diary line names the cause. If any floor is missing, the walk stops there and guessing begins. And the screen at floor four cannot show anything the floors below did not record: a screen renders, it does not create.

### TOY

TOY: a bell (1) → a screen with 12 dials (2) → one dial's history (3) → one ticket's three stages (4) → one line saying ‘index rebuilt 14:09’ (5). Five steps; remove step 4 and you are back to matching timestamps by eye.



an incident walks **DOWN** the stairs: from the alert to the log line that explains it

**W13** · The five-layer stack as a building — logs, traces, metrics, dashboards, alerts — and an incident walking down the stairs from the alert to the log line that explains it. This is Atlas A6 drawn slowly; redraw it from memory on Day D.

### R3

THE REAL  
NAME

## The real name — the observability stack and the dashboard

### WORD CARD · dashboard

|                              |                                                                                                                                                                    |
|------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>PLAIN MEANING</b>         | A screen that draws metrics (and sometimes trace summaries) so a person can read the state of the system at a glance — floor four of the stack.                    |
| <b>WHAT IT IS NOT</b>        | Not a source of data: it can only show what logs, traces and metrics already recorded. Adding a dashboard to a system with no trace ids adds a window onto a wall. |
| <b>HEARD AT KAVERI</b>       | "The tower dashboard: p95 by route and stage, error rate by stage, tokens and cost per stage — twelve panels, all from E7's counters."                             |
| <b>HOW THE EXAM WORDS IT</b> | "build a dashboard" (often the sounds-diligent option), "visualise", "the dashboard does not show why" (the tell).                                                 |
| <b>EXPANDS</b>               | <i>Loading Dock Atlas A6 (layer 4: dashboards)</i>                                                                                                                 |

The stack, technically: **logs** (events, structured, trace id on every line) → **traces** (spans joined by trace id) → **metrics** (aggregates per bounded label) → **dashboards** (render metrics) → **alerts** (rules on metrics, few, owned). Each layer answers a different question and is built from the one below. The Dock's Atlas A6 draws exactly this at **40,000** a day with the cardinality trap marked; E7 builds the bottom three layers in forty lines.

### WRONG BELIEF, CORRECTED

|                                  |                                                                                                                                                                                                                                                                                  |
|----------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>I used to think...</b>        | adding a dashboard fixes observability                                                                                                                                                                                                                                           |
| <b>Why it feels right</b>        | The incident review said 'we couldn't see it', and a dashboard is the thing you look at.                                                                                                                                                                                         |
| <b>The correction</b>            | <b>A dashboard is floor four. If the logs have no trace id and the metrics have no per-stage split, the dashboard shows the same fog in colour. Fix the floors below: structured logs, trace propagation, bounded-label metrics; then the dashboard has something to render.</b> |
| <b>The stem that punishes it</b> | A stem where 'build a dashboard' is offered for a team that cannot correlate logs across stages, and the correct option is trace-id propagation.                                                                                                                                 |

**ADD IT · a metrics layer with per-stage counters (adding is the right move here)**

A stem describes a pipeline with good structured logs and traces but ‘no way to know whether today is worse than yesterday’. Adding per-stage, per-route metrics (latency percentiles, error rate, tokens, cost) is **the right move** — the third floor is missing, and nothing below it answers ‘over time’.

**R4****THE EXAM****The exam — how a stem disguises the missing floor**

**The disguise.** A team ‘has extensive dashboards’ or ‘receives alerts’ and still cannot explain an incident. The tell is which question they cannot answer: ‘why’ → logs; ‘which request / which stage’ → traces; ‘is it worse than usual’ → metrics; ‘nobody noticed’ → alerts. Name the floor; that is the answer's shape.

**Family baited:** SD (dashboard) and CM (a D1 evaluator for an observability gap).

**Mini-stem 1.** Rich dashboards; incidents are found by customers, never by the team. — **Shape: alerts on p95 and error rate per route over a window, with owners — floor five is missing.**

**Mini-stem 2.** Alerts fire correctly; the on-call cannot find which stage or request caused it. — **Shape: trace-id propagation and spans per stage — floor two is missing.**

**CHECK YOURSELF**

1. List the five floors bottom-up and the question each answers.
2. Which direction does an incident walk, and why?
3. Can a dashboard show a per-stage breakdown if the logs have no trace id?
4. Explain to Meera in two sentences why the new screen didn't fix Tuesday.

----- ∞ fold — answers below -----

1. Logs — what happened here; traces — what happened to this request; metrics — how are we doing over time; dashboards — show me; alerts — tell me when a pattern holds.
2. Down: from the alert's pattern to the log line's cause; each floor narrows the search.
3. No — it renders what was recorded; without trace ids there is no per-request stage breakdown to render.
4. The screen only draws what the system wrote down, and on Tuesday the stages weren't joined so there was nothing to draw. The sticker on every ticket and the per-stage gauges are what made Thursday a five-minute walk.

### WHAT WATSON WROTE DOWN

- Three kinds of watching: **log** (event, structured, here) · **trace** (one request everywhere; spans; trace id) · **metric** (aggregate over time per bounded label).
- **D3 Law 4**: you cannot fix what you cannot see. Trace id at entry, copied onto every line — one field turns guessing into a query.
- Percentile = position in the sorted list. Canon E7: mean **310 ms** · p95 **2,900 ms** · slow ticket **4,100 ms** total; SLAs **2 s / 4 s / 5 s**. Promises are about tails.
- **Average is fine ≠ users are fine**. Tell: healthy mean + ‘users report’.
- Cardinality = distinct label values. Categories (stage, route, model, status) are safe; identities (ids, phones, query text) melt the store → put them on traces/logs.
- Alerts: few and dear — a percentile over a window, per route, with an owner. Never per event.
- Five floors: logs → traces → metrics → dashboards → alerts. Incidents walk down. A dashboard renders; it does not create.
- ► span = timed stage, not HTML. ► label = metric tag here, chunk text at Station 1.

**Law in force:** D3 Law 4 · You cannot fix what you cannot see.

**Read next in the Loading Dock:** ch. 3.4 · E7 (the trace-id stamp; the per-stage totals; the mean/p95 lines) · Atlas A6 (layers and the cardinality trap) · glossary entries for trace/span, p50/p95/p99 latency, cardinality, audit log

TONIGHT YOU WILL

**Saturday 3 Oct (Day A) · first 15 minutes · Station 5 · The toolbelt**

- see that a tool's description is weight on the belt whether or not the tool is used
- price the belt with a pencil: 6 unused schemas × 250 × 40,000 (P8), then meet the canon figures **2,314 / 811 / 60,120,000**
- spot the lookalikes, and trim by logs AND consequence — never by logs alone
- own the words: tool schema, capability bloat, lookalike tools

**Then:** read Loading Dock ch. 3.1, then type E2 (the bloat audit). Watch the before/after context bill line.

STATION 5

**The toolbelt**

Evaluate tool/agent configuration for capability bloat — a schema is weight on the belt

OBJECTIVE

**3.1**

expands Loading Dock ch. 3.1 · E2 · trap-of-traps (capability bloat) · Station 3 (the other half of the trap)

Climb 1 · Nine wrenches, three jobs

R1

THE SCENE

**The scene — the belt that came with everything**

Saturday, 9 am, the tool crib. Arjun's belt has nine wrenches on it. He uses three all day — customer, order, refund. The other six came with the belt: a bulk exporter nobody has run since the migration, a re-indexer that only the nightly job should hold, a carrier pinger, an exchange-rate fetcher, a legacy sync, and something called *escalate* that is not the escalate he uses. Nine wrenches weigh nine wrenches, every step of every shift, whether he swings them or not. And twice this month he has grabbed the wrong one — the two that look alike.

R2

PLAIN WORDS

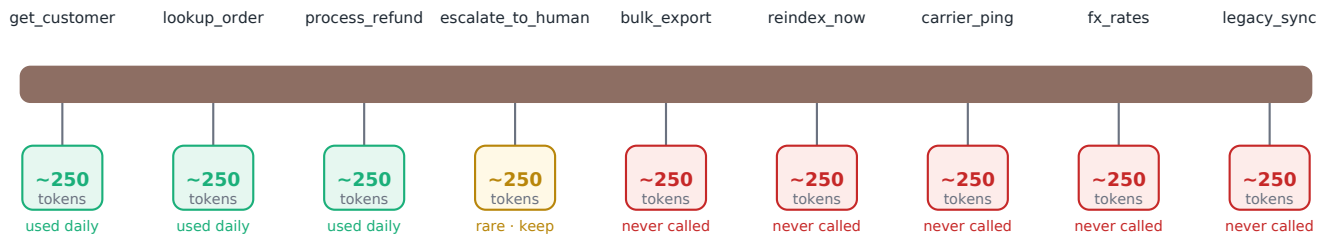
**Plain words — every description rides along on every call**

Each tool the model may use has to be described to it — name, what it does, what inputs it takes — and that description is sent with every call, because the model has no memory between calls. Nine descriptions cost nine descriptions' worth of words per call, forty thousand calls a day, whether the tool is ever chosen. Worse than the weight: two tools with similar names and similar descriptions make the model pick the wrong one sometimes, and the wrong wrench on a refund button is a consequence, not a cost.

TOY

TOY: 9 wrenches × 250 grams = 2.25 kg on the belt; 3 wrenches = 0.75 kg. 40,000 steps a day at 1.5 kg extra. Nobody notices one step; everybody notices the month.

**every schema hangs on the belt on EVERY call — whether or not the tool is used**



**canon E2: 9 tools = 2,314 tokens/call · trimmed to 3 = 811 · 1,503 × 40,000 calls/day = 60,120,000 tokens/day**

lookalikes are the consequence cost: lookup\_order vs bulk\_export · escalate\_to\_human vs legacy\_sync — trim by logs AND consequence

**W14** · The belt with weights: nine schemas at roughly 250 tokens each, all riding on every call; six never called; two lookalike pairs. Trim to three and the canon bill drops from 2,314 to 811 tokens per call — 60,120,000 tokens a day at 40,000 calls.

### R3

THE REAL NAME

## The real name — tool schema, capability bloat, lookalikes; the canon bill

### WORD CARD · tool schema

|                       |                                                                                                                                                      |
|-----------------------|------------------------------------------------------------------------------------------------------------------------------------------------------|
| PLAIN MEANING         | The written description of one tool the model is sent on every call — its name, a sentence on what it does, and the typed list of inputs it accepts. |
| WHAT IT IS NOT        | Not the tool's code (that stays on your side) and not free: it is tokens in the context window on every request.                                     |
| HEARD AT KAVERI       | "Nine schemas cost 2,314 tokens per call before the dispatcher has typed a word; three cost 811."                                                    |
| HOW THE EXAM WORDS IT | "tool definitions consume N tokens", "schema overhead", "tool descriptions in the context".                                                          |
| EXPANDS               | <i>Loading Dock ch. 3.1 · E2 (measure the tool-schema token cost of the unused six)</i>                                                              |

### WORD CARD · capability bloat

|                       |                                                                                                                                                   |
|-----------------------|---------------------------------------------------------------------------------------------------------------------------------------------------|
| PLAIN MEANING         | Giving the agent more tools than its job uses — each one weight on every call, a door that needs a key (Station 3), and a chance of a wrong pick. |
| WHAT IT IS NOT        | Not 'any agent with many tools': a job that uses twelve tools needs twelve. Bloat is the unused and the duplicated.                               |
| HEARD AT KAVERI       | "Six of KAVI's nine tools have not been called in thirty days; two of the remaining three have lookalikes."                                       |
| HOW THE EXAM WORDS IT | "the agent is configured with N tools of which few are used", "reduce tool surface", "capability bloat".                                          |
| EXPANDS               | <i>Loading Dock ch. 3.1 · trap-of-traps 'capability bloat and missing authorization'</i>                                                          |

### WORD CARD · lookalike tools

|                       |                                                                                                                                                 |
|-----------------------|-------------------------------------------------------------------------------------------------------------------------------------------------|
| PLAIN MEANING         | Two tools whose names or descriptions overlap enough that the model sometimes picks the wrong one — the cost that is a consequence, not tokens. |
| WHAT IT IS NOT        | Not a naming nit: on a money-moving or data-changing tool a wrong pick is an incident.                                                          |
| HEARD AT KAVERI       | "escalate_to_human and legacy escalate — one pages a person, the other writes to a queue nobody reads."                                         |
| HOW THE EXAM WORDS IT | "ambiguous tool descriptions", "the agent invokes the wrong tool", "consolidate overlapping tools".                                             |
| EXPANDS               | <i>Loading Dock ch. 3.1 (lookalikes) · E2</i>                                                                                                   |

The canon figures, in gold, from the Dock's E2: nine tool schemas cost **2,314** tokens per call; trimmed to the three actually used, **811**; the difference, **1,503** tokens, at **40,000** calls a day is **60,120,000** tokens a day that bought nothing. When you type E2, watch the two lines that print the context bill before and after the trim — that pair of numbers is the whole objective.

#### TRAP · Trim by logs alone

A tool with zero calls in thirty days is a candidate, not a verdict. `escalate_to_human` may be called twice a year and be the only thing standing between a customer and a wrong refund. The **trap** is an option that removes every unused tool by usage count. Trim by logs AND consequence: what happens on the day it is needed and is not there? The tell is 'remove all tools not called in the last N days'.

#### WRONG BELIEF, CORRECTED

|                                  |                                                                                                                                                                                            |
|----------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>I used to think...</b>        | more tools make a more capable agent                                                                                                                                                       |
| <b>Why it feels right</b>        | Each tool is a thing the agent can do; more things, more capable.                                                                                                                          |
| <b>The correction</b>            | <b>Every tool is tokens on every call, a door needing a key, and a chance of a wrong pick. Capability is the tools the job uses, described unambiguously. The rest is weight and risk.</b> |
| <b>The stem that punishes it</b> | A stem where the 'give the agent access to all forty internal APIs' option loses to 'expose the four it needs, with clear descriptions'.                                                   |

#### ADD IT · a tool the job actually needs (adding is the right move here)

A stem describes a support agent that must be able to page a human for refunds above a threshold, and offers 'add an `escalate_to_human` tool' among options that trim. Adding it is **the right move**: it is a used door with a consequence attached. The trimming reflex removes the unused; it never removes the safety valve.

#### R4

##### THE EXAM

#### The exam — how a 3.1 stem hides the belt

**The disguise.** Numbers of tools and calls scattered in the stem ('configured with 14 tools', '3 used regularly', '40,000 requests/day') and an option list where one trims by usage alone and one trims by usage and consequence. The tell is the pair of counts (tools configured vs used) and any duplicate-sounding names.

**Family baited:** OE (more tools) and TR (trimming the safety valve to save tokens).

**Mini-stem 1.** Fourteen tools, three used daily, two used only during incidents, nine never; 40,000 calls/day. — **Shape: remove the nine; keep the two incident tools (consequence); disambiguate the three; measure the schema bill before and after.**

**Mini-stem 2.** Two tools, `cancel_order` and `cancel_shipment`, both described as 'cancel the customer's order'; the agent picks wrong 8 % of the time. — **Shape: rewrite the descriptions to say when each applies, or merge them behind one tool with a typed argument; this is a lookalike problem, not a trimming problem.**

#### PENCIL LAB P8 · Price the belt

TOY figures: 6 unused schemas × 250 tokens each × 40,000 calls/day. (a) Tokens per call wasted? (b) Tokens per day? (c) Compare with the canon 60,120,000 — why so close? (d) Now the wrong-wrench cost, in words: `escalate_to_human` was trimmed because it had zero calls last month; on the 31st a customer's ₹40,000 refund needed a human. Say what happened, in one sentence, and what the trimming rule should have been.

----- ∞ fold here — the worked answer is below the line -----

**WORKED ANSWER · P8**

(a)  $6 \times 250 = 1,500$  tokens per call. (b)  $1,500 \times 40,000 = 60,000,000$  tokens per day. (c) The canon figure is  $2,314 - 811 = 1,503$  per call — the toy's 250-per-schema is a fair average of the real schemas, so the two land within 0.2 %. (d) The refund went through unescalated because the only door to a human was no longer on the belt; nothing errored. The rule: trim by **usage AND consequence** — a zero-call tool whose absence costs more than its schema stays.

**What you should feel:** the token bill is arithmetic anyone can do; the consequence bill is the one the stem is testing.

**CHECK YOURSELF**

1. Why does an unused tool cost anything?
2. Quote the canon E2 figures.
3. What is the trimming rule?
4. Explain to Meera in two sentences why we took six tools off KAVI.

----- ∞ fold — answers below -----

1. Its schema is sent on every call; 40,000 calls a day multiply it.
2. 9 schemas = 2,314 tokens/call;  $3 = 811$ ;  $1,503 \text{ saved} \times 40,000 = 60,120,000$  tokens/day.
3. Usage and consequence — never usage alone; a rarely used safety valve stays.
4. Every tool's description rides along on every one of forty thousand daily calls, and six of them were never used. Taking them off saves sixty million tokens a day and stops KAVI grabbing a lookalike by mistake.

**WHAT WATSON WROTE DOWN**

- A schema is weight on the belt on every call. Canon E2: **2,314 → 811** tokens/call;  **$1,503 \times 40,000 = 60,120,000$** /day.
- Bloat = unused + duplicated. Lookalikes are a consequence cost, not a token cost.
- **Trim by logs alone** is the trap; trim by logs AND consequence. Safety valves stay.
- Trap of traps for D3 = capability bloat (here) + missing authorization (Station 3): two halves of one door problem (**D3 Law 1**).
- Tells: tools configured vs used · 'remove all not called in N days' · duplicate-sounding names.

**Law in force:** D3 Law 1 · Every tool you add is a door; know who holds the key.

**Read next in the Loading Dock:** ch. 3.1 · E2 (the before/after context bill lines) · then Station 6

## TONIGHT YOU WILL

**Saturday 3 Oct (Day A) · second 15 minutes · Station 6 · The dial board**

- ☐ see that every dial buys accuracy with one of two currencies — seconds or rupees
- ☐ read each Dock dial in plain words and know which currency it spends
- ☐ write the 3.3 justification sentence — constraint + dial + direction + number — for four dial moves (P11)
- ☐ learn the tie-break when two options both clear the constraint
- ☐ own the words: accuracy-latency trade-off, constraint, dial, justification, tie-break; carry ₹6 · 2 s / 4 s / 5 s · depth 3 / concurrency 20 / max\_budget\_usd

**Then:** read Loading Dock ch. 3.3, then type E1-E4 as the Dock's Day A page says.

## STATION 6

**The dial board**

Evaluate accuracy-latency trade-offs and justify configuration decisions — two currencies, one sentence

## OBJECTIVE

**3.3**

expands Loading Dock ch. 3.3 · the SLAs and the ₹6 ceiling · D1 Law 3 (budgets) · Station 4 (p95)

**Climb 1 · Two currencies, eight dials, one sentence****R1**

## THE SCENE

**The scene — Meera wants a number and a reason**

Saturday, 10 am. Arjun proposes turning the reranker on for policy questions. Meera: “Why?” “It's more accurate.” “How much more, at what cost, against which promise?” Silence. She sends him back to the board with the eight dials on it and a card that says: *constraint · dial · direction · number*. Twenty minutes later: “Policy answers must land under four seconds at the ninety-fifth mark; turning the reranker on adds about 300 ms and cuts wrong-section answers from 2.9 % to 1.9 %; we are at 3.1 s, so we stay under four.” Meera: “Approved. That's a sentence I can defend.”

**R2**

## PLAIN WORDS

**Plain words — every dial buys accuracy with seconds or rupees, and you must say which**

There are only two currencies at the dock: how long the dispatcher waits, and what the call costs. Every dial on the board — which model, how many chunks, whether to re-sort them, how long an answer may be, whether to remember the standing prompt, how hard to think, how many retries, how many helpers — buys accuracy with one or both. A configuration decision is justified only when you can say **which promise it is measured against, which dial, which way, and by how much**. When two options both clear the constraint, the cheaper-in-the-other-currency one wins; when neither does, you have a scoping conversation, not a dial move.

**TOY**

TOY: two routes to the yard. Route A: 4 minutes, ₹40 toll. Route B: 6 minutes, free. The promise is ‘under 5 minutes’. A clears it, B does not — A wins even though it costs; if the promise were ‘under 10 minutes’, both clear and B wins on rupees.

every dial buys accuracy with one currency or the other; the justification sentence names which

|                                                                                                                                                         |                                                                                                                                                          |                                                                                                                                                       |                                                                                                                                                               |
|---------------------------------------------------------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------|
|  <b>model tier</b><br>Haiku ↔ Opus<br>accuracy ↑ · latency ↑ · cost ↑  |  <b>top_k</b><br>5 ↔ 20<br>recall ↑ · tokens ↑                          |  <b>rerank</b><br>off ↔ on<br>precision ↑ · +300 ms                  |  <b>max_tokens</b><br>200 ↔ 2,000<br>detail ↑ · latency ↑ · cost ↑         |
|  <b>cache</b><br>off ↔ on<br>cost ↓ ~90 % · latency ↓ (needs min size) |  <b>thinking budget</b><br>0 ↔ high<br>reasoning ↑ · latency ↑ · cost ↑ |  <b>retries / timeout</b><br>0 ↔ 3<br>reliability ↑ · tail latency ↑ |  <b>depth / concurrency</b><br>1 ↔ 3 / 20<br>coverage ↑ · cost ↑ (D1 caps) |

**CURRENCY 1 · seconds — SLAs 2 s / 4 s / 5 s (p95)**

**CURRENCY 2 · rupees — ₹6 ceiling per shipment**

**W15** · The two-currency dial board. Eight dials, each labelled with what it buys and what it spends; along the bottom, the two currencies with the Dock's numbers — SLAs of 2 s / 4 s / 5 s at p95 and the ₹6 ceiling per shipment. Redraw from memory on Day D.

### R3

THE REAL NAME

## The real name — the accuracy-latency trade-off, constraints, dials, the justification sentence

### WORD CARD · accuracy-latency trade-off

|                              |                                                                                                                                                                                    |
|------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>PLAIN MEANING</b>         | The fact that most things that make an answer better — a bigger model, more chunks, a rerank, more thinking — also make it slower or dearer, so you choose a point, not a maximum. |
| <b>WHAT IT IS NOT</b>        | Not a law that accuracy always costs latency: caching improves both; a router that sends easy questions to a small model improves both. Look for the free moves first.             |
| <b>HEARD AT KAVERI</b>       | "Rerank on: +300 ms, -1 point of wrong-section answers. Under the 4 s promise, so yes."                                                                                            |
| <b>HOW THE EXAM WORDS IT</b> | "evaluate accuracy-latency trade-offs", "justify the configuration", "within the latency budget".                                                                                  |
| <b>EXPANDS</b>               | <i>Loading Dock ch. 3.3</i>                                                                                                                                                        |

### WORD CARD · constraint (the promise)

|                              |                                                                                                                                                                      |
|------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>PLAIN MEANING</b>         | The number a decision is measured against — an SLA at a percentile, a cost ceiling per unit, a compliance line — stated by the business, not chosen by the engineer. |
| <b>WHAT IT IS NOT</b>        | Not a preference ('as fast as possible'); a constraint has a number and a consequence for missing it.                                                                |
| <b>HEARD AT KAVERI</b>       | "Three promises: 2 s / 4 s / 5 s at p95 by route; ₹6 per shipment all-in."                                                                                           |
| <b>HOW THE EXAM WORDS IT</b> | "the SLA requires...", "cost must not exceed...", "regulatory requirement that..." — underline it; it names the currency.                                            |
| <b>EXPANDS</b>               | <i>Loading Dock ch. 3.3 (the SLAs and the ₹6 ceiling) · Station 4 (SLA card)</i>                                                                                     |

**WORD CARD · dial (configuration parameter)**

|                              |                                                                                                                                                         |
|------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>PLAIN MEANING</b>         | One setting you can turn: model tier, top_k, rerank on/off, max_tokens, caching, thinking budget, retries and timeouts, subagent depth and concurrency. |
| <b>WHAT IT IS NOT</b>        | Not the architecture (that is D1); a dial is turned inside a chosen design.                                                                             |
| <b>HEARD AT KAVERI</b>       | "Eight dials on the board; each one names what it buys and what it spends."                                                                             |
| <b>HOW THE EXAM WORDS IT</b> | "configuration decision", "tune top_k", "select the model tier", "set max_budget_usd".                                                                  |
| <b>EXPANDS</b>               | <i>Loading Dock ch. 3.3 · D1 Law 3 for the subagent caps (depth 3 / concurrency 20 / max_budget_usd)</i>                                                |

**WORD CARD · justification sentence**

|                              |                                                                                                                                                                               |
|------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>PLAIN MEANING</b>         | Constraint + dial + direction + number: 'Because policy answers must be under 4 s at p95, turn rerank on: +300 ms, 2.9 % → 1.9 % wrong sections, 3.1 s → 3.4 s, still under.' |
| <b>WHAT IT IS NOT</b>        | Not 'it is more accurate' and not 'best practice says'; a justification names the promise and the price.                                                                      |
| <b>HEARD AT KAVERI</b>       | "Meera signs sentences, not adjectives."                                                                                                                                      |
| <b>HOW THE EXAM WORDS IT</b> | "justify configuration decisions", "which option best balances...", "explain to stakeholders".                                                                                |
| <b>EXPANDS</b>               | <i>Loading Dock ch. 3.3</i>                                                                                                                                                   |

| Dial                                          | Turn it up and you buy                                     | You pay in                        | Kaveri setting and the sentence                                                      |
|-----------------------------------------------|------------------------------------------------------------|-----------------------------------|--------------------------------------------------------------------------------------|
| model tier                                    | reasoning, fewer wrong answers                             | seconds and rupees                | Haiku for lookups (2 s promise, ₹6 ceiling); Sonnet-class for policy answers (4 s)   |
| top_k                                         | recall — the right chunk more often in the shortlist       | tokens (rupees), a little latency | 20 after rerank; 150 before — recall wide, precision narrow                          |
| rerank                                        | precision — the right chunk on top                         | ~300 ms                           | on for policy questions under the 4 s promise; off for lookups (they never retrieve) |
| max_tokens                                    | longer, fuller answers                                     | seconds and rupees                | short for status answers; longer for procedures                                      |
| prompt cache                                  | nothing lost; the standing prompt costs ~90 % less to read | a minimum size (Station 8)        | on — the tool schemas and standing instructions are stable and always used           |
| thinking budget                               | harder reasoning                                           | seconds and rupees                | off for the router; on, capped, for customs-triage                                   |
| retries / timeout                             | reliability                                                | tail latency                      | one retry with a short timeout; the p95 promise is a promise about tails             |
| subagent depth / concurrency / max_budget_usd | coverage and parallelism                                   | rupees, fast                      | depth 3 / concurrency 20 / max_budget_usd — D1 Law 3's caps, unchanged               |

The tie-break rule, in one line: **when both options clear the constraint, the one cheaper in the other currency wins; when only one clears it, that one wins regardless of the other currency; when neither does, change the design, not the dial.** Cite D1 Law 3 whenever the dial is a loop or a budget — the subagent caps are D1 canon, not new.

**WRONG BELIEF, CORRECTED**

|                                  |                                                                                                                                                                                                                                                                                        |
|----------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>I used to think...</b>        | the cheapest model is the right default                                                                                                                                                                                                                                                |
| <b>Why it feels right</b>        | Cost is the currency the finance review sees, and a small model is 'good enough for most things'.                                                                                                                                                                                      |
| <b>The correction</b>            | <b>The default is the constraint. If the promise is 2 s at p95 and ₹6 per shipment, the small model may be right for lookups and wrong for the customs-triage answer that costs a customer a day. Pick per route, against the promise; the tie-break is cost only when both clear.</b> |
| <b>The stem that punishes it</b> | A stem where the cheapest option misses the accuracy or latency constraint named two sentences earlier.                                                                                                                                                                                |

**TRAP · Optimising a currency the stem did not name**

Half the wrong options on 3.3 are correct answers to a different constraint: the fastest configuration when the stem said cost, the cheapest when it said latency, the most accurate when it said both had a ceiling. The **trap** is answering before underlining the constraint word. Tell: 'must', 'within', 'not exceed', 'SLA', 'ceiling'.

**R4****THE EXAM****The exam — how a 3.3 stem hides the currency**

**The disguise.** A constraint stated early ('p95 under 4 s', '₹6 per shipment') and options described in the other currency's adjectives. The tell is the constraint word and the percentile; the shape of the answer is the justification sentence.

**Family baited:** TR above all; OE (a bigger model where a rerank was the cheaper fix).

**Mini-stem 1.** Policy answers: 3.1 s at p95, 2.9 % wrong-section rate, promise 4 s; options: bigger model (+1.2 s), rerank (+0.3 s), top\_k 20 → 60 (+0.4 s, accuracy down). — **Shape: rerank: clears the promise with margin and cuts the error to 1.9 %; the bigger model breaks it; the wider top\_k hurts.**

**Mini-stem 2.** Status lookups: 0.4 s, ₹1.10 per call, ceiling ₹6 per shipment across ~5 calls; a proposal to move lookups to the large model 'for consistency'. — **Shape: no: lookups never retrieve; the small model clears both constraints; consistency is not a currency.**

**PENCIL LAB P11 · Write the 3.3 justification sentence for four dial moves**

For each move write one sentence with all four parts — **constraint · dial · direction · number** — and end with 'still under' or 'breaks it': (1) turning rerank on for policy questions (3.1 s → 3.4 s at p95; 2.9 % → 1.9 %; promise 4 s); (2) raising max\_tokens for procedure answers from 300 to 900 (+0.6 s; cost +₹0.40 per call; promise 4 s and ₹6 per shipment with ~5 calls); (3) switching lookups from Haiku to Sonnet (0.4 s → 1.1 s; ₹1.10 → ₹3.30 per call; promise 2 s and ₹6 per shipment, 5 calls); (4) enabling the prompt cache on an 800-token standing prompt on Haiku 4.5 (Station 8: the smallest block Haiku 4.5 will remember is 4,096 tokens).

----- ∞ fold here — the worked answer is below the line -----

**WORKED ANSWER · P11**

(1) **Because policy answers must be under 4 s at p95, turn rerank ON: +0.3 s (3.1 → 3.4 s) and wrong-section answers 2.9 % → 1.9 %; still under.** (2) **Because procedures must be under 4 s and ₹6 per shipment, raise max\_tokens 300 → 900: +0.6 s (3.4 → 4.0 s at p95 — at the line) and +₹0.40 per call (≈ +₹2 per shipment); breaks the margin — do it only for the procedure route and re-measure, or cap at 600.** (3) **Because lookups must be under 2 s and ₹6 per shipment over ~5 calls, moving to Sonnet costs 0.4 → 1.1 s (still under) but ₹1.10 → ₹3.30 per call ≈ ₹16.50 per shipment: breaks the ceiling; stay on Haiku.** (4) **Because the standing prompt is 800 tokens and the smallest block Haiku 4.5 will remember is 4,096, the cache dial does nothing here; either grow the cached block past the minimum (schemas + instructions together) or move the route to a model with a lower minimum — otherwise the ‘saving’ is zero.**

**What you should feel:** the sentence forces the number; the number decides. (4) is the one people write without checking — a dial that silently does nothing is a Silent Failure in a cost sheet.

**CHECK YOURSELF**

1. Name the two currencies and the tie-break rule.
2. What are the four parts of the justification sentence?
3. Quote the Dock's constraints.
4. Explain to Meera in two sentences why we turned the reranker on.

----- ∞ fold — answers below -----

1. Seconds and rupees. Both clear → cheaper in the other currency wins; one clears → it wins; neither → change the design.
2. Constraint, dial, direction, number.
3. SLAs 2 s / 4 s / 5 s at p95 by route; ₹6 ceiling per shipment; D1 Law 3 caps depth 3 / concurrency 20 / max\_budget\_usd.
4. Policy answers have to come back under four seconds for nineteen dispatchers in twenty, and we are at 3.1. The reranker adds a third of a second and cuts wrong-section answers by a third — we stay under and answer better.

**WHAT WATSON WROTE DOWN**

- Two currencies: seconds (SLAs **2 s / 4 s / 5 s** at p95) · rupees (**₹6** ceiling per shipment). Every dial buys accuracy with one or both.
- Justification = constraint + dial + direction + number. Adjectives are not justifications.
- Tie-break: both clear → cheaper in the other currency; one clears → that one; neither → redesign.
- Free moves first: caching, routing easy questions to a small model — better on both currencies.
- Subagent caps are D1 canon: depth **3** / concurrency **20** / max\_budget\_usd — cite D1 Law 3.
- **TR** is the family here: underline must / within / not exceed / SLA / ceiling before reading options.

**Law in force:** D1 Law 3 (budgets) in force; no new D3 law is born here.

**Read next in the Loading Dock:** ch. 3.3 · the Day A page · then type E1-E4

TONIGHT YOU WILL

Sunday 4 Oct (Day B) · first 15 minutes · Station 7 · The standard connector

☐ see the three primitives as dock furniture — the wrench, the filing drawer, the laminated checklist

☐ read the 2026-07-28 DRIFT in plain words: what ‘stateless’ means (every letter carries its own full address)

☐ decide MCP / weld / agent-to-agent for six integrations by counting clients and churn (P10)

☐ own the words: primitive, MCP tool / resource / prompt, JSON-RPC, client / server (MCP roles), stateless, elicitation, agent-to-agent

Then: read Loading Dock ch. 3.7, then type E1 (kaveri\_mcp.py grown up). Watch the import-probe line and the mcp\_\_kaveri\_\_ names.

STATION 7

The standard connector

Evaluate connection protocols; select the integration mechanism — MCP, weld, or agent-to-agent

OBJECTIVE

3.7

expands Loading Dock ch. 3.7 · E1 · Atlas A4 · glossary (MCP server / resource / prompt, transport) · DRIFT ‘MCP spec 2026-07-28’ · Station 3 Climb 4 (transports)

Climb 1 · Three pieces of furniture, one standard plug

R1

THE SCENE

The scene — the truck that fits every dock

Sunday morning. Three carriers' trucks at three bays. Bay 4 has a custom-welded ramp built for one carrier's odd tailgate — it took a week, it fits one truck, and when that carrier changed trucks last spring it took another week. Bay 6 has the standard dock plate: any truck built to the standard rolls up and connects in a minute. And at bay 6, once connected, the driver finds three things waiting: a rack of tools he may use, a filing drawer of documents he may read, and a laminated checklist for the tricky jobs — ‘customs hold: do these six steps’. The same three things at every standard bay, described the same way, so a new driver never asks where anything is.

R2

PLAIN WORDS

Plain words — a standard way to ask ‘what do you offer?’ and then use it

When the model's program needs something from another program — look up a shipment, read an SOP, run a checklist — someone has to describe what is on offer and how to ask for it. You can weld a custom connection for each pair (fast for one, a week each, brittle when either side changes), or agree on one standard: a fixed way to say ‘here is what I offer’ and ‘please do this’. The standard offers exactly three kinds of thing: **actions** the model may ask to run, **documents** it may read, and **ready-made instruction templates** it may use. Everything else is plumbing.

TOY

TOY: 3 carriers × 2 tools each = 6 welds to build and maintain, one per pair. One standard plug: 3 carriers each build to it once; every tool on the model side connects to all three.

One more plain idea, because the standard changed this year. Old version: the driver and the bay had a long conversation at the start (‘hello, what version are you, what can you do’) and remembered it for the whole visit. New version: **every single letter carries its own full address** — who is asking, which version, what they can handle — so any bay attendant can answer any letter without having met the driver before. That is the whole of the change, and it is why the new standard works behind a row of identical bays instead of one.

Prepared by Ravi Ranjan · CCAO-F, CCDV-F & CCAR-F

p. 70

**R3**THE REAL  
NAME**The real name — primitives, tools / resources / prompts, JSON-RPC, client and server; the 2026-07-28 DRIFT****WORD CARD · primitive (MCP)**

|                              |                                                                                                                                   |
|------------------------------|-----------------------------------------------------------------------------------------------------------------------------------|
| <b>PLAIN MEANING</b>         | One of the three kinds of thing an MCP server can offer: tools, resources, prompts — the whole vocabulary of the plug.            |
| <b>WHAT IT IS NOT</b>        | Not a low-level data type; here ‘primitive’ means ‘a basic offering’.                                                             |
| <b>HEARD AT KAVERI</b>       | “kaveri_mcp.py offers two tools, one resource (sop://214) and one prompt (customs_triage) — all three primitives in forty lines.” |
| <b>HOW THE EXAM WORDS IT</b> | “MCP primitives”, “expose as a tool vs a resource”, “server capabilities”.                                                        |
| <b>EXPANDS</b>               | <i>Loading Dock glossary ‘MCP server/resource/prompt’ · E1</i>                                                                    |

**WORD CARD · MCP tool / resource / prompt**

|                              |                                                                                                                                                                                                                |
|------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>PLAIN MEANING</b>         | Tool = an action the model may ask to run (get_shipment_status). Resource = a document the model may read by address (sop://214). Prompt = a reusable instruction template the server offers (customs_triage). |
| <b>WHAT IT IS NOT</b>        | ► Resource is not ‘resources’ in the budget sense; ► prompt is not the system prompt. A resource is read, a tool is run, a prompt is filled in.                                                                |
| <b>HEARD AT KAVERI</b>       | “Read the SOP as a resource, run the lookup as a tool, start triage from the prompt — three primitives, three verbs.”                                                                                          |
| <b>HOW THE EXAM WORDS IT</b> | “expose the SOP as a resource rather than a tool”, “server-provided prompt”, “tool naming mcp_server_tool”.                                                                                                    |
| <b>EXPANDS</b>               | <i>Loading Dock glossary ‘MCP server / resource / prompt’ · E1 · Atlas A4</i>                                                                                                                                  |

► **SAME WORD, TWO WORLDS · resource** — **(1)** MCP sense: a document a server offers by address (sop://214) — read, not run. **(2)** budget sense: people, money, compute — ‘we lack the resources’. **The tell:** a URI or ‘read’ → MCP sense; a plan or a headcount → budget sense.

**WORD CARD · client / server (MCP roles)**

|                              |                                                                                                                                                                                                                 |
|------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>PLAIN MEANING</b>         | The server is the program that offers primitives (kaveri_mcp.py); the client is the model-side program that discovers and uses them (KAVI, via the Agent SDK); the host is the application the client lives in. |
| <b>WHAT IT IS NOT</b>        | ► Not web client/server in the browser sense — an MCP server can run as a child process on the client's own machine (stdio).                                                                                    |
| <b>HEARD AT KAVERI</b>       | “KAVI is the client; kaveri_mcp.py is the server; tools arrive named mcp_kaveri_get_shipment_status.”                                                                                                           |
| <b>HOW THE EXAM WORDS IT</b> | “the MCP client connects to...”, “server exposes...”, “host application”.                                                                                                                                       |
| <b>EXPANDS</b>               | <i>Loading Dock glossary ‘MCP server’ · Agent SDK MCP integration</i>                                                                                                                                           |

**WORD CARD · JSON-RPC**

|                              |                                                                                                                                        |
|------------------------------|----------------------------------------------------------------------------------------------------------------------------------------|
| <b>PLAIN MEANING</b>         | The letter format MCP messages use: a small JSON envelope with a method name, parameters, and an id so replies match requests.         |
| <b>WHAT IT IS NOT</b>        | Not something you write by hand — the SDK does; you only need to know every MCP message is one of these envelopes on either transport. |
| <b>HEARD AT KAVERI</b>       | “Every call to kaveri_mcp.py is a JSON-RPC letter over stdio today and over Streamable HTTP when hosted.”                              |
| <b>HOW THE EXAM WORDS IT</b> | “JSON-RPC 2.0 messages”, “request/response envelope”, “_meta on every request” (the 2026-07-28 change).                                |
| <b>EXPANDS</b>               | <i>Loading Dock DRIFT box ‘MCP spec 2026-07-28’</i>                                                                                    |

| WORD CARD · stateless (MCP 2026-07-28) |                                                                                                                                                                                                                   |
|----------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| PLAIN MEANING                          | Each request carries everything the server needs — protocol version, client identity, capabilities — in its <code>_meta</code> , so no session is kept and any server instance behind a load balancer can answer. |
| WHAT IT IS NOT                         | Not ‘the server has no data’ (it may have a database); stateless is about the conversation, not the storage.                                                                                                      |
| HEARD AT KAVERI                        | “The hosted kaveri server can run as three identical copies behind one address because every letter carries its own full address.”                                                                                |
| HOW THE EXAM WORDS IT                  | “stateless protocol”, “no initialize handshake”, “server/discover”, “horizontal scaling of MCP servers”.                                                                                                          |
| EXPANDS                                | Loading Dock DRIFT box ‘MCP spec 2026-07-28’                                                                                                                                                                      |

| WORD CARD · elicitation |                                                                                                                                                                                                          |
|-------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| PLAIN MEANING           | The server asking the caller for more information mid-request — under the 2026-07-28 spec, by returning ‘input required’ with its questions, and the client retrying the call with the answers attached. |
| WHAT IT IS NOT          | Not the model asking the user (that is the host's job) and not the old held-open stream the earlier spec used for server-initiated requests.                                                             |
| HEARD AT KAVERI         | “customs_triage returns <code>input_required</code> : ‘which bonded yard?’ — KAVI asks the dispatcher and retries with the answer.”                                                                      |
| HOW THE EXAM WORDS IT   | “server needs additional input from the user”, “multi round-trip request”, “elicitation”.                                                                                                                |
| EXPANDS                 | Loading Dock DRIFT box ‘MCP spec 2026-07-28’ (elicitation current)                                                                                                                                       |

| DRIFT · MCP spec 2026-07-28 (carried verbatim from the Loading Dock; re-verified 28 Sep 2026 — holds)                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                 |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                          |
|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Loading Dock / older workbook reading                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                 | Current docs (checked 28 Sep 2026)                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                       |
| Stateless; <code>_meta</code> on every request; <code>server/discover</code> ; sampling & logging deprecated; transports stdio + Streamable HTTP; SSE-as-transport gone; elicitation current.                                                                                                                                                                                                                                                                                                                                                                                                                                                                                         | Unchanged, with detail: the initialize/initialized handshake is eliminated and each request carries protocol version, client identity and capabilities in <code>_meta</code> ; <code>server/discover</code> is an optional discovery RPC; sampling, logging and roots are deprecated with a twelve-month support window; HTTP+SSE is deprecated, stdio and Streamable HTTP are current; server-initiated requests become multi-round-trip ( <code>input_required</code> ); list responses carry <code>ttlMs</code> / <code>cacheScope</code> ; authorization hardened (issuer validation, client-ID metadata documents). |
| <b>IN PLAIN WORDS</b> · What ‘stateless’ means: every letter carries its own full address, so any of several identical servers can answer it and nothing has to be remembered between letters. Why an older workbook or stem may still show the old reading: most training material describes the 2025 spec — handshake first, sessions, SSE streams, sampling as a live feature. What to answer: <b>if the stem says 2026-07-28 or ‘current spec’, answer stateless / <code>_meta</code> / Streamable HTTP / elicitation-by-retry; if it describes a handshake and sessions, it is describing the older revision — say so, and answer within that frame.</b> Never silently resolve. |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                          |

Naming, carried from CCAR-F: tools arrive on the model side as `mcp_server_tool` — `mcp_kaveri_get_shipment_status` — and `allowed-tools` pre-approval is turn-scoped. The Dock’s E1 grows `kaveri_mcp.py` to all three primitives with the `__name__` guard so the import probe runs free before any paid call; on Day B, watch the line that registers the resource address and the line that prints the `mcp_kaveri_` names.

**WRONG BELIEF, CORRECTED**

|                                  |                                                                                                                                                                                                                                                                       |
|----------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>I used to think...</b>        | an MCP server is always the professional choice                                                                                                                                                                                                                       |
| <b>Why it feels right</b>        | It is the standard, it is what the certification talks about, and the Dock's own server is the hero of E1.                                                                                                                                                            |
| <b>The correction</b>            | <b>MCP earns its keep when several clients need the same capability or the capability changes often. One caller, one stable internal API, no reuse — a direct call (a weld) is smaller, faster and honest. The professional choice is the one the counts justify.</b> |
| <b>The stem that punishes it</b> | A stem where 'wrap the internal billing API in an MCP server' loses to 'call it directly' for a single-consumer, stable integration.                                                                                                                                  |

**ADD IT · an MCP server (adding is the right move here)**

A stem describes three carrier APIs that change quarterly, consumed by KAVI, the dispatch tool and a reporting agent. Building one MCP server per carrier (or one facade) is **the right move**: many clients × churn is exactly what the standard plug is for. Welds here would be nine connections to rebuild every quarter.

**R4****THE EXAM****The exam — how a 3.7 stem hides the counts**

**The disguise.** A protocol question dressed as a preference. The tell is two counts: how many clients will use the capability and how often it changes; plus the direction of autonomy — if the other side is itself an agent with its own goals and tools, it is agent-to-agent, not a tool.

**Family baited:** OE (MCP for a one-off) and CM (MCP where the other party is an agent).

**Mini-stem 1.** One nightly script reads one stable internal endpoint. — **Shape: direct API call; no server; keep it in one file.**

**Mini-stem 2.** A partner runs its own customs-clearance agent that negotiates document lists and asks clarifying questions. — **Shape: agent-to-agent: exchange goals and results, not tool calls; keep the boundary and the audit on your side.**

**PENCIL LAB P10 · Decide MCP / weld / agent-to-agent for six integrations by counting clients and churn**

For each integration write two numbers — **clients** (how many distinct programs will use it) and **churn** (changes per year) — and whether the other side is an **agent**; then choose: **MCP server**, **weld** (direct API/CLI call), or **agent-to-agent**. (1) Three carrier ETA APIs used by KAVI, the dispatch tool and the reporting job; change quarterly. (2) The nightly re-index reading the SOP bucket; one caller; stable. (3) The GST portal's e-way bill service; one caller today, three next year; changes with every budget. (4) A partner's customs-clearance agent that asks clarifying questions. (5) An internal FX-rate endpoint; one caller; unchanged in four years. (6) The TMS shipment lookup used by KAVI, dispatch, the customer portal and two carriers; changes twice a year.

----- ∞ fold here — the worked answer is below the line -----

**WORKED ANSWER · P10**

(1) clients 3, churn 4/yr → **MCP**. (2) clients 1, churn ~0 → **weld**. (3) clients 1 → 3, churn high → **MCP** (build for next year's clients; the churn alone argues for one place to absorb it). (4) the other side is an agent with its own goals → **agent-to-agent**, with your audit and scope boundary intact. (5) clients 1, churn 0 → **weld**; an MCP server here is OE. (6) clients 5, churn 2/yr → **MCP**: the canonical case.

**What you should feel:** the decision is a multiplication (clients × churn) plus one yes/no (is it an agent?). Atlas A4 is this lab drawn as a decision tree — redraw it from memory on Day D.

## CHECK YOURSELF

1. Name the three primitives and their verbs.
2. What does 'stateless' mean in the 2026-07-28 spec?
3. MCP, weld or agent-to-agent — what two counts and one question decide?
4. Explain to Meera in two sentences why the carriers get a standard plug and the FX feed gets a weld.

----- ∞ fold — answers below -----

1. Tool — run; resource — read; prompt — fill in.
2. Every request carries version, identity and capabilities in `_meta`; no handshake, no session; any server instance can answer.
3. Number of clients, rate of change; and is the other side an agent with its own goals?
4. Three carriers, three of our systems, and they all change every quarter — one standard connector means we rebuild nothing when they do. The FX feed has one reader and hasn't changed in four years; a direct call is smaller and honest.

### WHAT WATSON WROTE DOWN

- Three primitives: tool (run) · resource (read, by URI) · prompt (template). ▸ resource ≠ budget resources; ▸ prompt ≠ system prompt.
- Client = model side (KAVI); server = offers primitives (kaveri\_mcp.py); host = the app. Names arrive as `mcp__server__tool`.
- Spec **2026-07-28**: stateless (`_meta` on every request), `server/discover`, sampling/logging/roots deprecated, stdio + Streamable HTTP, HTTP+SSE gone, elicitation = `input_required` + `retry`.
- Transport shapes authN (Station 3): stdio = same building; Streamable HTTP = highway, token per request.
- MCP vs weld vs A2A = clients × churn, plus 'is the other side an agent?'. One caller + stable = weld; many + churn = MCP; agent = A2A.
- **'MCP is always professional'** is the HL bait; **ADD IT** when the counts say so.

**Law in force:** D3 Law 1 (every door) and D3 Law 5 (map first) both in force; no new law here.

**Read next in the Loading Dock:** ch. 3.7 · E1 (the import probe; the resource address; the `mcp__kaveri__names`) · Atlas A4 · DRIFT box 'MCP spec 2026-07-28' · then Station 8

TONIGHT YOU WILL

Sunday 4 Oct (Day B) · second 15 minutes · Station 8 · The catalogue at the gate

☐ see the difference between handing the driver the catalogue and handing him the warehouse

☐ understand prompt caching in plain words — why there is a minimum size and why reads are ~90 % off — and read the UPDATED DRIFT

☐ sum the two context bills from E8's line items, corrected, and find the corpus size at which the cached monolith wins (P9)

☐ learn the small-stable-always-used flip, and context editing in plain words (the server sweeps the desk)

☐ own the words: progressive disclosure, monolithic context, prompt cache, cache minimum, context editing, tool search / deferred tools, round-trip

☐ meet **D3 Law 5**: send the map first, the territory on demand; carry **362,460 / 4,205 / 4,500** (corrected) · **~100k trigger / keep 3**

Then: read Loading Dock ch. 3.8 with the CANON ERRATUM box open, then type E5-E8. In E8, expect 4,205, not 4,625.

STATION 8

The catalogue at the gate

Evaluate progressive discovery vs monolithic context strategy — the map first, the territory on demand

OBJECTIVE

3.8

expands Loading Dock ch. 3.8 · E8 · Atlas A7 · glossary (progressive disclosure, monolithic context) · DRIFT boxes 'context editing' and 'cache minimums'

CANON ERRATUM · Loading Dock E8 / Atlas A7

The Dock's E8 expected output prints the progressive bill as **4,625** tokens per call and the gap as **78×**; Atlas A7's '4.6k' inherits the slip. Its own line items sum differently: **9 × 25 + 260 + 20 × 180 + 120 = 225 + 260 + 3,600 + 120 = 4,205**, and **362,460 ÷ 4,205 ≈ 86×** (4,625 would give 78×). Re-done by hand on 28 Sep 2026 and confirmed. The Walk prints **4,205** and **~86×** everywhere; the Dock's E8 expected output and Atlas A7 need a patch before Sunday's typing — when you type E8, expect **4,205** and treat a printed 4,625 as the known slip, not as your bug.

## Climb 1 · The catalogue, the warehouse, and the map that is sometimes the territory

R1

THE SCENE

The scene — what you hand the driver at the gate

Sunday, 11 am. A new driver at the gate needs one thing from the warehouse: the customs-hold checklist. Two ways to help him. Hand him the **warehouse** — every rack, every box, every document, wheeled out to the gate on forty pallets, most of which he will push past to find one page. Or hand him the **catalogue**: two pages listing what is in there, one line each, and a runner who fetches whatever he points at. The catalogue is fast to read and almost weightless; the fetch costs a trip. The warehouse costs forty pallets every time, for every driver, and he still has to find the page.

But the old foreman, leaning on the gate: “The drivers who come every morning for the same three things? Don't make them read a catalogue. Put those three things on a shelf by the gate. It's small, it never changes, and everyone uses it.”

## R2

## PLAIN WORDS

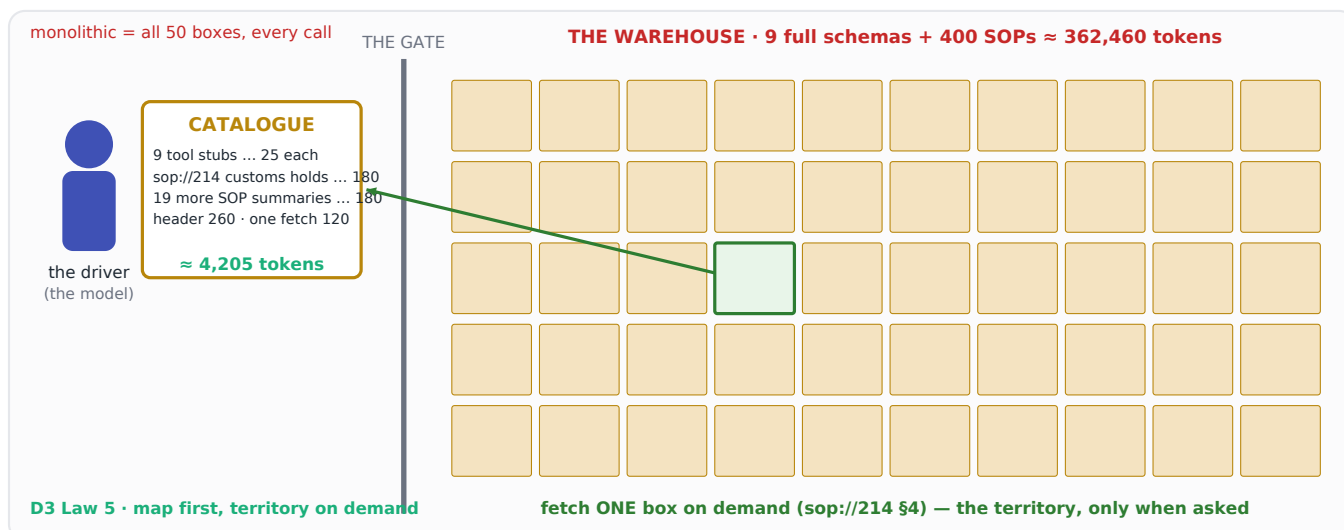
**Plain words — a map is cheap to carry; the territory is fetched when pointed at; unless the territory is small and always needed**

The model can only ‘see’ what is in the window, and everything in the window costs on every call. Two strategies. **Everything in:** put every tool description and every document in the window, always — nothing is ever missing, and every call pays for all of it. **Map first:** put a short catalogue in the window — one line per tool, one line per document — and let the model ask for the full description or the full document only when it needs it; every call pays for the catalogue, plus one extra trip when something is fetched. The map wins by a huge margin when the territory is large; the extra trip is the price.

## TOY

TOY: 12 documents of 1,000 words each. Everything in: 12,000 words per question. Map first: 12 one-line entries (~120 words) + 1 fetched document (1,000) = 1,120 words per question, plus one extra trip. Ten times lighter — unless the question needs all twelve.

The foreman's shelf by the gate is the flip: when the set is **small, stable and used on every call** — the three standing tools and the standing instructions — it is cheaper to leave it in the window every time *and remember it* than to fetch it. ‘Remember it’ is the next climb; it is what makes the flip pay.



**W17** · The catalogue at the gate. The driver holds a two-page catalogue (about 4,205 tokens) and points at one box, which the runner fetches. Behind the gate, the warehouse — every schema and every SOP — is about 362,460 tokens, and wheeling it out on every call is the monolithic strategy. Redraw from memory on Day D — it is Atlas A7's left half.

## R3

## THE REAL NAME

**The real name — progressive disclosure, monolithic context, tool search, round-trips; D3 Law 5****WORD CARD · progressive disclosure (progressive discovery)**

|                              |                                                                                                                                                                                              |
|------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>PLAIN MEANING</b>         | Sending the model a short index of what exists — tool names with one-line descriptions, document titles — and letting it fetch the full schema or document only when it decides it needs it. |
| <b>WHAT IT IS NOT</b>        | Not lazy loading of code and not ‘hide tools from the model’: the model can see everything exists; it only pays for what it opens.                                                           |
| <b>HEARD AT KAVERI</b>       | “KAVI sees nine tool stubs and twenty SOP titles ( $\approx 4,205$ tokens) and fetches <code>sop://214</code> when Padma's hold comes up.”                                                   |
| <b>HOW THE EXAM WORDS IT</b> | “progressive disclosure”, “list tools then fetch on demand”, “defer_loading”, “tool search”, “reduce context by loading tool definitions lazily”.                                            |
| <b>EXPANDS</b>               | <i>Loading Dock glossary ‘progressive disclosure’ (‘hand the driver the catalogue, not the warehouse’) · E8 · Atlas A7</i>                                                                   |

**WORD CARD · monolithic context**

|                              |                                                                                                                        |
|------------------------------|------------------------------------------------------------------------------------------------------------------------|
| <b>PLAIN MEANING</b>         | Putting every tool schema and every document into the window on every call, so nothing ever has to be fetched.         |
| <b>WHAT IT IS NOT</b>        | Not always wrong: for a small, stable, always-used set it is the cheaper, more accurate choice — especially cached.    |
| <b>HEARD AT KAVERI</b>       | “All nine schemas plus all 400 SOPs, every call: 362,460 tokens before the question — the warehouse on forty pallets.” |
| <b>HOW THE EXAM WORDS IT</b> | “include all tools and documents in every request”, “monolithic context”, “stuff the context window”.                  |
| <b>EXPANDS</b>               | <i>Loading Dock glossary ‘monolithic context’ · E8 · Atlas A7</i>                                                      |

**WORD CARD · tool search / deferred tools**

|                              |                                                                                                                                                                                                                                                                                                                                                                                         |
|------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>PLAIN MEANING</b>         | The API's built-in form of the catalogue: tools marked <code>defer_loading</code> are not in the window until the model searches for them by name or description ( <code>tool_search_tool_regex_20251119</code> / <code>tool_search_tool_bm25_20251119</code> ) — recommended past ~10 tools or ~10k tokens of definitions; every MCP tool in a toolset can be deferred in one setting. |
| <b>WHAT IT IS NOT</b>        | Not a beta any more (GA as of Sep 2026) and not free of rules: keep the 3–5 most-used tools non-deferred, and never defer the search tool itself.                                                                                                                                                                                                                                       |
| <b>HEARD AT KAVERI</b>       | “Defer everything on the carrier toolset; keep <code>get_shipment_status</code> and <code>lookup_order</code> loaded — the shelf by the gate.”                                                                                                                                                                                                                                          |
| <b>HOW THE EXAM WORDS IT</b> | “tool search tool”, “ <code>defer_loading: true</code> ”, “>10 tools / >10k tokens of definitions”, “keep frequently used tools non-deferred”.                                                                                                                                                                                                                                          |
| <b>EXPANDS</b>               | <i>Loading Dock ch. 3.8 (progressive discovery) — the API-native form; docs checked 28 Sep 2026</i>                                                                                                                                                                                                                                                                                     |

**WORD CARD · round-trip (extra fetch)**

|                              |                                                                                                                                                                        |
|------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>PLAIN MEANING</b>         | The price of the catalogue: one more request-response cycle when the model decides to open a schema or a document — latency and a few tokens.                          |
| <b>WHAT IT IS NOT</b>        | ► Not a conversation turn (a user message and a reply); a round-trip is one fetch inside the machinery of answering a single question.                                 |
| <b>HEARD AT KAVERI</b>       | “The customs-hold answer costs one round-trip to fetch <code>sop://214 §4</code> — about 120 tokens and a hundred milliseconds — against 362,460 tokens of warehouse.” |
| <b>HOW THE EXAM WORDS IT</b> | “additional latency from on-demand fetching”, “extra round-trip per tool discovery”.                                                                                   |
| <b>EXPANDS</b>               | <i>Loading Dock E8 (the <code>fetch_on_demand</code> pattern)</i>                                                                                                      |

► **SAME WORD, TWO WORLDS · turn** — **(1)** conversation sense: one user message and one assistant reply. **(2)** round-trip sense: one internal fetch inside answering a single message. **The tell:** ‘user says ... model replies’ → turn; ‘fetch / look up / discover’ → round-trip.

The canon bills, corrected: monolithic = **362,460** tokens per call (all schemas + all SOPs); progressive = **4,205** tokens per call (nine catalogue stubs × 25 + a 260-token catalogue header + twenty SOP summaries × 180 + a 120-token fetch), a gap of **≈ 86×**. The Dock prints 4,625 and 78× — see the CANON ERRATUM box at the head of this station. P9 sums the line items by hand.

**D3 LAW 5 · Send the map first, the territory on demand.**

Born here. Small, stable, always-used things ride in the window (and are cached); everything else is listed by name and fetched when pointed at. Broken: all 400 SOPs in every call ‘so the model never lacks context’ — 362,460 tokens before the question, and the model still has to find §4 in the pile.

### ADD IT · progressive discovery (tool search) for a large toolset (adding is the right move here)

A stem describes an agent aggregating four MCP servers with 200+ tools whose definitions cost ~55k tokens per call. Adding the tool-search tool with defer\_loading on the toolsets is **the right move**: the docs' own thresholds are 10+ tools or >10k tokens of definitions, and the saving is >85 %. It would be OE on a three-tool agent — that set rides on the shelf by the gate.

#### R4

##### THE EXAM

### The exam — how a 3.8 stem hides the flip

**The disguise.** A big number (tools, documents, tokens) and a reflex answer ('load everything so nothing is missed') or its mirror ('defer everything to save tokens'). The tell is the three adjectives: small, stable, always used → monolithic and cached; any of the three missing → progressive, with the extra round-trip priced.

**Family baited:** HL ('progressive is always leaner') and OE (a catalogue for three tools).

**Mini-stem 1.** Agent with 3 tools and a 4,500-token standing prompt, identical on every call. —

**Shape: monolithic, cached: everything rides in the window; reads at ~10 % — a catalogue would add a round-trip for nothing.**

**Mini-stem 2.** Agent with 180 tools across five MCP servers and a 400-document SOP store. —

**Shape: progressive: tool search with deferred toolsets for the tools; a retrieval pipeline (Station 1) for the documents; keep the 3-5 daily tools loaded.**

### CHECK YOURSELF

1. What does the model pay for under each strategy?
2. Quote the corrected canon bills.
3. When does monolithic win?
4. Explain D3 Law 5 to Meera in two sentences.

----- ∞ fold — answers below -----

1. Monolithic: everything, every call. Progressive: the catalogue every call plus one round-trip per thing it opens.
2. 362,460 monolithic vs 4,205 progressive per call, ≈ 86×; the Dock's 4,625 / 78× is the erratum.
3. Small, stable, always-used sets — especially cached.
4. We hand KAVI a two-page catalogue of what exists and let it fetch the one document it needs, instead of wheeling out the whole warehouse on every question. The three things it uses every time stay on a shelf by the gate.

## Climb 2 · Remembering the shelf, and sweeping the desk

#### R1

##### THE SCENE

### The scene — the guard who reads the same three pages every morning

Monday. Every morning the same guard reads the same three laminated pages at the gate — the standing rules, the three daily tools, the carrier list — before the first truck. Twenty minutes, every day, for pages that have not changed since March. Meera: "Read them once. Sign the bottom. Tomorrow, check the signature and get on with it." The guard: "And if the pages change?" "Then you read them again, once." And a second instruction, for the desk inside: "The pile of yesterday's fetched documents — sweep it at lunch. Keep the last three; the rest go back to the shelf. Nobody can think at a desk buried in Tuesday."

**R2****PLAIN WORDS****Plain words — pay once to remember what does not change; let the desk be swept**

If the first part of what you send is identical every call — the standing instructions, the tool descriptions, the catalogue — the provider can remember it: you pay full price once to store it, and a small fraction each time it is re-read, until it changes. Two rules follow. It only works on a block that is **big enough to be worth remembering** — below a minimum size the provider will not bother, and the ‘saving’ is zero (P11 caught this). And it only helps on the **unchanging prefix**: anything that varies from call to call must come after the remembered block, or it breaks the match.

**TOY**

TOY: a 12-page briefing read every morning at 10 rupees a page: ₹120 a day. Remember it: ₹150 once (a little extra to file it), then ₹12 a day to re-check. By day two you are ahead; by the month you have paid ₹500 instead of ₹3,600.

The desk-sweeping: over a long job the window fills with the results of old fetches. The server can be told to clear the oldest tool results automatically once the window passes a size, keeping the last few — so the model keeps its recent working set and loses the clutter, without you rebuilding the conversation by hand.

**MONOLITHIC · everything in the window****362,460 tokens/call**

9 schemas + all 400 SOPs, every call

**PROGRESSIVE · catalogue first, fetch on demand****4,205 tokens/call**

9 stubs × 25 + 260 header + 20 summaries × 180 + 120 fetch (corrected)

**SMALL STABLE SET · cached****4,500 tokens/call**

4,500 tokens, always used, cache-read at ~10 % → ≈ 450 effective

**362,460 ÷ 4,205 ≈ 86× (the Dock's E8 prints 4,625 and 78× — CANON ERRATUM)**

**W16** · The three context bars: the monolithic warehouse, the corrected progressive catalogue, and the small stable always-used set that — cached — costs about a tenth of its size to re-read. Redraw from memory on Day D; it is Atlas A7's right half with the erratum fixed.

**R3****THE REAL NAME****The real name — the prompt cache (Chapter 0 card), cache minimum, context editing; two DRIFT boxes****WORD CARD · cache minimum**

|                              |                                                                                                                                                |
|------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>PLAIN MEANING</b>         | The smallest prefix a given model will bother to remember; below it, <code>cache_control</code> silently does nothing.                         |
| <b>WHAT IT IS NOT</b>        | Not one number: it varies by model — 4,096 on Haiku 4.5; 1,024 on Sonnet 4.x; 512 on the newest tier (see the UPDATED DRIFT).                  |
| <b>HEARD AT KAVERI</b>       | “An 800-token standing prompt on Haiku 4.5 never caches; bundle it with the schemas past 4,096, or it is a dial that does nothing.”            |
| <b>HOW THE EXAM WORDS IT</b> | “minimum cacheable prompt length”, “the cache is not being hit”, “Haiku requires 4,096 tokens”.                                                |
| <b>EXPANDS</b>               | <i>Loading Dock DRIFT box ‘cache minimums by tier (1,024–4,096; Haiku 4.5 = 4,096; reads ~90 % off)’ — updated in this book's front matter</i> |

**WORD CARD · context editing (server-side)**

|                              |                                                                                                                                                                                                 |
|------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>PLAIN MEANING</b>         | Telling the API to sweep old tool results out of the conversation automatically once it passes a size — keep the last few, clear the rest — without changing the history you hold on your side. |
| <b>WHAT IT IS NOT</b>        | Not summarisation and not memory: cleared results are gone from what the model sees (unless the memory tool wrote them down first); your client still holds the full transcript.                |
| <b>HEARD AT KAVERI</b>       | “Trigger at ~100k input tokens, keep the last 3 tool results, never clear web_search results — the desk swept at lunch.”                                                                        |
| <b>HOW THE EXAM WORDS IT</b> | “clear_tool_uses_20250919”, “trigger / keep / clear_at_least / exclude_tools”, “clear_thinking_20251015”, “memory_20250818”, “long-running agent context growth”.                               |
| <b>EXPANDS</b>               | Loading Dock DRIFT box ‘server-side context editing’                                                                                                                                            |

► **SAME WORD, TWO WORLDS · cache** — **(1)** prompt-cache sense (this station): the provider remembers your unchanging prefix; reads at ~10 %. **(2)** browser / CDN sense: a stored copy of a page or file near the user. **The tell:** ‘prefix’, ‘cache\_control’, ‘minimum tokens’ → prompt cache; ‘page’, ‘CDN’, ‘stale asset’ → browser cache.

**DRIFT · Server-side context editing (carried verbatim from the Loading Dock; re-verified 28 Sep 2026 — holds)**

| Loading Dock / older workbook reading                                                                                                                  | Current docs (checked 28 Sep 2026)                                                                                                                                                                                                                                                                                                                                                                                  |
|--------------------------------------------------------------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| clear_tool_uses_20250919; trigger ~ <b>100k</b> input tokens; <b>keep 3</b> ; exclude_tools; clear_at_least; clear_thinking_20251015; memory_20250818. | Unchanged: clear_tool_uses_20250919 with defaults trigger <b>100,000</b> input tokens, keep <b>3</b> tool uses, optional clear_at_least and exclude_tools, clear_tool_inputs false; clear_thinking_20251015 (model-dependent default keep; must be listed first when combined); memory tool memory_20250818; beta header context-management-2025-06-27; still beta; clearing invalidates the cache from that point. |

**IN PLAIN WORDS** · What it does: the server sweeps the oldest tool results out of the window once it passes the trigger, keeping the last three. Why an older workbook or stem may differ: earlier material assumed you had to prune the conversation yourself on the client. What to answer: **server-side context editing with trigger/keep, exclude what must survive, and pair it with the memory tool if cleared results must be kept somewhere; remember the sweep breaks the cache after that point.**

**UPDATED DRIFT · Cache minimums by tier (carried from the Loading Dock — UPDATED; see front matter)**

| Loading Dock / older workbook reading                                           | Current docs (checked 28 Sep 2026)                                                                                                                                                                                         |
|---------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>1,024–4,096</b> by tier; <b>Haiku 4.5 = 4,096</b> ; reads ~ <b>90 %</b> off. | <b>512–4,096</b> by model: newest tier <b>512</b> ; Opus 4.5 / 4.6 and Haiku 4.5 <b>4,096</b> ; Sonnet 4.x <b>1,024</b> . Reads <b>0.1×</b> standard (deeper on the newest tier); 5-minute writes 1.25×, 1-hour writes 2×. |

**IN PLAIN WORDS** · What to answer: **the minimum varies by model — check the model named; Haiku 4.5 is 4,096; reads are about 90 % off; a prefix under the minimum silently does not cache.** A stem quoting 1,024–4,096 is quoting the older table and is still right about Haiku.

**WRONG BELIEF, CORRECTED**

|                                  |                                                                                                                                                                                                                                                                                               |
|----------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>I used to think...</b>        | progressive discovery is always leaner                                                                                                                                                                                                                                                        |
| <b>Why it feels right</b>        | Ten times fewer tokens in the E8 demo, and every talk says 'don't stuff the context'.                                                                                                                                                                                                         |
| <b>The correction</b>            | <b>Progressive adds a round-trip per fetch and a catalogue on every call. For a small, stable, always-used set, monolithic and cached is lighter and faster — the 4,500-token shelf reads at ~450 effective tokens with no fetch. The counts decide, not the fashion (D3 Law 3's cousin).</b> |
| <b>The stem that punishes it</b> | A stem where deferring three tools behind tool search loses to leaving them loaded and cached.                                                                                                                                                                                                |

**TRAP · The cache that never hits**

Two ways to turn the cache dial and save nothing: a prefix under the model's minimum, and variable content (the date, the user's name, the question) placed *before* the stable block. Both fail silently — the bill just does not fall. The **trap** is an option that 'enables caching' without checking size and order. Tell: a small standing prompt, Haiku, 'the cache hit rate is 0 %'.

**R4****THE EXAM****The exam — how a stem hides the cache and the sweep**

**The disguise.** 'Costs did not fall after enabling caching' (size or order); 'the agent slows and degrades after forty tool calls' (the desk); 'the cache stops hitting mid-conversation' (a sweep or a changed prefix). The tell is a number against the minimum, a changing prefix, or a long-running agent.

**Family baited:** SF (a dial that silently does nothing) and VG (cache read as answer cache).

**Mini-stem 1.** Haiku 4.5, 1,200-token system prompt with cache\_control, 0 % hit rate. — **Shape:** **below the 4,096 minimum; bundle the stable schemas and catalogue into the cached block, or change model.**

**Mini-stem 2.** A research agent's context grows past 150k tokens over an hour and answers degrade. — **Shape:** **server-side context editing: clear old tool results at a trigger, keep the last few, exclude what must survive; memory tool for anything that must persist.**

**PENCIL LAB P9 · Sum the two context bills from E8's line items (corrected) and find where the cached monolith wins**

E8's progressive line items: 9 tool stubs × 25 tokens · a 260-token catalogue header · 20 SOP summaries × 180 tokens · one 120-token fetch. E8's monolithic bill: 362,460 tokens (all schemas + all SOPs). (a) Sum the progressive bill. (b) Divide: the gap. (c) The Dock prints 4,625 and 78× (the canon erratum) — which of the two numbers is inconsistent with the line items? (d) Now the flip: a cached prefix reads at 0.1×. Below what stable-corpus size S does a cached monolith (0.1 × S per call, after the first) cost less than the progressive catalogue (4,205 per call)? (e) E8's second scenario has a 4,500-token small stable always-used set: cached, what does it cost per call, and how does it compare?

----- ∞ fold here — the worked answer is below the line -----

**WORKED ANSWER · P9**

(a)  $225 + 260 + 3,600 + 120 = 4,205$ . (b)  $362,460 \div 4,205 = 86.2\times$ . (c) Both printed figures are the same erratum: 4,625 is 420 tokens above the line items, and  $78\times$  is simply  $362,460 \div 4,625$  — so the gap inherits the error; the line items are the truth. (d)  $0.1 \times S < 4,205 \rightarrow S < 42,050$  tokens: a stable, always-used corpus under about 42k tokens is cheaper to keep in the window and cache than to catalogue — and it saves the round-trip too. (e)  $4,500 \times 0.1 = 450$  effective tokens per call: about a tenth of the progressive bill, with no fetch. That is the flip: small, stable, always used → monolithic and cached.

**What you should feel:** the erratum was found by adding four numbers; the flip was found by dividing one. Both are pencil work. When E8 prints 4,625 on Day B, you already know why.

## CHECK YOURSELF

1. What does a prompt cache store, and what does it cost?
2. Two silent ways a cache never hits?
3. Quote context editing's defaults.
4. Below what stable corpus size does the cached monolith beat the 4,205 catalogue?
5. Explain to Meera in two sentences why the guard signs the pages once.

----- ∞ fold — answers below -----

1. Your unchanging prefix;  $\sim 1.25\times$  to write once,  $\sim 0.1\times$  to read on later calls until it changes.
2. Prefix below the model's minimum; variable content placed before the stable block.
3. `clear_tool_uses_20250919`: trigger 100k input tokens, keep 3 tool uses; `exclude_tools` and `clear_at_least` optional; `clear_thinking_20251015`; `memory_20250818`.
4. About 42,050 tokens ( $0.1 \times S < 4,205$ ).
5. The same three pages cost twenty minutes every morning and never change; reading them once and checking the signature costs almost nothing. If the pages change, she reads them again — once.

### WHAT WATSON WROTE DOWN

- **D3 Law 5**: map first (catalogue  $\approx 4,205$  tokens, corrected), territory on demand (one round-trip); warehouse = **362,460**; gap  $\approx 86\times$ . **Dock prints 4,625 / 78x — canon erratum.**
- The flip: small + stable + always used  $\rightarrow$  monolithic and cached. **4,500**-token shelf  $\rightarrow \sim 450$  effective/call. Cached monolith wins below  $\sim 42k$  stable tokens.
- Tool search = the API's catalogue: `defer_loading`, `tool_search_tool_regex/bm25_20251119`; past  $\sim 10$  tools / 10k tokens; keep 3–5 daily tools loaded; GA.
- Prompt cache: stable prefix first; write  $1.25\times$ , read  $\sim 0.1\times$  ( **$\sim 90\%$**  off); minimum varies — **Haiku 4.5 = 4,096**, Sonnet 4.x **1,024**, newest tier **512** (UPDATED DRIFT). Under the minimum = silent zero.
- Context editing: `clear_tool_uses_20250919`, trigger  **$\sim 100k$** , **keep 3**, `exclude_tools`, `clear_at_least`; `clear_thinking_20251015` first when combined; `memory_20250818` to keep what is swept; the sweep breaks the cache after it.
- $\blacksquare$  cache  $\neq$  browser cache ·  $\blacksquare$  turn  $\neq$  round-trip.

**Law in force:** D3 Law 5 · Send the map first, the territory on demand.

**Read next in the Loading Dock:** ch. 3.8 with the ERRATUM open · E8 (expect 4,205) · Atlas A7 ('4.6k'  $\rightarrow$  4,205) · DRIFT boxes 'context editing' and 'cache minimums' (updated) · then the exhibit companions

# Exhibit companions — when you type this, watch...

The code stays in the Loading Dock; you hand-type E1-E8 there on Days A and B, prediction first, DRY\_RUN=1 first. Each quarter-page below names the two or three lines that carry the station's lesson and what the printed output should make you feel. Nothing here is typed. The line names are descriptions, not numbers, so they survive any re-formatting of the Dock.

## E1 · kaveri\_mcp.py grown up (3.7 · Station 7 · type on Day A)

### Watch these lines:

- ▶ the line that registers the resource by address (`sop://214`) — a resource is read, not run
- ▶ the line that registers the prompt (`customs_triage`) — the third primitive, a template
- ▶ the `if __name__ == "__main__": mcp.run()` guard — the import probe runs free before any paid call, and the tools arrive on the model side as `mcp_kaveri_...`

**What the printed output should make you feel:** three primitives in forty lines, and the probe costing nothing — the standard plug is small.

## E2 · The bloat audit (3.1 · Station 5 · type on Day A)

### Watch these lines:

- ▶ the loop that sums each schema's token count — nine weights on the belt
- ▶ the two lines that print the context bill before and after the trim: **2,314** → **811**
- ▶ the multiplication by **40,000** — **60,120,000** tokens a day the six unused tools cost

**What the printed output should make you feel:** a number that nobody would notice per call and everybody notices per month; and the two lookalike names sitting next to each other in the list.

## E3 · The gate pass (3.2 · Station 3 · type on Day A)

### Watch these lines:

- ▶ the allowlist check that runs *before* `process_refund`'s body — the lock, in code, not in the prompt
- ▶ the line that writes a **DENIED** entry to the ledger with identity, action, target and reason
- ▶ the carrier token's scope before and after narrowing — **all** → **own** — one line

**What the printed output should make you feel:** that nothing was forged and everything was permitted until one word changed; and that the denial line is the gate working, not an error.

## E4 · Chunking three ways (3.5 · Station 1 · type on Day A)

### Watch these lines:

- ▶ the fixed-size cut — count where the boundary falls inside §4
- ▶ the line that prints *which* chunk was retrieved for the e-way bill question under each strategy
- ▶ the heading-aware cut's chunk that begins at §4 and holds all six steps

**What the printed output should make you feel:** the same question, two chunkers, one fluent wrong answer (cancellation) and one right one — D3 Law 2 in a single printed line.

## E5 · The index and the hybrid (3.5 / 3.6 · Station 1-2 · type on Day B)

### Watch these lines:

- ▶ the stub embedder under `DRY_RUN` — positions without a paid call
- ▶ the two score columns — keyword and vector — printed side by side before fusion
- ▶ the fused column and the query on which each single channel fails

**What the printed output should make you feel:** P3 in code: the register wins the code, the foreman wins the paraphrase, and the fused list puts §4 on top either way.

**E6 · Retrieval by data shape** (3.6 · Station 2 · type on Day B)**Watch these lines:**

- ▶ the routing line that sends the consignment id to a structured lookup — *no RAG*
- ▶ the ‘how many late this week’ query going to the aggregate route — and, mis-routed on purpose, to retrieval
- ▶ the cost line for the mis-route: ₹1,800 against ₹4

**What the printed output should make you feel:** a fluent wrong number and its price; and the router from D1 with retrieval as one of its destinations, not the destination.

**E7 · The dock cameras** (3.4 · Station 4 · type on Day B)**Watch these lines:**

- ▶ the line that creates the trace id at entry and the three places that log it
- ▶ the per-stage counters — tokens, latency, cost — labelled by stage and route only (notice what is *not* a label)
- ▶ the two lines that print mean and p95 side by side: 310 ms and 2,900 ms; the slow ticket's 4,100 ms total

**What the printed output should make you feel:** the gap between the two latency lines — that gap is the whole of 3.4 — and a wall you could read.

**E8 · The catalogue at the gate** (3.8 · Station 8 · type on Day B)**Watch these lines:**

- ▶ the `list_tools` stubs and the twenty SOP summaries — the catalogue,  $\approx$  4,205 tokens
- ▶ the `fetch_on_demand` call — one round-trip, one document
- ▶ the two context bills — 362,460 and 4,205 — and the small-stable-always-used case that flips to monolithic + cache

**What the printed output should make you feel:** that the printed progressive bill may read 4,625 / 78x — the known CANON ERRATUM, not your bug; the line items sum to 4,205 / ~86x. Then the flip: a 4,500-token shelf, cached, at ~450 effective.

**CANON ERRATUM · Loading Dock E8 / Atlas A7**

E8 companion reminder: the Dock's expected output prints 4,625 tokens/call and 78x; the line items sum to 4,205 and the gap is  $\approx$  86x. Patch the Dock's E8 and Atlas A7 before Sunday; do not ‘fix’ your code to match 4,625.

# The walk ends — Monday, KF-2481

Monday, 9.40 am. The re-indexed shelves, cut at the headings and labelled, hand the clerk SOP 214 §4 whole; the register and the foreman agree on it; the supervisor puts it on top. KAVI reads six steps and cites the section. Sunita generates the fresh bill from the original invoice, enters the bonded yard as the dispatch-from place, attaches the officer's release note, and sends the new bill number to the transporter — through a carrier token that can now read exactly one carrier's consignments and nothing else, with the denial of its first wrong-bay attempt sitting in the ledger. The trace for the call shows router 12 ms, retrieval 280 ms, generate 310 ms; the wall shows a p95 under the 4 s promise all morning; the bell stays quiet. At 11.05 the sofa leaves the yard.

The lesson, priced: a fixed-size chunker cost one customer a weekend and a wrong answer that was never an error. A mis-routed count cost ₹1,800 where the right drawer cost ₹4. Six unused schemas cost 60,120,000 tokens a day. An unread manifest cost eighteen months of every customer's data being readable by one carrier — and nothing, because nobody looked. A wall with no sticker cost two days of guessing. The warehouse on every call cost 362,460 tokens where the catalogue cost 4,205. All of it against a ₹6 ceiling per shipment and promises of 2 s / 4 s / 5 s. Meera signed the sentences, not the adjectives.

## Same word, two worlds

Fifteen words the exam uses in two senses. The inline ► flags at each ambush are collected here; the last column is the Walk page of the card that owns each sense.

| Word            | World 1                                                                                   | World 2                                                   | The tell                                                       | Cards         |
|-----------------|-------------------------------------------------------------------------------------------|-----------------------------------------------------------|----------------------------------------------------------------|---------------|
| token           | LLM: a word-piece, counted and billed                                                     | auth: a pass a caller shows                               | counted / window / bill → LLM; issued / expires / scope → auth | p. 5 · p. 38  |
| index           | book: the alphabetical register (keyword)                                                 | vector: a store of positions answering 'what is nearest?' | 'BM25 / exact' → book; 'embedding / similarity' → vector       | p. 27 · p. 18 |
| prompt          | system prompt: standing instructions every call                                           | MCP prompt: a reusable template a server offers           | 'instruct the model' → system; 'server exposes' → MCP          | p. 6 · p. 71  |
| resource        | MCP: a document offered by address (sop://214)                                            | budget: people, money, compute                            | a URI / 'read' → MCP; a plan / headcount → budget              | p. 71         |
| context         | business: the situation at Kaveri                                                         | window: everything the model sees on one call, in tokens  | 'tokens / fits / budget' → window                              | p. 5          |
| key             | API key: a permanent credential                                                           | dict key / cache key: a lookup handle                     | 'rotate / secret' → API key; 'lookup / prefix' → the others    | p. 40         |
| scope           | auth: the list of doors a token may open                                                  | Python: where a variable name is visible                  | 'token / permission / read all' → auth                         | p. 38         |
| span            | trace: one timed stage of one request                                                     | HTML: an inline tag                                       | 'duration / parent / trace' → trace                            | p. 50         |
| cache           | prompt cache: the provider remembers your prefix; reads ~10 %                             | browser / CDN: a stored copy near the user                | 'prefix / minimum tokens / cache_control' → prompt cache       | p. 6          |
| client / server | MCP roles: client = model side; server = offers primitives (may be a local child process) | web: browser and remote host                              | 'stdio / primitives / mcp__' → MCP roles                       | p. 71         |
| transport       | MCP: the road messages travel (stdio · Streamable HTTP)                                   | logistics: trucks and carriers                            | 'stdio / HTTP / remote server' → MCP                           | p. 45         |
| primitive       | MCP: a basic offering (tool · resource · prompt)                                          | programming: a basic data type                            | 'server offers' → MCP                                          | p. 71         |

| Word  | World 1                                                       | World 2                                                          | The tell                                                       | Cards         |
|-------|---------------------------------------------------------------|------------------------------------------------------------------|----------------------------------------------------------------|---------------|
| hold  | customs: a consignment stopped at the yard                    | API: a paused or queued request                                  | 'officer / bonded yard / e-way bill' → customs                 | story         |
| turn  | conversation: one user message and one reply                  | round-trip: one internal fetch inside answering one message      | 'fetch / discover / look up' → round-trip                      | p. 77         |
| label | shelving: the contextual sentence or version field on a chunk | metrics: a name=value tag on a gauge (each combination a series) | 'chunk / version' → shelving; 'metric / cardinality' → metrics | p. 21 · p. 55 |

# Plain → Exam phrasebook

How the Walk says it → how a CCAR-P stem says it. Read the left column aloud; if the right column does not come back within a breath, that card wants another read.

| Plain (the Walk)                                     | Exam (the stem)                                                            | Obj.      |
|------------------------------------------------------|----------------------------------------------------------------------------|-----------|
| who's allowed to press the refund button             | authorization scope on process_refund                                      | 3.2       |
| does the badge say who they are?                     | authentication (authN)                                                     | 3.2       |
| does the badge say which rooms?                      | authorization (authZ) / scope                                              | 3.2       |
| the pass that opens every door                       | wildcard / over-permissive scope (OP)                                      | 3.2       |
| give each caller only the doors its job needs        | principle of least privilege                                               | 3.2       |
| the forklift's own badge                             | service account / machine identity                                         | 3.2       |
| a day pass issued after an ID check                  | OAuth access token with scopes                                             | 3.2       |
| HR vouching for you at the gate                      | SSO / federated identity (IdP)                                             | 3.2       |
| write down every opening and every refusal           | audit log including denied requests                                        | 3.2       |
| a sign is not a lock                                 | code-level enforcement beats prompt instruction (config beats instruction) | 3.2       |
| same building vs the highway                         | stdio vs Streamable HTTP transport                                         | 3.2 / 3.7 |
| give the new hire a filing cabinet and a clerk       | retrieval-augmented generation (RAG)                                       | 3.5       |
| just read it all to the model                        | whole corpus in context (below ~200k tokens)                               | 3.5 / 3.8 |
| cut where the document changes subject               | heading-aware / structural chunking                                        | 3.5       |
| cut every N words                                    | fixed-size chunking                                                        | 3.5       |
| repeat the last sentence at each cut                 | chunk overlap                                                              | 3.5       |
| file by what it's about                              | embedding + vector index (semantic search)                                 | 3.5       |
| same direction on the map                            | cosine similarity / nearest neighbour                                      | 3.5       |
| write on the box what it is and where it came from   | contextual retrieval (contextual embeddings + contextual BM25)             | 3.5       |
| the supervisor re-sorts the shortlist                | reranking (retrieve ~150 → top-20)                                         | 3.5       |
| how many boxes the clerk brings back                 | top_k                                                                      | 3.5       |
| did we catch it / did we bury it                     | recall / precision (retrieval sense)                                       | 3.5       |
| the picker came back without the armrest             | retrieval failure rate (5.7 → 1.9 %)                                       | 3.5       |
| re-do the shelves every Sunday                       | scheduled re-index; version metadata                                       | 3.5       |
| a stale box keeps being fetched                      | context poisoning                                                          | 3.5       |
| check the register                                   | keyword / BM25 (lexical) search                                            | 3.6       |
| ask the old foreman                                  | semantic (dense) search                                                    | 3.6       |
| ask both, trust where they agree                     | hybrid search with score fusion                                            | 3.6       |
| read the shape of the question first                 | retrieval strategy by data shape × query pattern (A3)                      | 3.6       |
| 'how many' is not a paragraph question               | aggregate query, not RAG (RAG cannot count)                                | 3.6       |
| 'where is KF-2481' is not a search                   | structured lookup by key                                                   | 3.6       |
| pull the tracking number out and write it on the box | metadata extraction + filtered retrieval                                   | 3.6       |
| the diary, the sticker, the gauge                    | structured logs, traces (trace id / spans), metrics                        | 3.4       |
| the average melted the tail                          | p95 / p99 tail latency vs mean                                             | 3.4       |
| a promise about the slowest one in twenty            | SLA defined on p95 over a window                                           | 3.4       |
| a gauge per shipment melts the store                 | high-cardinality labels / metric explosion                                 | 3.4       |
| the bell that rang all morning                       | alert fatigue; alert on percentiles over windows                           | 3.4       |
| the screen is a window, not a sensor                 | dashboards render metrics; they do not create data                         | 3.4       |
| an incident walks down the stairs                    | alert → dashboard → metric → trace → log                                   | 3.4       |

| Plain (the Walk)                                        | Exam (the stem)                                                                     | Obj. |
|---------------------------------------------------------|-------------------------------------------------------------------------------------|------|
| weight on the belt                                      | tool-schema token cost per call                                                     | 3.1  |
| two wrenches that look alike                            | ambiguous / overlapping tool definitions                                            | 3.1  |
| trim by logs AND consequence                            | remove unused tools; keep low-frequency high-consequence ones                       | 3.1  |
| seconds or rupees                                       | accuracy-latency-cost trade-off                                                     | 3.3  |
| constraint · dial · direction · number                  | configuration justification against an SLA / cost ceiling                           | 3.3  |
| the standard dock plate vs a welded ramp                | MCP server vs direct API/CLI integration                                            | 3.7  |
| the other truck has its own driver with his own errands | agent-to-agent integration                                                          | 3.7  |
| every letter carries its own full address               | stateless MCP (2026-07-28): <code>_meta</code> on every request                     | 3.7  |
| hand the driver the catalogue, not the warehouse        | progressive disclosure / tool search with <code>defer_loading</code>                | 3.8  |
| the shelf by the gate                                   | small, stable, always-used set → monolithic + prompt cache                          | 3.8  |
| sign the pages once                                     | prompt caching (write 1.25×, read ~0.1×; minimum varies by model)                   | 3.8  |
| sweep the desk at lunch, keep the last three            | server-side context editing ( <code>clear_tool_uses</code> , trigger ~100k, keep 3) | 3.8  |

# Wrong beliefs, corrected — the index

Twelve crimson boxes, one page each. Before the exam, read only the left column and say the correction out loud; open the page only where you hesitate.

| I used to think...                                       | Page  |
|----------------------------------------------------------|-------|
| embeddings understand meaning like a person does         | p. 18 |
| more chunks in the prompt = better answers               | p. 22 |
| RAG replaced keyword search                              | p. 30 |
| one good retrieval pipeline should answer every question | p. 34 |
| logged in securely = secure                              | p. 38 |
| a firm system prompt is a control                        | p. 43 |
| the average is fine, so users are fine                   | p. 53 |
| adding a dashboard fixes observability                   | p. 58 |
| more tools make a more capable agent                     | p. 63 |
| the cheapest model is the right default                  | p. 68 |
| an MCP server is always the professional choice          | p. 73 |
| progressive discovery is always leaner                   | p. 81 |

## The reverse map — Loading Dock → Walk page

From Day A onward the Walk is the fog dictionary: when a Dock sentence is fog, find its chapter, glossary entry, exhibit or Atlas page here and go to the Walk page named.

| Loading Dock                                                                               | Walk station (page) | Climbs (pages)                                                                           | Labs · pictures                                            |
|--------------------------------------------------------------------------------------------|---------------------|------------------------------------------------------------------------------------------|------------------------------------------------------------|
| ch. 3.1 capability bloat · E2                                                              | Station 5 (p. 61)   | Nine wrenches (61)                                                                       | P8 (63) · W14 (62)                                         |
| ch. 3.2 authN / authZ · E3 · A5                                                            | Station 3 (p. 36)   | Who you are (36) · Four passes (39) · The manifest (42) · Same building (45)             | P7 (44) · W8 (37) · W9 (40)                                |
| ch. 3.3 trade-offs · the SLAs · ₹6                                                         | Station 6 (p. 65)   | Two currencies (65)                                                                      | P11 (68) · W15 (66)                                        |
| ch. 3.4 observability · E7 · A6                                                            | Station 4 (p. 48)   | Three kinds of watching (48) · The afternoon (51) · The wall (54) · The stairs (57)      | P5 (54) · P6 (51) · P12 (56) · W10-W13 (49-58)             |
| ch. 3.5 RAG · E4 · E5 · A1 · A2                                                            | Station 1 (p. 9)    | New hire (9) · The sofa (12) · Meaning map (16) · Label in the box (20) · Re-shelve (23) | P1 (15) · P2 (19) · W2-W5 (10-17)                          |
| ch. 3.6 retrieval by shape · E5 · E6 · A3                                                  | Station 2 (p. 26)   | Register and foreman (26) · Both (29) · Fluent wrong number (32)                         | P3 (31) · P4 (34) · W6 (27) · W7 (32)                      |
| ch. 3.7 MCP · E1 · A4 · DRIFT 2026-07-28                                                   | Station 7 (p. 70)   | Three pieces of furniture (70)                                                           | P10 (73) · DRIFT (72)                                      |
| ch. 3.8 progressive vs monolithic · E8 · A7 · DRIFT context editing · DRIFT cache minimums | Station 8 (p. 75)   | The catalogue (75) · The shelf and the desk (78)                                         | P9 (81) · W16 (79) · W17 (76) · ERRATUM (75) · DRIFTS (80) |
| the story page · miss families · the five laws                                             | Chapter 0 (p. 4)    | the nervous system · how a stem is built · seven base cards                              | W1 (4) · laws, slowly (back matter)                        |
| Atlas A1 pipeline                                                                          | W5 zoom 3 (p. 11)   | Station 1, Climb 1                                                                       | —                                                          |
| Atlas A2 chunking picker                                                                   | W3 (p. 13)          | Station 1, Climb 2                                                                       | P1                                                         |
| Atlas A3 retrieval matrix                                                                  | W7 (p. 32)          | Station 2, Climb 3                                                                       | P4                                                         |
| Atlas A4 integration decision                                                              | P10 (p. 73)         | Station 7                                                                                | —                                                          |

| Loading Dock                       | Walk station (page)                         | Climbs (pages)        | Labs · pictures |
|------------------------------------|---------------------------------------------|-----------------------|-----------------|
| Atlas A5 auth boundary map         | W8 + W9 (pp. 37, 40)                        | Station 3, Climbs 1–3 | P7              |
| Atlas A6 observability stack       | W13 (p. 58) · W11 (p. 52) · W12 (p. 55)     | Station 4             | P5 · P6 · P12   |
| Atlas A7 progressive vs monolithic | W16 + W17 (pp. 79, 76) — corrected to 4,205 | Station 8             | P9 · ERRATUM    |
| E1–E8 (typed on Days A/B)          | Exhibit companions (pp. 83–84)              | one quarter-page each | no code here    |

## Glossary → card

| Loading Dock glossary entry                           | Walk card |
|-------------------------------------------------------|-----------|
| chunking (fixed / semantic / heading-aware / overlap) | p. 13     |
| embedding                                             | p. 17     |
| vector index                                          | p. 18     |
| BM25 / keyword                                        | p. 27     |
| hybrid search                                         | p. 29     |
| reranking                                             | p. 21     |
| top_k                                                 | p. 18     |
| recall vs precision (retrieval sense)                 | p. 24     |
| context poisoning                                     | p. 24     |
| authN vs authZ                                        | p. 37     |
| scope                                                 | p. 38     |
| least privilege                                       | p. 42     |
| token (auth sense)                                    | p. 38     |
| service account                                       | p. 40     |
| audit log                                             | p. 43     |
| trace / span                                          | p. 50     |
| p50 / p95 / p99 latency                               | p. 52     |
| cardinality                                           | p. 55     |
| MCP server / resource / prompt                        | p. 71     |
| transport (stdio vs HTTP)                             | p. 45     |
| progressive disclosure                                | p. 76     |
| monolithic context                                    | p. 77     |

# The laws, slowly

Five D3 laws, each in plain words with one example of it being broken. The two laws of laws from CCAR-F stand behind all five: **config beats instruction** · **valid is not true**. D1's laws remain in force; D1 Law 3 (budgets and loops) is cited wherever a dial is a budget.

## D3 LAW 1 · Every tool you add is a door; know who holds the key.

**In plain words:** A capability is a way in. For each one, say who may use it, how far it opens, and where each use is written down. If you cannot, it is a hole with a name.

**Broken looks like:** **the carrier token: genuine, rotated, eighteen months old, 'all bays', and nobody could say who had read what.**

## D3 LAW 2 · Chunk along meaning, not length.

**In plain words:** Cut documents where they change subject — at the author's joints — so each piece holds one complete idea a reader could act on alone.

**Broken looks like:** **every 200 words through SOP 214, so §4's procedure straddled two chunks and Padma got the cancellation steps, fluently.**

## D3 LAW 3 · Retrieval follows the shape of the data, not the fashion.

**In plain words:** Read what the data looks like and what people actually ask before choosing a search: ids want a lookup, counts want a query, codes want the register, symptoms want the foreman, most desks want both.

**Broken looks like:** **'we moved everything to vector search because that is what RAG means' — and EWB-01 searches started returning EWB-02, and 'how many late this week' came back as last month's 312.**

## D3 LAW 4 · You cannot fix what you cannot see.

**In plain words:** Write down what happened (structured logs), join it per request (a trace id on every line), summarise it over time by bounded categories (metrics), show it (dashboards), and ring a bell rarely and on purpose (alerts). Then an incident walks down the stairs instead of into a guess.

**Broken looks like:** **three diaries that could not be joined, a theory built from timestamps by eye, and an index rebuild at 14:09 that nobody connected to the slowness until Thursday.**

## D3 LAW 5 · Send the map first, the territory on demand.

**In plain words:** Put a short catalogue in the window and fetch the full thing only when pointed at — except for the small, stable, always-used set, which rides in the window every time and is cached.

**Broken looks like:** **all 400 SOPs and nine schemas on every call 'so the model never lacks context': 362,460 tokens before the question, against a 4,205-token catalogue.**

## DRIFT AND ERRATUM — WHAT TO CARRY INTO THE ROOM

**MCP 2026-07-28** (p. 72): stateless, \_meta on every request, server/discover, sampling/logging/roots deprecated, stdio + Streamable HTTP, HTTP+SSE gone, elicitation by input\_required + retry. **Contextual retrieval** (p. 22): 5.7 → 3.7 → 2.9 → 1.9 % (35 / 49 / 67 %); ~150 → top-20; skip RAG under ~200k. **Context editing** (p. 80): clear\_tool\_uses\_20250919, trigger ~100k, keep 3, exclude\_tools, clear\_at\_least; clear\_thinking\_20251015; memory\_20250818. **Cache minimums — UPDATED** (pp. 3, 80): 512–4,096 by model; Haiku 4.5 = 4,096; reads ~90 % off; under the minimum = silent zero. **CANON ERRATUM** (pp. 75, 77, 79, 84, 91): E8 progressive bill = 4,205, gap ≈ 86× (the Dock prints 4,625 / 78×).

# Word-card index — 76 cards

Every card in the book, in page order. The seven Chapter 0 cards are the base words; everything after them was introduced at its first use. If a term in a Dock sentence is fog, this is the first place to look.

| Card                                           | p. | Card                                                  | p. |
|------------------------------------------------|----|-------------------------------------------------------|----|
| token (LLM sense)                              | 5  | context window                                        | 5  |
| tool                                           | 5  | system prompt                                         | 6  |
| latency                                        | 6  | MCP (Model Context Protocol)                          | 6  |
| prompt cache                                   | 6  | RAG (retrieval-augmented generation)                  | 10 |
| retrieval                                      | 10 | pipeline (RAG sense)                                  | 11 |
| chunking                                       | 13 | embedding                                             | 17 |
| cosine similarity                              | 17 | nearest neighbour                                     | 18 |
| vector index                                   | 18 | top_k                                                 | 18 |
| contextual retrieval                           | 21 | reranking                                             | 21 |
| failure rate (retrieval)                       | 21 | re-index (re-shelve)                                  | 23 |
| metadata (version label)                       | 24 | context poisoning                                     | 24 |
| recall vs precision (retrieval sense)          | 24 | keyword search (BM25)                                 | 27 |
| semantic search                                | 27 | query pattern                                         | 28 |
| data shape                                     | 28 | hybrid search                                         | 29 |
| score fusion                                   | 30 | structured lookup                                     | 33 |
| aggregate query                                | 33 | metadata filtering (free text with fields written in) | 33 |
| authentication (authN)                         | 37 | authorization (authZ)                                 | 37 |
| token (auth sense)                             | 38 | scope                                                 | 38 |
| API key                                        | 40 | OAuth (token issued after an identity check)          | 40 |
| service account                                | 40 | SSO (single sign-on)                                  | 41 |
| mTLS (mutual TLS)                              | 41 | least privilege                                       | 42 |
| wildcard (in a scope)                          | 43 | audit log (the ledger)                                | 43 |
| transport (MCP sense)                          | 45 | stdio (standard input/output)                         | 46 |
| Streamable HTTP                                | 46 | observability                                         | 49 |
| structured log                                 | 49 | trace / span / trace id                               | 50 |
| metric                                         | 50 | percentile latency (p50 / p95 / p99)                  | 52 |
| SLA (service-level agreement)                  | 53 | series (time series) and label                        | 55 |
| cardinality                                    | 55 | alert                                                 | 56 |
| dashboard                                      | 58 | tool schema                                           | 62 |
| capability bloat                               | 62 | lookalike tools                                       | 62 |
| accuracy-latency trade-off                     | 66 | constraint (the promise)                              | 66 |
| dial (configuration parameter)                 | 67 | justification sentence                                | 67 |
| primitive (MCP)                                | 71 | MCP tool / resource / prompt                          | 71 |
| client / server (MCP roles)                    | 71 | JSON-RPC                                              | 71 |
| stateless (MCP 2026-07-28)                     | 72 | elicitation                                           | 72 |
| progressive disclosure (progressive discovery) | 76 | monolithic context                                    | 77 |
| tool search / deferred tools                   | 77 | round-trip (extra fetch)                              | 77 |
| cache minimum                                  | 79 | context editing (server-side)                         | 80 |

**End of the walk.** Now read the Loading Dock; it should read like a summary. Star anything still foggy — the reverse map names the page. Prepared by Ravi Ranjan · CCAO-F, CCDV-F & CCAR-F